PHINMA UNIVERSITY OF ILOILO
College of Information Technology Education

TrustFile: A File Security Platform that scans and protects files before access

Joshua C. Taño, Reymart Aportadera, Eduard J. Corcino,Rencel B. Dejaro, Ma. Robella Beatrice M. Lemeric

College of Information Technology Education, PHINMA University of Iloilo, Philippines

Abstract

The rapid proliferation of file based malware poses significant risks to digital assets across organizations. Traditional signature based detection mechanisms struggle against zero day threats, while commercial antivirus solutions often function as non-transparent "black boxes," offering little educational insight to end users. This paper presents TrustFile, a web based file security platform designed to evaluate file safety prior to user execution. TrustFile integrates a comprehensive, multi stage static analysis pipeline encompassing cryptographic file hashing to identify known digital fingerprints, Shannon entropy calculation to detect encrypted or packed content, magic byte format verification to prevent file extension spoofing, embedded script detection to surface hidden executable code, and file size anomaly verification. This pipeline works alongside a rule based heuristic classification system, cloud threat intelligence lookups via the VirusTotal API, and an explainable artificial intelligence (AI) analysis layer leveraging the Llama 3.1 70B model via the NVIDIA NIM API. By categorizing submitted files into a five tier risk framework (Benign, Low Risk, Suspicious, High Risk, and Malicious) and delivering tailored, plain language diagnostic explanations, TrustFile bridges the critical gap between automated threat detection and user security awareness. The system evaluates files before they are opened, transforming every scan into a practical cybersecurity learning opportunity. Experimental evaluation demonstrates the platform's efficacy in identifying obfuscated and disguised threats while maintaining low false positive rates across diverse file formats.

Index Terms: Cybersecurity, Static Malware Analysis, Explainable AI, Shannon Entropy, Heuristic Classification, Threat Intelligence, File Security.

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

The rapid growth of information technology has changed the way people, businesses, schools, and government offices create, store, and share information. Digital files are now part of almost every daily task, used for communication, record keeping, collaboration, and storage. Common formats such as PDF, DOCX, XLSX, PPTX, ZIP/RAR archives, images, audio, video, and executable files move constantly through email, cloud storage, USB drives, websites, and messaging platforms. This convenience has boosted productivity across sectors, but it has also opened new doors for cybercriminals to distribute malicious files disguised as ordinary documents. Organizations receive large volumes of files every day from clients, employees, suppliers, and partners, many containing confidential records, financial data, contracts, and other sensitive information. This constant exchange makes offices, agencies, and businesses attractive targets, since attackers can hide malicious code inside files that look completely legitimate, leaving users with little way to tell a safe file from a dangerous one just by looking at it.

The growing dependence on digital files has also changed the way organizations manage their day-to-day operations. Processes once completed on paper are now almost entirely digital, allowing information to be accessed, updated, and transmitted within seconds. Educational institutions distribute learning materials through learning management systems, businesses exchange financial records through cloud-based platforms, healthcare providers maintain electronic medical records, and government agencies process applications and official documents electronically. This shift has improved efficiency, reduced operational costs, and strengthened collaboration, but it has also increased the number of digital assets that require protection. Employees routinely download reports, invoices, spreadsheets, presentations, compressed archives, and software installers from internal and external sources. Many of these files come from trusted senders, but others arrive from unfamiliar websites, third-party vendors, or unknown email addresses, and users often cannot tell a legitimate file from one that has been deliberately modified to carry malicious code. Even experienced computer users may unknowingly interact with infected files, because many threats are specifically designed to imitate authentic documents.

Another important concern is that organizations increasingly rely on interconnected digital environments. Cloud computing, remote work arrangements, collaborative online platforms, mobile devices, and Internet of Things (IoT) devices allow employees to reach files from almost anywhere. These technologies provide real operational advantages, but they also widen the attack surface available to cybercriminals. A single infected file uploaded to a shared storage service or collaborative workspace can spread to many users, departments, or even partner organizations, and a single compromised device may give attackers an opportunity to move laterally across connected systems. As a result, the security of an individual file has become directly connected to the overall

security of the organization, which makes it important to detect malicious files before they can interact with other digital assets.

The increasing sophistication of cyberattacks has shown that protecting digital systems requires more than securing networks and servers. Individual files have become one of the common ways malicious software enters organizational environments. Every downloaded attachment, shared document, archived file, or executable application is a potential risk if it is not verified before use. For this reason, organizations are placing greater emphasis on preventive measures that can identify suspicious files before they are opened, which reduces the likelihood of malware infections, unauthorized access, and costly security incidents. This reflects a wider shift in cybersecurity from simple threat removal toward proactive risk management. Rather than responding only after malicious software has infected a system, professionals increasingly emphasize approaches such as file verification, threat intelligence integration, static analysis, and heuristic evaluation, which allow organizations to reduce risk during the earliest stages of a potential attack.

The malware threat itself has grown far more advanced over time. What began as simple viruses has evolved into a wide range of threats, including worms, Trojan horses, ransomware, spyware, adware, rootkits, and keyloggers, each with its own way of infecting a system. Viruses spread when infected programs are run; worms move across networks on their own; Trojans hide malicious behavior inside what looks like a normal application; ransomware locks up files until a ransom is paid; and spyware quietly collects passwords and financial information in the background. The damage these attacks cause is no longer only technical; it is also financial and reputational. According to IBM's 2024 Cost of a Data Breach Report, the global average cost of a data breach reached US$4.88 million in 2024, a 10% increase from the year before and the largest yearly increase since the pandemic. IBM attributed much of that increase to business disruption and post-breach response costs, not only the initial intrusion itself.

The continuous evolution of malware has also been driven by the rapid advancement of technology and the wide availability of cyberattack tools. In the past, developing malicious software required extensive technical expertise. Today, cybercriminals can obtain ready-made malware kits, exploit frameworks, and attack tools through underground marketplaces and cybercrime-as-a-service platforms. These resources lower the barrier to entry and allow individuals with limited technical skills to launch cyberattacks. As a result, malware campaigns have become more frequent and more organized, and they pose a growing challenge to organizations of all sizes.

Cybercriminals also continually adapt their techniques to evade security controls. As antivirus software and other security technologies improve, attackers develop new ways around them through obfuscation, encryption, code mutation, and social engineering. This ongoing cycle of attack and defense means that organizations must constantly update their protective measures, and it shows that no single detection method is enough on its own. It also supports the need for complementary approaches that examine the characteristics of a file itself instead of relying only on databases of previously identified threats.

Beyond financial losses, malware attacks can have lasting consequences for an organization's reputation and public trust. Customers, clients, students, and business partners expect organizations to protect the information entrusted to them. When confidential data is compromised through a malware infection, affected organizations may experience reduced customer confidence, negative public perception, legal liabilities, and increased regulatory scrutiny. Recovering from such incidents often requires not only technical remediation but also communication efforts, policy revisions, employee retraining, and investment in stronger cybersecurity infrastructure. These indirect consequences may continue long after the malware itself has been removed, which shows that the impact of a cybersecurity incident extends far beyond the immediate technical damage.

Attackers rarely need to break through technical defenses when they can simply persuade a person to open the wrong file. Phishing emails disguised as invoices, job offers, shipping notices, or official government correspondence remain one of the most common ways malware gets in, relying on urgency, curiosity, or authority to lower a user's guard. This problem grew noticeably during the COVID-19 pandemic, when fear, urgency, and a dispersed, distracted workforce gave attackers favorable conditions to scale up their campaigns. The pandemic did not create the problem of file-based malware, but it showed how quickly the problem can spread under the right conditions. Security reports consistently find that a human element is involved in a large share of breaches, which means that even organizations with advanced security technologies remain vulnerable when a user opens a malicious attachment or downloads an infected file from an untrusted source.

Commercial antivirus software remains the most common line of defense, but it has a real limitation: it usually functions as a "black box." A file is flagged, a threat name appears, and that is often the extent of what the user is told, with little explanation of why the file was flagged or what made it dangerous. Much of this comes down to how traditional antivirus tools work. Most rely on signature-based detection, which compares a file against a database of known malware signatures. This approach is fast and reliable for threats that have already been identified and catalogued, but it is structurally unable to catch zero-day malware, or threats that are new, modified, or intentionally disguised to avoid matching any known signature [1], [2]. Malware authors exploit this gap deliberately, using polymorphic and metamorphic techniques that alter a file's code just enough to slip past signature matching [1].

This opacity also has a quieter cost beyond the individual scan. A user who has only ever been told that a file is safe or unsafe learns nothing about why, and is no better equipped to evaluate the next suspicious file that slips past the software. Over time, users become permanently dependent on the verdict of their security tool instead of building any independent judgment of their own.

Modern malware also goes out of its way to look harmless. Attackers use file extension spoofing, double extensions, packing, encryption, and steganography to bury malicious code inside what appears to be an ordinary document or media file. An executable can be renamed to resemble a PDF, and because some operating systems hide file extensions by default, the disguise can go unnoticed [6]. A malicious script can also be tucked inside an Office document or an image file without any visible sign of tampering. One of the more established techniques for catching this kind of disguise is entropy analysis, which measures how "random" the data inside

a file is, since encrypted or packed content tends to have much higher entropy than typical plain text or standard file structures. Lyda and Hamrock [3] showed, in their foundational work published in IEEE Security & Privacy, that entropy analysis lets analysts flag packed or encrypted malware quickly, without needing to fully reverse engineer the file first, a principle that later studies have built on using statistical and machine learning models across different file types [4].

Files that seem harmless can also carry active content. Office documents may contain macros that run code once they are enabled, PDF files can embed scripts, and compressed archives can hide executable files inside folders that appear ordinary. In each case the file looks like a normal document to the user, and the harmful part is only revealed when the file is opened or its contents are extracted. This is why examining a file before it is opened is so valuable: the check happens while the file is still inert, before any hidden code has a chance to run.

Heuristic analysis was developed to close part of the signature-detection gap by examining a file's structure and content for suspicious patterns instead of requiring an exact match. Because it evaluates characteristics and not identity, it can flag files that no signature covers yet. It introduces a different tradeoff, however. Heuristic systems are inherently prone to false positives, sometimes flagging legitimate software as malicious because its structure resembles patterns associated with threats [7]. Too many false alarms erode user trust, while rules loose enough to avoid them can let genuinely harmful files through. Attackers also work around heuristics directly: packing or encrypting a payload raises a file's entropy, which can resemble the entropy of legitimate compressed files, and padding a file with ordinary content can dilute that signal. This back-and-forth between evasion and detection is why relying on any single check in isolation is no longer sufficient, and why an approach that weighs multiple indicators together, such as file hashes, entropy, script content, size, and file type, has a real advantage over any one method alone.

File size is one of those indicators that is easy to overlook but often quietly revealing. Malware is sometimes appended to an otherwise legitimate file, inflating it beyond what is normal for its format; conversely, an unusually small file can indicate a stub loader, a lightweight file whose only job is to download the actual malicious payload after execution. Neither pattern is something a user would notice by eye, and building a system that can flag it reliably requires accounting for the fact that "normal" file size varies significantly across formats, compression levels, and embedded content, which makes reliable detection a challenge in itself.

Two other sources of evidence complement these checks. The first is the cryptographic hash, a fixed-length digital fingerprint calculated from a file's contents. Identical files produce identical hashes, so a file that has already been analyzed can be recognized quickly, and even a one-byte change to a file produces a completely different hash. The second is external threat intelligence. Services such as VirusTotal combine the verdicts of many independent antivirus engines into one report, giving a system access to detection knowledge far broader than anything a single locally built tool could hold. Each source has blind spots of its own, which is why combining local analysis with external intelligence gives a more reliable picture than either one alone.

What all of this points to is a gap that goes beyond any single piece of software: users are rarely equipped to make an informed judgment about a file's safety on their own. Cybersecurity training in schools and workplaces tends to focus on password hygiene, phishing awareness, and keeping software updated, but rarely touches on practical, file-level analysis concepts like hashing, entropy, or file type verification. As a result, most users are left entirely dependent on whatever their antivirus tool decides to tell them, with no way to independently reason about a file before deciding whether to open it.

This is the gap that TrustFile, the system developed in this study, was built to close. TrustFile scans a file before the user opens it. A user uploads the file to the web-based platform, and the system examines its contents without running it, using static analysis techniques: cryptographic hashing to produce the file's digital fingerprint, entropy analysis to flag encrypted or packed content, script detection to surface embedded executable code, file size checks to catch abnormal sizes, and file type verification to confirm that a file's extension matches its internal structure. The result from an external malware intelligence service, VirusTotal, is added to this evidence. A rule-based heuristic analysis then weighs all of these indicators together and assigns the file one of five threat levels: Benign, Low Risk, Suspicious, High Risk, or Malicious. The threat level is decided only by this heuristic analysis.

Once the threat level has been assigned, an AI layer explains it. The AI does not decide whether a file is safe or dangerous and does not change the classification. Its only role is to take the finished result and the indicators behind it and describe them in plain language: which indicators were found, what they mean, why they matter for that file, and what risk they carry. Keeping the decision in a rule-based engine and the explanation in the AI means the same file receives the same classification every time, while technical and non-technical users alike can still understand the reasons behind it.

The five threat levels give users a graded result instead of a simple safe-or-unsafe verdict. A Benign result means that no significant indicators of risk were found. Low Risk means that only minor indicators were found. Suspicious means that notable indicators were found and the user should be cautious. High Risk means that multiple strong indicators were found, and Malicious means that the evidence indicates the file is harmful and should not be opened. A graded scale lets users respond in proportion to the risk, for example by opening a Benign file, treating a Suspicious file with caution, and avoiding a High Risk or Malicious file altogether.

Because every scan comes with an explanation, each scan is also a chance for a user to learn what makes a file dangerous, so TrustFile works not only as a detection tool but also as a quiet form of cybersecurity education. It is aimed at the environments where the problem is most acute: offices, government agencies, schools, and businesses that handle a constant stream of incoming files. By explaining its findings instead of concealing them, TrustFile addresses both the technical gap and the educational gap left open by conventional antivirus solutions, and it gives users the information they need to make an informed decision before a file is ever opened.

1.2 Statement of the Problem

Digital files have become one of the most common ways for malware to enter an organization, yet most users cannot tell whether a file is safe before they open it. A file's name, icon, and extension reveal very little about what it actually contains, and attackers exploit this through extension spoofing, double extensions, packing, encryption, and hidden executable code. Offices, schools, government agencies, and private businesses that receive hundreds of files every day are especially exposed, since a single careless click on an attachment can lead to data theft, ransomware, or a system-wide infection.

Existing tools only partly solve this problem. Commercial antivirus software usually works as a "black box" that reports a threat name without explaining why a file was flagged. Signature-based detection cannot recognize malware it has never seen, while heuristic methods can catch new threats but may produce false positives that erode user trust. At the same time, most users have little practical knowledge of how to judge a file, so they keep opening unverified files and ignoring warnings they do not understand.

Specifically, this study aims to address the following problems:

1. Users cannot easily identify whether a file contains malware before opening it. Visible cues such as file names, icons, extensions, and a sender's reputation do not reliably show what a file contains, and attackers use disguised extensions and hidden code to take advantage of this.

2. Many systems rely on basic detection methods and may miss new or hidden threats. Signature-based tools leave a window of exposure for new malware, and obfuscation and packing techniques are designed to avoid detection.
3. There is a lack of systems that explain how malware is detected. Most security software returns only a "threat detected" or "file clean" result, without showing which indicators were checked or what they mean.
4. Users have limited knowledge about identifying malicious files. Few users understand the basic concepts that security professionals use to judge a file, and this shows in risky habits such as opening unverified files.

To address these problems, the researchers developed TrustFile, a web-based file security platform that evaluates files before a user opens them. It combines static analysis, rule-based heuristic classification, external malware intelligence, and AI-generated plain-language explanations.

1.4 Conceptual Framework

This study adopts the Input–Process–Output (IPO) model to explain how TrustFile transforms an uploaded file into a verified threat classification and a plain-language explanation. The framework is built around the study's five specific objectives: a web-based platform that scans files before access, a rule-based heuristic classification with AI-generated explanations, entropy analysis for concealed content, connection to external malware intelligence, and clear presentation of results. Prior work shows that no single technique is sufficient. Signature-based methods cannot recognize threats that have not yet been catalogued [1], [2], while heuristic and

structural analysis can surface suspicious characteristics in files that match no known signature [4], [7]. TrustFile therefore arranges several complementary checks into one pipeline and keeps the decision about a file's risk separate from the explanation of that decision.

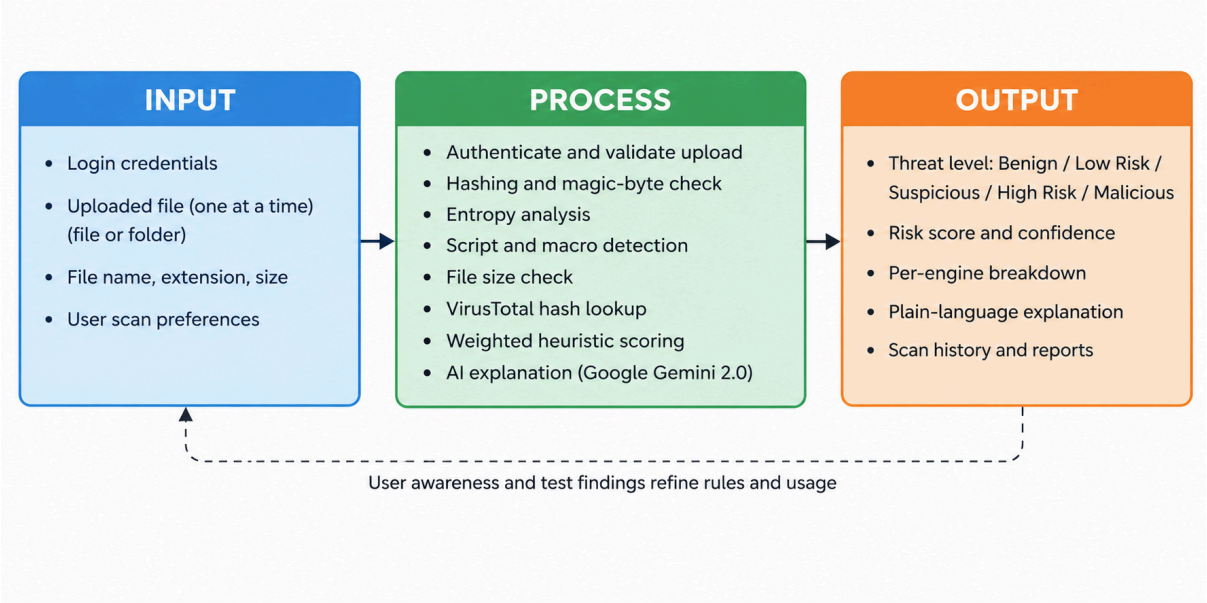


Figure 1.1. IPO Conceptual Framework of TrustFile

Figure 1.1 illustrates the relationship among the inputs, processes, and outputs of TrustFile. The inputs consist of the user's account credentials, the file submitted for scanning, and the file's basic attributes such as name, extension, and size. The system processes these through validation, local static analysis, external threat intelligence lookup, heuristic classification, and AI-assisted explanation. The outputs are the threat classification, the risk score, the per-engine findings, the plain-language explanation, and the stored scan record. The framework shows the functional flow of the proposed system. It does not claim that the system executes files or independently predicts future malware behavior.

Components of the Framework

Input. The input component includes everything the system needs before analysis can begin. A registered user signs in and submits either a single file or a folder through the upload page in the browser. The files may be documents, archives, executables, images, or other formats commonly exchanged in offices and schools. The system also reads each file's name, extension, declared type, and size. The quality of the final result depends on receiving the files intact, since every later stage examines the actual bytes rather than the label. This reflects the problem identified in Chapter 1, that names, icons, and extensions reveal very little about what a file contains [6].

Process. The process component represents the operations TrustFile performs on each submitted file, and none of them execute it. The application first authenticates the user and validates the upload, treating every file as untrusted input. Local static analysis then runs five core techniques: cryptographic hashing, magic-byte file type verification, Shannon entropy calculation with safeguards against false positives on legitimately compressed formats, embedded script and macro detection, and file size anomaly checking. In parallel, the system sends the

file's hash to the VirusTotal API, stores the response in a cache, and continues the scan if the external service is slow or unavailable. The heuristic engine then combines every signal through weighted multi-signal scoring with confidence tiers and maps the result to one of five threat levels: Benign, Low Risk, Suspicious, High Risk, or Malicious. Only after the level is fixed does the AI analysis layer, powered by Google's Gemini 2.0 model, translate the classification and its supporting indicators into plain language. The AI layer explains the decision and never changes it.

Output. The output component consists of the information presented to the user after processing. It includes the color-coded threat level, the risk score and confidence indication, a per-engine breakdown that shows the heuristic, VirusTotal, and AI findings in separate sections, a plain-language explanation tailored to the indicators found in that file, and practical guidance on what to do next. The system also stores each result so the user can review it in the History page, the Reports page, and the Dashboard. Presenting engines separately rather than as a single verdict addresses the "black box" problem stated in Section 1.2.

Feedback and Awareness. The framework includes a feedback loop beneath the main flow. Users who read explanations scan after scan become better at judging files on their own, and test findings during the Agile development cycles are used to refine the heuristic rules and thresholds. This loop connects the system's detection role to its educational purpose.

Component	Description in TrustFile	Examples
Input	Data supplied by the authorized user and needed by the system to begin a scan.	Login credentials; uploaded file or folder; file names, extensions, and sizes.
Process	System operations that validate, analyze, classify, and explain each submitted file without executing it.	Upload validation; hashing; magic-byte verification; entropy analysis; script and macro detection; size check; VirusTotal lookup; weighted scoring; five-tier classification; AI explanation.
Output	Information produced by the system for review and action.	Threat level; risk score and confidence; per-engine breakdown; plain-language explanation; stored scan history; dashboard and report summaries.

Table 1.1. Components of the IPO Conceptual Framework

Relationships Among the Elements

The framework assumes a sequential relationship among the three elements. First, an authenticated user provides a file. Second, TrustFile validates it, runs the local analysis and the external lookup, and passes all findings to the heuristic engine, which decides the threat level. Third, the AI layer explains the decided result, and the system displays and stores it. Each stage depends on the one before it. If the file is malformed or the user is not authenticated, no analysis takes place. If the external service is unavailable, the local analysis still completes, so the result is based on fewer signals but is not lost. The AI layer receives only the finished

classification, which keeps the risk decision consistent and reproducible while the explanation remains specific to each file.

Specific objective	Framework element	Expected system function
Develop a web-based system for uploading files for scanning before access	Input and Process	Accept authenticated browser uploads of a file or folder and validate them before any analysis.
Implement rule-based heuristic classification into five levels with AI-generated explanations	Process and Output	Combine all signals through weighted scoring, assign the threat level, and generate a plain-language explanation of the result.
Apply entropy analysis to detect encrypted, compressed, or potentially malicious files	Process	Measure data randomness and use it as one input among several, with safeguards for legitimate high-entropy formats.
Connect the system with external API-based malware detection services	Process	Query VirusTotal using the file hash, cache responses, and continue the scan if the service is unavailable.
Display clear scan results and an explanation to help users understand the analysis	Output and Feedback	Present the threat level, per-engine breakdown, explanation, and guidance, and store results for later review.

Table 1.2. Relationship of the Framework to the Specific Objectives

How the Literature Supports the Framework

The framework is a study-specific representation of TrustFile's functions rather than a claim that one published model fully defines the system. The literature cited in this chapter provides supporting context. Guo [1] and Sommestad et al. [2] discuss the limits of signature-based detection against new threats, which supports combining hash lookup with local heuristic checks. Gupta et al. [3] address the architecture and implementation of attachment malware scanning, which is consistent with arranging analysis stages into one pre-access pipeline. Gibert et al. [4] use structural entropy in malware classification, supporting the role of entropy as a detection signal. Bazrafshan et al. [7] survey heuristic malware detection techniques, and Sikorski and Honig [8] describe static analysis practices used to examine files without running them. The MITRE ATT&CK entry on double file extensions [6] supports verifying a file's real type rather than trusting its name. Khalid et al. [9] review fileless malware detection, which is relevant to the embedded script and macro checks, and Lavieille and Atlas [5] discuss explainability in cybersecurity analysis, which supports the explanation layer. The specific inputs, processing stages, and outputs in Figure 1.1 are based on the stated scope and features of TrustFile.

In summary, the IPO conceptual framework shows how TrustFile converts a user-submitted file into a rule-based threat classification and an understandable explanation. It links the study's objectives to the system's main functions and clarifies how static analysis, external intelligence, heuristic scoring, and AI explanation work together to support safer decisions before a file is opened.

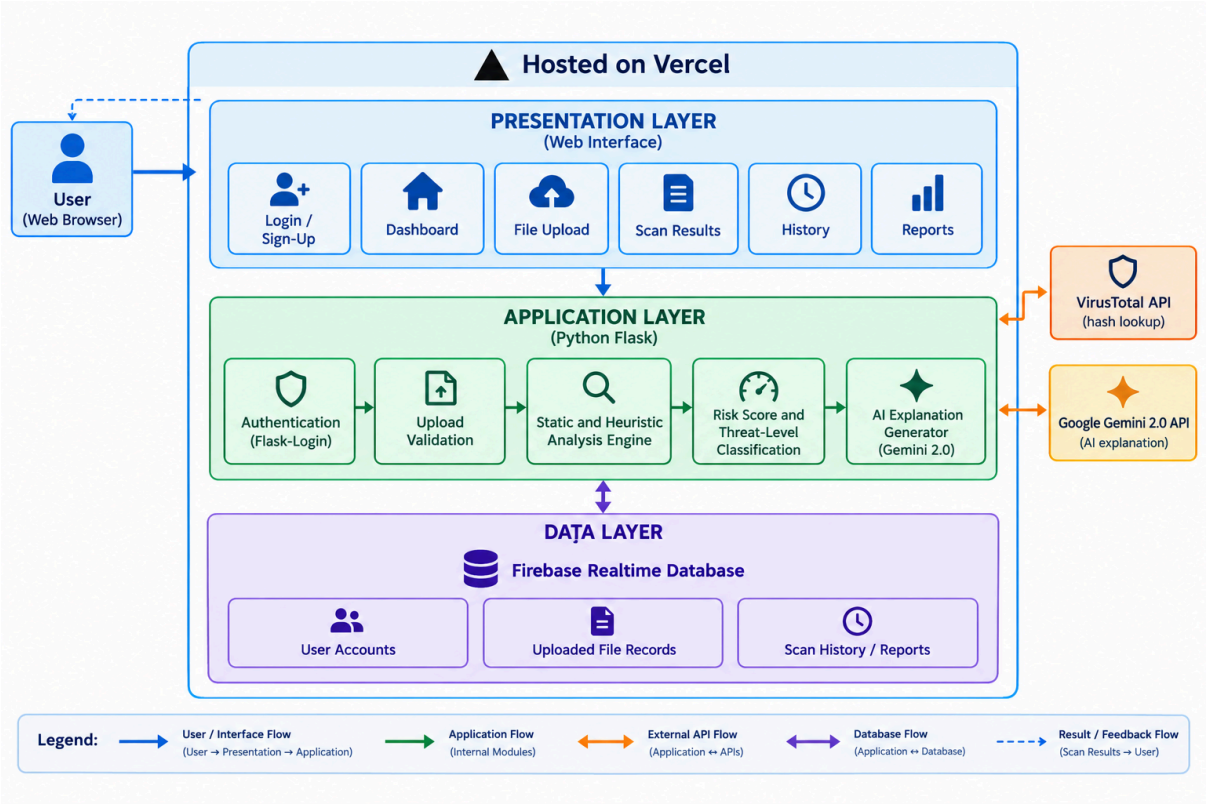


Figure 1.2. System Architecture Framework of TrustFile

Explanation of the Conceptual Framework of TrustFile

The architecture framework of TrustFile illustrates how the user, the three system layers, the external services, and the hosting platform work together. It follows the three-tier client-server design, which separates the presentation, application, and data layers so that a hostile file only ever interacts with the application layer and never directly with the interface or the stored records.

User. The user is the primary actor and may be an employee, student, or staff member who needs to check a file before opening it. The user reaches the system through an ordinary web browser, with no installation required.

Presentation Layer. This layer is the interface rendered by Flask's Jinja2 templates. The Login and Sign-Up pages control entry to the system. The Dashboard gives an overview of recent activity and file safety. The File Upload page is where each scan begins, and it lets the user choose between uploading a file or a folder. The Scan Results page shows the threat level and the separate heuristic, VirusTotal, and AI sections. The History page keeps earlier results, and the Reports page summarizes the severity breakdown and recent trends.

Application Layer. This layer is the Flask application that holds the system's decision logic. Authentication and Session Management, handled through Flask-Login, ensures that every request comes from an identified user. Upload Handling and Validation checks each submission before analysis. The Static and Heuristic Analysis Engine performs hashing, magic-byte verification, entropy analysis, embedded script and macro detection, size checking, and further inspection of archives and executables. Risk Score Aggregation and Threat-Level Classification combines these findings with the VirusTotal result and assigns one of the five levels. The AI-Assisted Explanation component then prepares the plain-language explanation.

External Services. The VirusTotal API supplies aggregated reputation data for the file's hash. Google's Gemini 2.0 model writes the plain-language explanation. The browser never contacts either service directly. All outbound requests pass through the application server, and VirusTotal receives only the file's hash.

Data Layer. Firebase Realtime Database stores the system's records in five groups: user accounts, password reset records, uploaded file records, the files associated with each user, and a cache of VirusTotal lookup results. File records hold the scores, classifications, and detection details that feed the History page, the Reports page, and the Dashboard. Because the database is cloud-hosted and independent of the application server, it keeps records safe even when the server's local storage is reset. Queries are scoped to the requesting user's identifier so that no one can read another user's scan history.

Hosting. The web application is deployed on Vercel, a cloud hosting platform that makes TrustFile reachable from any internet-connected device without local installation. Improvements to the scanning logic, heuristic rules, or explanation layer are released to every user at once. Vercel's execution environment does not provide persistent local storage for the application, which is one reason accounts, scan records, and cached lookups are kept in the separate Firebase data layer.

Arrows and Information Flow. The arrows represent the direction of data between components. Blue arrows show requests moving from the user through the presentation layer to the application layer. Green arrows show the internal analysis flow from validation through static analysis to classification. Orange arrows show calls to the external services and their responses. Purple arrows show reads and writes between the application layer and the database. A dashed arrow from the Scan Results page back to the user represents the feedback and awareness loop.

Overall, the architecture framework demonstrates how TrustFile brings together secure upload, multi-stage static analysis, external threat intelligence, rule-based classification, and AI-generated explanation in one web-based system. By centralizing these functions and keeping detection separate from explanation, the system aims to help users decide whether a file is safe before they open it.

1.3 Objectives of the Study

1.3.1 General Objective

The general objective of this study was to design, develop, and evaluate TrustFile, a web based file security platform that assessed the safety of digital files before user access by combining multiple static analysis techniques, rule based heuristic analysis, and external malware intelligence services to determine each file's threat classification paired with an AI generated explanation layer that translated those technical findings into plain language. Beyond detection alone, the system was built to improve cybersecurity awareness through scan results that were transparent and genuinely understandable intended for offices, government agencies, private organizations, educational institutions, and businesses that regularly handle large volumes of incoming digital files.

1.3.2 Specific Objectives

Specifically, the study aims to:

1. To develop a web based system that allows users to upload files for security scanning before access.

The first objective centered on building a browser based platform that required no additional software installation; users could scan a file the same way they'd check an email, from any internet connected device, regardless of operating system. This choice was deliberate: a web based architecture sidesteps compatibility issues entirely, and lets updates be pushed centrally rather than requiring every user to manually download a new version each time the system improves.

The upload mechanism itself was built to transmit files securely over encrypted channels, and to accept the range of formats organizations actually deal with day to day documents, compressed archives, executables, multimedia, and images regardless of file size. Every file that comes through undergoes a full security assessment before the user can access its contents, so the check always happens before exposure, not after.

Usability was treated as a first class requirement rather than an afterthought. Features like drag and drop uploading, visible progress indicators, scan status notifications, and clearly laid out result pages were built specifically so that someone with no cybersecurity background could still use the system confidently and understand what it was telling them.

2. Implement a rule based heuristic classification system that categorizes threats into BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS levels, paired with AI generated explanations for clear risk communication with non technical staff.

The second objective was about how risk gets decided and then communicated two distinct steps handled by two distinct components. A binary "safe or unsafe" result throws away information a user actually needs. It treats a file with one minor, easily explained anomaly the same as one with an outright malware hash match. TrustFile instead sorted every scanned file into one of five threat levels, giving users a much more honest picture of where a file actually fell on the risk spectrum.

The heuristic engine sat at the center of this decision making process, weighing everything the system had found file hashing, entropy analysis, script detection, size anomalies, file type verification, and the VirusTotal lookup result through a defined set of rules and thresholds to arrive at a single classification. Considering how these indicators related to each other, rather than evaluating each one in isolation, is what allowed the heuristic engine to produce a more reliable classification than any single check could on its own.

Under this framework: files with no suspicious characteristics at all were marked BENIGN; files with minor anomalies that had a reasonable, legitimate explanation were LOW RISK; files showing unusual behavior that genuinely warranted a closer look were SUSPICIOUS; files where several high confidence indicators pointed toward malicious activity were HIGH RISK; and files confirmed malicious through strong evidence a direct hash match against known malware, or overwhelming corroborating indicators were labeled MALICIOUS, triggering the appropriate warnings.

Once the heuristic engine had reached this classification, the finished result along with the specific indicators that had contributed to it was passed to the AI analysis layer (built on the NVIDIA NIM API and the Llama 3.1

70B Instruct model), whose sole role was to translate that already decided technical outcome into a clear, plain language explanation. This division lets users, and organizations, respond proportionally to a classification they could trust as consistent and rule based, while still receiving an explanation written in language a non technical reader could actually understand.

3. To apply entropy analysis to detect encrypted, compressed, or potentially malicious files.

The third objective integrated entropy analysis as one of TrustFile's core static analysis techniques. Rooted in information theory, entropy measures how random the data inside a file is and because encryption and compression both sharply increase that randomness, unusually high entropy is often a sign that a file is hiding something: an encrypted payload, a packed executable, or code deliberately obfuscated to dodge conventional signature based scanning.

This mattered because a large share of modern malware relies on exactly this kind of obfuscation to slip past traditional antivirus tools. But the researchers were also careful not to treat high entropy as automatic proof of malicious intent; plenty of entirely legitimate files (compressed archives, encrypted backups, standard multimedia formats) are naturally high entropy simply because of how those formats work. For that reason, entropy analysis was never used as a standalone verdict. It was one input among several combined with hashing, heuristic analysis, script detection, type verification, size analysis, and the AI engine's reasoning specifically to avoid false positives that would otherwise undermine trust in the system.

4. To connect the system with external API based malware detection services for detection.

The fourth objective extended TrustFile's detection beyond what it could determine locally, by tapping into external threat intelligence services with access to malware databases, behavioral profiles, and reputation data built from global sensor networks and security research resources far beyond what any single organization could replicate on its own.

Integrating with these services meant balancing several competing demands at once. Query efficiency keeps latency and resource use low while still maximizing coverage. Privacy protection typically achieved by sending only a file's cryptographic hash rather than its actual contents ensures sensitive files are never unnecessarily exposed to a third party. Reliability requires fallback behavior for when an external service is temporarily unreachable, so the system degrades gracefully instead of failing outright. Cost management keeps API usage sustainable through caching and smart query batching.

Among the services considered, VirusTotal aggregates detection results from more than 70 different antivirus engines and URL/domain reputation tools into a single consensus report, so that a submitted file's fingerprint can be checked against dozens of independent detection systems at once rather than just one (VirusTotal, n.d.; Devoteam, 2025). MetaDefender offers a comparable multi-scanning approach alongside file sanitization

capabilities. Each service carries its own tradeoffs in coverage, accuracy, speed, and cost, which is why the integration was designed to draw on more than one source rather than depending on a single provider.

Beyond direct detection, this external intelligence layer also serves a broader purpose: comparing local results against outside sources highlights where TrustFile's own detection logic could be sharpened, and patterns emerging across a global user base can act as early warning of new attack campaigns well before they show up locally.

5. Display clear scan results and an explanation to help users understand the analysis

The fifth objective is where TrustFile's educational purpose becomes most concrete. A detection result that isn't understood doesn't actually build security resilience. At best it produces uncritical trust in the tool, and at worst it teaches users to ignore warnings altogether once enough of them turn out to be confusing or seemingly irrelevant. Presenting results well means applying real principles of information design: leading with what matters most (the classification and any immediate action needed) before supporting technical detail; using color, icons, and layout to convey severity at a glance; replacing jargon with plain language without sacrificing accuracy; tying findings to real world consequences the user can actually picture; and always pairing a result with a concrete, feasible next step.

The explanation layer is what answers the "why." A hash match comes with a description of the malware family it belongs to and how it typically behaves. An entropy anomaly comes with a plain explanation of what elevated randomness usually means and what legitimate reasons might explain it. A detected script explains what kind of script it is and why its presence is suspicious in that specific file's context. A size anomaly explains what a normal range looks like for that file type and what a deviation might suggest. A type mismatch explains exactly how the file's real format differs from what it claimed to be, and the spoofing techniques that produce that kind of mismatch.
Rather than pulling from generic, one size fits all templates, TrustFile's explanation system builds each response from the actual combination of indicators found in that specific file producing an explanation that's genuinely tailored to what was detected, not a boilerplate warning. The cumulative goal across all five objectives is the same one that runs through the whole study: a user base that, scan by scan, becomes measurably better at recognizing threats independently, reducing blind reliance on automated tools while strengthening both individual and organizational security awareness over time.

1.4 Scope and Limitations

1.4.1 Scope of the Study

This study focused on the development of TrustFile, a web based system that allowed users to scan and analyze individual files for possible malware directly within their browser, before those files were opened or executed.

Before detailing what TrustFile did, it was necessary to define clearly what this research did and did not attempt, since that boundary shaped how the system was fairly evaluated. TrustFile was built and deployed as a web based system, meaning users accessed it through an ordinary browser rather than installing anything locally. The system was hosted on PythonAnywhere, a cloud based Python hosting platform, so that anyone with an internet connection could reach it without configuring a server or dealing with any technical setup on their end. This was a deliberate design decision. The organizations this study targeted: offices, private companies, government agencies, and similar institutions that handled a high volume of incoming files, did not necessarily have the technical staff or infrastructure needed to maintain a locally installed security tool. A lightweight, browser accessible platform made far more practical sense for that context. Because the system was centrally hosted, any improvement made to the scanning logic, the AI analysis engine, or the heuristic rules could also be rolled out immediately to every user, with no separate update process required on their part.

In terms of how scanning actually worked, the system's scope was strictly upload based. A user submitted one file at a time through the web interface; that file was transmitted to the PythonAnywhere hosted server, where the system performed the full range of static scans (hashing, entropy analysis, script detection, size verification, and file type verification), queried external threat intelligence APIs, and then passed the combined results to the heuristic engine for classification, after which the AI analysis layer generated a plain language explanation of the outcome. What this study deliberately excluded was batch scanning of multiple files at once, scanning of entire folders or directories, and any form of continuous or scheduled monitoring. The intent was to give a user a reliable way to check one specific file they were about to open, not to build a full endpoint protection suite. Keeping the scope this narrow kept the project realistic within an academic timeframe while still addressing the core problem this study set out to solve: that people lacked an easy, explainable way to check whether a file was safe before opening it.

At the heart of the system was its heuristic engine, which worked from the combined results of several scanning techniques rather than treating each one as an isolated pass or fail check. The engine considered everything together (the hash comparison, the entropy value, whether a script was detected, whether the file size fell outside the norm, whether the file's actual type matched what it claimed to be, and the external threat intelligence result) and applied a defined set of rules and thresholds that weighed how these pieces related to one another before arriving at a single classification. This was, in effect, an attempt to replicate how an experienced human analyst would reason through a file: not fixating on any single red flag, but weighing several pieces of evidence at once. Because the rules looked for suspicious structural patterns and known obfuscation techniques rather than exact signature matches, this layer mattered specifically for catching threats new enough that no external database had encountered them before. Once the heuristic engine had decided the classification, the AI analysis layer, built on the NVIDIA NIM API and the Llama 3.1 70B Instruct model, took that finished result and its supporting indicators and translated them into a plain language explanation. The AI layer did not alter or override the classification; its sole role was explanation.

The system also connected to external malware intelligence APIs to check file hashes against databases of known threats, while everything else (script detection, structural pattern analysis, and checking whether a file's

actual structure matched its stated extension) was carried out locally on the server. Based on the combined output of all these checks, the heuristic engine sorted every scanned file into one of five threat levels:

- BENIGN - the file passed every check cleanly, with no anomalies detected anywhere, and was allowed through without restriction.
- LOW RISK - the file showed small deviations that typically had an innocent explanation, such as slightly elevated entropy from ordinary compression; access was still granted, accompanied by a note explaining what was flagged and why it was likely nothing.
- SUSPICIOUS - the file showed something genuinely unusual, not enough to call outright malicious, but enough to warrant a closer look or a temporary restriction while that review took place.
- HIGH RISK - several indicators lined up together, or a single indicator was strong enough on its own (a hit from an external threat intelligence API, an entropy value far outside the expected range, a script attempting to contact an external server), meaning the file was likely malicious but not yet confirmed, and was quarantined pending further review.
- MALICIOUS - the file was confirmed harmful, most commonly through a direct hash match to a known threat, triggering an immediate block, quarantine, and alert.

The purpose of using five levels instead of a simple safe/unsafe flag was to allow a response proportional to the severity of what was actually found. Every classification was paired with a plain language explanation of what triggered it, along with relevant tips, so that users walked away understanding a little more about what made their file risky or not.

Technically, this amounted to a hybrid detection setup that relied on both external intelligence and local analysis, since each covered gaps the other left open. The external side worked through file hashing: generating a unique fingerprint of a file's contents and checking it against databases of known malicious hashes maintained by established threat intelligence services. This method was fast and effective against anything already documented, but it could not, by definition, catch something entirely new, or even a previously known file with a single byte altered to produce a different hash [1]. This is where local, server side scanning came in. Script detection searched for executable code hidden inside files that should not normally contain any (JavaScript embedded in a PDF, a macro buried inside a Word document, PowerShell commands hidden in an archive), any of which could indicate trouble even when the file appeared entirely ordinary on the surface. Heuristic rules looked for suspicious structural patterns and known obfuscation tricks. File type verification checked whether a file's actual internal structure matched what its extension claimed, catching cases where, for instance, an executable had been dressed up to resemble a document. Entropy analysis flagged files that were unusually random looking, which could point to encryption or packing used to conceal malicious code, though this had to be interpreted carefully, since a number of entirely harmless files, such as compressed archives and certain media formats, were naturally high entropy as well, and the system needed to reliably distinguish between the two cases. All of these local results, together with the external lookup, fed into the heuristic engine, which tied the entire detection process together, picking up on combinations of small anomalies that might not mean much individually but added up to something worth flagging when considered together. This mattered specifically for

genuinely novel or deliberately obfuscated threats that matched no known signature. The AI analysis layer then explained the resulting classification to the user rather than determining it.

Beyond detection and classification, this study also placed real weight on helping users understand what was happening, which is why explanations were built into every scan result rather than added as an afterthought. Whenever the system flagged something, it explained precisely what triggered the flag (a hash match to a known malware family, an unusually high entropy value and what that typically indicated, a detected script and why its presence was suspicious in that particular type of file, a size deviation from what was expected, or a file type that did not match its stated extension), all explained in plain terms that assumed no prior cybersecurity background. Alongside these explanations, the system offered general guidance on safe file handling, recognizing common attack tactics, understanding different malware categories, and maintaining day to day system security. Folding this education directly into the scanning process, rather than treating it as a separate lesson, was intended to help users build genuine security awareness over time, with the underlying goal being that users came away understanding why a file was flagged, not simply that it was.

1.4.2 Limitations of the Study

Within the scope described above, the system operated within clear limits, which are stated plainly here so that the results of this study are interpreted fairly. The choice of a web based, centrally hosted platform came with trade offs the researchers considered worth stating. Browsers were sandboxed environments by design, meaning TrustFile could not reach into the operating system the way a native application could; it had no ability to hook into the file system at a low level or intercept file operations as they occurred elsewhere on the device. The system's availability was also inherently tied to a stable internet connection and to whatever resource limits and uptime came with the specific PythonAnywhere hosting tier in use. Given that this study set out to assess files before they were opened, rather than provide continuous, system embedded protection, these trade offs were judged acceptable for the scope of an academic project.

The system was built around static analysis only, meaning it examined a file's contents and structure without ever executing it [8]. It offered no real time protection and did not attempt to replicate the full feature set of commercial antivirus software. It could only scan files a user actively chose to upload, and, like any security tool, it could not guarantee it would catch every possible threat. It was also, first and foremost, an academic project, tested against a curated set of sample files rather than deployed as a finished commercial product.

A few specific boundaries were worth spelling out directly. Because the system only ever examined a file's contents and structure without running it, it had no way of catching malware that revealed itself solely through execution: logic bombs that remained dormant until a specific condition was met, fileless malware that existed entirely in memory without ever writing a file to disk, or malware that only downloaded its actual payload after an initial, seemingly harmless file had already been run [8], [9]. Dynamic analysis, which involves actually executing a file in a sandboxed environment to observe its behavior directly, would have been capable of catching these cases, but building that capability was outside what this study set out to do [8]. Static analysis

was safer and faster to implement, but it traded away that behavioral insight, and that trade off was made deliberately within this study's scope.

Real time protection was excluded for a related reason. Traditional antivirus software runs continuously in the background, scanning files the instant they are touched anywhere on a device, but doing so properly requires deep integration with the operating system, kernel level access, and constant background resource consumption, none of which was realistic for a browser based tool running on shared cloud hosting such as PythonAnywhere. Within this study, scanning therefore only occurred when a user actively chose to upload a file; there was no passive, always on protection layered across everything a user did on their device.

The system was also limited to files a user could legitimately access on their own. It could not reach files locked behind operating system permissions, files inside encrypted volumes without the corresponding key, or anything on a system it had no access to in the first place, and this limitation was not purely technical. Scanning files a user did not have legitimate access to would have required elevated privileges that raised genuine privacy and ethical concerns well beyond the intent of this study.

None of this was meant to overstate what TrustFile could do. No security tool, however well designed, catches every threat, and the specific combination of techniques used here (static analysis, reliance on files the user actively chose to upload, and a defined set of detection methods) inevitably left some gaps. That limitation is shared by security research generally, not unique to this project, and it does not diminish what TrustFile was able to demonstrate within the scope it set for itself. Finally, it bears repeating that this was an academic research project, not a finished commercial security product ready for public release, a distinction that mattered for how the system was evaluated and what could reasonably be expected of it. In keeping with that, testing relied on a deliberately curated set of sample files representing both benign and malicious characteristics, rather than exposing the system to live malware in an uncontrolled, real world setting, a choice made to validate the system's capabilities responsibly, without risking actual harm in the process.

1.5 Significance of the Study

The proposed system, TrustFile: A File Security Platform Leveraging Multi-Engine Static Analysis and Explainable AI Detection, is expected to deliver significant benefits to the following stakeholders:

End Users - End users experienced safer file interactions through automatic pre-access scanning, which ensured that only verified, threat-free files reached their workstations without requiring technical expertise or manual intervention on their part. This stakeholder group encompassed everyone who interacted with digital files in a personal or professional capacity students exchanging assignments and resources, employees processing work documents, consumers downloading software and media, and general computer users engaging with digital content as part of daily life. The significance for this group centered on a fundamental shift: file security moved from a specialized technical concern that demanded real expertise into an accessible, largely automated service available to anyone, regardless of their technical background. This safety came from the systematic layering of

multiple detection techniques, AI-driven analysis, and heuristic analysis before a file ever reached a user's workstation. Rather than depending on users to personally recognize threats, or relying on remediation after an infection had already occurred, TrustFile intercepted potentially dangerous files at the point of access, preventing exposure before it could happen at all. This shifted the underlying security model from reactive to proactive, which in practical terms translated into fewer successful infections and fewer of the downstream consequences lost data, compromised credentials, disrupted work that came with them.

The phrase "without requiring technical expertise" addressed a real accessibility gap in how file security was typically practiced. Existing tools often quietly assumed a level of user knowledge that most people simply did not have understanding what a file extension actually signified, recognizing the subtler indicators of a phishing attempt, correctly interpreting a security warning, or even knowing how to run a manual scan in the first place. By automating these functions and presenting results in accessible language, complete with clear, AI-generated explanations, TrustFile extended meaningful protection to users who would otherwise have remained exposed simply because they had never had the technical background needed to protect themselves.

Removing the need for manual intervention also reduced both the mental burden placed on users and the room for human error. Users did not have to remember to run a scan, did not have to interpret ambiguous technical indicators on their own, and were not left making risk judgments under uncertainty with incomplete information. The system took on these responsibilities consistently and thoroughly, removing the user as a potential point of failure in the security chain, while still keeping them informed and gradually more knowledgeable through the explanations it provided with every scan.

The Organization - The organization demonstrated a responsible and intelligent approach to file security, establishing TrustFile as a practical model for protecting digital assets in government offices, private sector organizations, and corporate environments that regularly received and downloaded files. This stakeholder group included businesses, government agencies, educational institutions, non-profit organizations, private sector companies, and any entity that managed digital assets and faced file-based security risks as a routine part of its operations. The significance for organizations extended beyond a direct improvement in security posture, reaching into reputational, operational, and strategic benefits as well. Demonstrating a responsible approach to file security meant putting in place visible, effective controls that protected organizational data, kept operations running smoothly, and fulfilled an organization's duty of care to the people it served. In a climate of increasing regulatory scrutiny around data protection under frameworks such as GDPR, HIPAA, and PCI-DSS, along with various sector-specific regulations having demonstrable security measures in place offered tangible evidence of compliance effort and due diligence. TrustFile supported this by producing auditable scan records, applying its checks consistently across every file, and documenting exactly what was detected and why. Describing the system's approach as "intelligent" reflected its integration of multiple detection techniques AI-driven analysis, heuristic analysis, and external threat intelligence working together, rather than any single detection method carrying the full weight on its own. This layered strategy reflected established security best practices and positioned any adopting organization as embracing a more sophisticated, contemporary approach to protection, rather than depending on a single point of failure the way older signature-only tools often did.

The intelligence built into the system also extended to how the AI analysis engine reasoned across combinations of indicators rather than evaluating them individually, allowing its assessments to remain relevant even as new patterns of malicious file characteristics continued to emerge over time. The specific focus on government offices, private sector organizations, and corporate environments reflected how relevant file security was in exactly the professional contexts where files constantly arrived from outside the organization. Government agencies handled sensitive citizen data, classified information, and critical infrastructure controls that made them attractive, high-value targets for attackers. Private sector companies managed proprietary business information, financial records, and client databases, all of which carried real consequences if compromised. Corporate environments, more broadly, dealt with a constant stream of incoming documents from partners, clients, and vendors, any one of which could carry a hidden threat. Budget limitations, aging legacy systems, and uneven technical literacy among staff often made proper security implementation harder in these settings than it appeared on paper. TrustFile's web-based, straightforward, and low-cost approach was designed with exactly these constraints in mind, offering a practical model that could reasonably be adopted and adapted across many different kinds of organizations, regardless of their existing technical infrastructure.

Positioning TrustFile as a practical model meant more than simply proving that the underlying technology worked it meant documenting how it was implemented, honestly evaluating how effective it actually was, and sharing those findings in a way that allowed other organizations to learn from and build on the work. In that sense, this research contributed to the broader body of knowledge around accessible file security solutions, offering something other organizations could reasonably adapt to their own particular needs and constraints.

Future Researchers - Future researchers were able to use this study as a reference, build on it to advance learning-based detection systems, and draw on it to help improve security applications in research and development more broadly. This stakeholder group covered academic researchers, graduate students, industry researchers, and developers who went on to investigate file security, malware detection, web-based security tools, AI-driven applications in cybersecurity, heuristic analysis techniques, and related areas. The significance for this group lay in the study's contribution to a growing, cumulative body of knowledge, and in its role as a starting point for further work rather than an endpoint in itself. As a reference, this study offered a documented methodology, implementation details, evaluation results, and honestly reported lessons learned all of which future researchers could consult, critique, and build upon as they saw fit. The documented approach to the hybrid detection architecture, the AI analysis engine's design, the integration of heuristic analysis, the API integration strategy, the interface design choices made for a security tool aimed at non-technical users, and the integration of educational content into the scanning workflow all offered concrete technical guidance that went beyond abstract discussion. The evaluation framework used in this study also gave future researchers a benchmark against which their own systems could reasonably be measured.

Building on this study to advance learning systems pointed specifically to the potential for extending TrustFile's capabilities using more advanced AI techniques in future work. The AI-driven, five-tier classification approach described in the Objectives provided a foundation that later work could refine further through more

sophisticated reasoning approaches, a broader and more diverse set of indicators, or more precise weighting of how different anomalies related to one another. The heuristic analysis component, similarly, could be expanded with a more comprehensive set of rules and behavioral indicators over time. The static analysis techniques used in this study could eventually be supplemented with dynamic analysis, behavioral modeling, or even automated reverse engineering, for a fuller picture of a file's actual risk. And the web-based architecture itself left room for future extension into mobile applications, browser extensions, or deeper enterprise integration.

Improving security applications in research and development, more broadly, reflected the real influence that academic work of this kind could have on practical security tools used in the wider world. Findings about which detection techniques actually worked, how effective AI-driven and heuristic analysis proved to be in practice, what interface choices genuinely helped non-technical users, which educational approaches actually built understanding rather than just producing another warning users learned to ignore, and how all of these pieces fit together architecturally all of this fed into commercial product development, open-source security projects, and the security implementations organizations eventually adopted. In that sense, this research formed part of the pipeline that turned academic insight into protections people actually used.

1.6 Definition of Terms

Abnormal File Size: Refers to a file whose size falls outside the range normally expected for its declared type or content, a deviation that can indicate an embedded malicious payload, appended code, or a stripped down stub file.

API (Application Programming Interface): A set of protocols enabling separate software applications to communicate and exchange threat intelligence data.

Artificial Intelligence (AI) Analysis: The decision making and translation layer that synthesizes multi engine detection indicators into plain language assessments.

Benign Classification: The classification assigned when a file passes all static and threat intelligence checks with zero anomalous indicators.

Black Box: A system whose internal reasoning is hidden from the user, producing opaque verdicts without explanatory context.

Data Breach: An incident in which sensitive, protected, or confidential information is accessed, exposed, or extracted by an unauthorized party whether through a successful malware infection, a compromised credential, or a direct network intrusion. Data breaches carry consequences well beyond the immediate loss of data: they generate direct financial costs from investigation and remediation, interrupt normal business operations, expose organizations to regulatory penalties, and cause lasting reputational damage. As referenced earlier in this chapter, IBM's 2024 Cost of a Data Breach Report placed the global average cost of a single breach at roughly

US$4.88 million, underscoring why preventing a malicious file from executing in the first place TrustFile's core purpose carries financial significance well beyond the individual file being scanned.

Defense in Depth: A security strategy layering multiple overlapping defensive controls to eliminate single points of failure.

Shannon Entropy: A mathematical measure of randomness in byte sequences, used to detect packing, encryption, or obfuscation in binary payloads.

Entropy: A concept borrowed from information theory that measures how random or unpredictable the data within a file is. Encryption and compression both sharply increase a file's entropy, which is why entropy analysis became a useful indicator for identifying files that might be hiding malicious content an unusually high entropy value can point to an encrypted payload, a packed executable, or otherwise obfuscated code, while legitimate files such as plain text documents or standard executables typically show much lower entropy because their internal data follows structured, predictable patterns.

False Positives and False Negatives: Two categories of error inherent to any detection or classification system, representing an unavoidable trade off in security accuracy. A false positive occurs when the system incorrectly flags a legitimate, harmless file as malicious, a false alarm that, if it happens too often, wastes administrative time, erodes user trust in the system, and can push frustrated users toward disabling security features altogether. A false negative occurs when the system fails to catch an actual threat, letting a malicious file pass through undetected, a more dangerous failure mode, since it results directly in a successful infection rather than merely an inconvenience. Minimizing false negatives while keeping false positives at an acceptable level was a central design consideration throughout TrustFile's development, addressed by combining multiple complementary detection techniques rather than depending on any single check that might individually be prone to one error type or the other.

File Extension Spoofing and Double Extensions: A deceptive technique in which attackers manipulate a file's visible filename and extension to disguise its true, and dangerous, underlying type. Because most operating systems determine how a file is handled based on its extension, an attacker can make an executable appear to be a harmless document simply by naming it accordingly. Double extension techniques take this further by constructing filenames with two extensions such as "document.pdf.exe" or "invoice.docx.scr" relying on the fact that many operating systems hide known file extensions by default, so that a user sees only "document.pdf" and has no visual indication that the file is, in reality, an executable. TrustFile countered this specific deception through file type verification, which examined a file's actual internal structure rather than trusting its visible name.

File Type Verification: A static analysis technique that confirms whether a file's actual internal structure matches what its extension and metadata claim it to be, rather than relying on visible cues that an attacker could easily fake. This check worked by examining a file's magic numbers, the fixed byte sequences that appear at the start

of a file regardless of its name and comparing them against what would be expected for the file's stated type. A mismatch between a file's claimed type and its actual internal structure was treated as a strong warning sign, since this is precisely the technique attackers rely on when disguising an executable as an ordinary document. File type verification worked as a direct countermeasure to file extension spoofing and double extension tricks, catching exactly the kind of deception that a purely visual inspection would miss.

Fileless Malware: A category of malicious software that operates entirely within a computer's memory (RAM) rather than writing any file to disk, which allows it to evade traditional, file based security tools almost entirely. Instead of installing an executable, fileless malware typically abuses legitimate, pre-installed system tools such as PowerShell or macros embedded in Office documents to carry out malicious commands directly in memory. Because it leaves no persistent file behind, fileless malware bypasses signature based detection and standard antivirus scans that depend on examining stored files. This represented one of the clearest limitations acknowledged in this study's Scope and Delimitation: because TrustFile analyzed a file's contents and structure without ever executing it, it had no way of catching a threat that revealed itself only once running in memory a gap that would require dynamic, behavior based analysis to close.

Hashing: A cryptographic process that generates a unique, fixed length string of characters known as a hash value from a file's contents, using algorithms such as MD5, SHA 1, or SHA 256. The defining property of a hash function is that even the smallest change to a file, down to a single altered bit, produces a completely different hash value, making hashes extremely reliable for identifying whether two files are truly identical. Within TrustFile, file hashing served as the fastest of the system's detection techniques: a scanned file's hash could be checked directly against external databases of known malicious hashes, allowing instant identification of previously encountered threats without needing to examine the file's contents in any deeper way. Its clear limitation, addressed elsewhere in this chapter, is that it can only recognize threats that have already been seen and catalogued before a single altered byte is enough to produce an entirely different hash and slip past this check undetected.

Heuristic Analysis: A detection technique that identifies potentially new or previously unknown malware by examining a file's structural characteristics and behavioral indicators, rather than requiring an exact match against a known malware signature. Where signature based detection can only recognize threats it has already catalogued, heuristic analysis applies rules designed to catch suspicious patterns commonly associated with malicious files, unusual code structures, signs of obfuscation, or characteristics resembling known attack techniques even when no exact match exists. This gives heuristic analysis a genuine advantage in catching zero day threats and polymorphic malware. Its corresponding weakness is a tendency toward false positives, since legitimate software occasionally uses unusual coding techniques that can resemble the patterns heuristic rules are built to catch. Within TrustFile, heuristic analysis ran alongside the AI analysis engine, catching suspicious characteristics that matched no known signature or pattern the AI engine had previously learned from.

High Risk Level: The classification assigned to a file that showed strong suspicious characteristics without having been definitively confirmed as malicious for example, an entropy value far outside the expected range,

a script attempting to establish contact with an external server, or a file type that failed to match its claimed extension. A HIGH RISK classification meant the file was likely malicious, but not yet confirmed with the certainty required for the MALICIOUS label, and was accordingly quarantined pending closer administrative review rather than blocked outright or allowed through.

Keyloggers: A category of spyware specifically designed to covertly record every keystroke a user types, capturing passwords, financial details, and other sensitive information without the user's knowledge. Software keyloggers can run as kernel level drivers that intercept keystrokes directly at the operating system level, or as ordinary applications quietly monitoring window activity in the background. Because keyloggers operate independently of whatever other precautions a user takes a strong password offers no protection if every keystroke used to type it is already being recorded they represent a particularly dangerous category of malware, and one of the reasons malware in general, as discussed in the Background of the Study, remains such a persistent concern for organizations handling sensitive data.

Logic Bombs: Malicious code deliberately inserted into an otherwise normal file or system, engineered to remain completely dormant and undetectable until a specific triggering condition occurs a particular date, the absence of a specific user account, or some other predefined event at which point it executes its destructive payload. Because a logic bomb exhibits no malicious behavior whatsoever until triggered, it can pass through security scans and code review entirely unnoticed. This represents one of the clearest limits of static analysis acknowledged in this study's Scope and Delimitation: a system that examines a file without executing it has no way of detecting a threat that only reveals its true nature once a specific, often unpredictable, condition is eventually met.

Low Risk Level: The classification assigned to a file that was largely clean but showed a small, usually explainable deviation commonly, mildly elevated entropy resulting from ordinary compression rather than anything malicious. Files receiving this classification were still granted normal access, accompanied by a brief note explaining what had been flagged and why it was most likely nothing to be concerned about. As with the benign classification, a low risk label reflects a high degree of confidence rather than an absolute guarantee, since detection systems operate in terms of probability rather than certainty.

Magic Numbers: Also known as file signatures, magic numbers are fixed byte sequences located at the very start of a file that reliably identify its true format, independent of whatever filename or extension the file has been given. A PDF file, for instance, begins with the bytes corresponding to "%PDF," while a standard Windows executable begins with the characters "MZ." Because these byte sequences are embedded as part of a file's actual content rather than its filename, they cannot be altered simply by renaming a file, which made them the technical foundation underlying TrustFile's file type verification allowing the system to confirm a file's real identity even when its visible name had been deliberately falsified.

Malicious Threat Level: The highest classification within TrustFile's five tier system, reserved for files confirmed to be genuinely harmful and requiring immediate action. Reaching this classification required a high

degree of confidence, typically either a direct hash match against a known threat in an external intelligence database, or several converging indicators of malicious behavior strong enough, together, to leave little doubt. Files given this classification triggered immediate blocking, quarantine, and an administrative alert, reflecting the severity of the confirmed risk.

Malware: A general term for any software deliberately developed to damage, disrupt, or gain unauthorized access to a computer system, typically for the purpose of stealing data, extorting a victim, or disrupting operations. Malware can cause damage in numerous ways: deleting or corrupting files, stealing credentials and financial information, disabling security software, or, in more severe cases, damaging firmware or hardware directly. Malware is typically categorized according to how it spreads (attaching to files, propagating independently across a network, or disguising itself as legitimate software), what it ultimately does once active, and how effectively it conceals its presence. Viruses, worms, Trojan horses, ransomware, spyware, rootkits, and keyloggers all discussed elsewhere in this chapter represent different categories within this broader term.

Packers and Crypters: Tools used by malware developers to compress or encrypt malicious code specifically to make it harder for antivirus software to detect and analyze. A packer compresses an executable, obscuring its internal code structure until a small decompression routine unpacks it back into memory at runtime; common examples include UPX and ASPack, tools that are also occasionally used for entirely legitimate purposes, such as reducing file size or protecting software from reverse engineering. A crypter, by contrast, encrypts the malicious payload outright, hiding the harmful code completely until a decryption routine embedded in the file reveals it at execution leaving no trace of the original malicious code visible on disk beforehand. Both techniques significantly alter a file's outward appearance, rendering signature based detection ineffective since the packed or encrypted file no longer matches any known malware signature. Both also sharply raise a file's entropy, which is precisely why entropy analysis served as such a valuable complementary technique within TrustFile's detection process.

Phishing: A social engineering technique in which attackers send fraudulent communications most commonly emails disguised as legitimate correspondence from a trusted source, such as a bank, employer, shipping company, or government agency, in order to trick a recipient into opening a malicious attachment or divulging sensitive information. Phishing exploits psychological pressure points such as urgency, curiosity, or authority rather than any technical vulnerability, which is precisely why it remains, as discussed in the Background of the Study, one of the most common and effective ways malicious files are distributed in the first place. Because phishing succeeds by manipulating human judgment rather than bypassing a technical defense, no purely automated detection system can fully eliminate the risk it poses which is part of why TrustFile placed such emphasis on user education alongside detection.

Polymorphic and Metamorphic Malware: Two advanced techniques malware authors use to evade signature based detection by continually altering a file's appearance. Polymorphic malware encrypts its malicious payload using a variable key and pairs it with a decryption routine that changes with each new infection, so that while the underlying malicious function stays identical, the encrypted body and accompanying routine look different

every time producing an entirely new file hash with each generation. Metamorphic malware goes further still, completely rewriting its own code with every generation, altering not just its outward appearance but its internal structure and instruction sequence, while preserving the same underlying malicious function. Both techniques illustrate the ongoing arms race between malware developers and security researchers referenced throughout this chapter, and are part of the reason detection approaches that go beyond simple signature matching heuristic and AI driven analysis among them remain necessary.

Pre Access Scanning: The process of examining a file for potential threats before a user is permitted to open or execute it, intercepting the file at the exact point where damage would otherwise occur. This represented the core preventive philosophy behind TrustFile's design: rather than depending on a user's own judgment, or on cleaning up an infection after it had already happened, pre access scanning applied the system's full combination of static analysis techniques, AI driven analysis, and heuristic analysis to a file the moment it was uploaded, before the user ever gained access to its actual contents.

Ransomware: A category of malware that encrypts a victim's files and demands payment, typically in cryptocurrency, before restoring access to them. Modern ransomware campaigns frequently pair this encryption with a "double extortion" tactic, in which attackers also threaten to publicly release the stolen data if the ransom goes unpaid, adding a second layer of pressure beyond the simple loss of access. As referenced in the Background of the Study, ransomware represents one of the most financially damaging categories of malware currently in circulation, and its typical delivery method a malicious file disguised as an ordinary attachment is precisely the kind of threat TrustFile's pre access scanning approach was designed to intercept before execution ever occurred.

Regulatory Compliance (GDPR, HIPAA, PCI DSS): The set of legal and industry frameworks governing how organizations must handle sensitive data, with penalties attached for failing to demonstrate adequate protective measures. The General Data Protection Regulation (GDPR) governs personal data protection primarily within the European Union; the Health Insurance Portability and Accountability Act (HIPAA) governs the protection of medical and health related information within the United States; and the Payment Card Industry Data Security Standard (PCI DSS) governs the handling of payment card information across any organization that processes it. As discussed in the Significance of the Study, demonstrating consistent, auditable file security practices of the kind TrustFile was designed to produce through its scan logs and documented classifications offers organizations tangible evidence of compliance effort under any of these frameworks.

Rootkits: A category of malware designed to gain deep, persistent access to a system while actively concealing its own presence from both the user and security software. The term combines "root," referring to the highest level of administrative access on Unix based systems, with "kit," referring to the collection of tools used to establish and maintain that access. Rootkits operating at the kernel level are especially dangerous, since they run with the highest possible system privileges, allowing them to intercept system calls, hide files and processes, and disable security software outright. Because rootkits actively subvert the very mechanisms typically used to detect them, they are notoriously difficult to identify through conventional means.

Script Detection: A static analysis technique that identifies executable code embedded within files that would not normally be expected to contain any JavaScript hidden inside a PDF, a macro embedded in a Word document, or PowerShell commands buried inside a compressed archive. The presence of such a script is not automatically proof of malicious intent, but it is a meaningful warning sign, since this is precisely the mechanism by which many disguised malicious files operate: the file appears entirely ordinary on the surface while quietly carrying executable code capable of running once opened. Within TrustFile, script detection results were never evaluated alone; they were combined with the system's other static analysis techniques and fed into the AI analysis engine to help determine an overall classification.

Signature Based Detection: The foundational method used by traditional antivirus software, in which a scanned file is compared against a database of previously identified malware fingerprints, or signatures, and flagged if a match is found. This approach is fast and highly reliable against threats that have already been discovered and catalogued, but it carries an unavoidable structural limitation: it cannot recognize malware that is new, modified, or deliberately altered to avoid producing a matching signature, leaving a window of vulnerability between a threat's emergence and the moment a corresponding signature is finally issued. As discussed throughout this chapter, this limitation is precisely why TrustFile combined signature based hash comparison with heuristic analysis, entropy analysis, and AI driven reasoning, rather than depending on signature matching as its sole line of defense.

Social Engineering: The psychological manipulation of a person into taking an action, or disclosing information, that compromises security targeting human judgment rather than any technical vulnerability. Common techniques include phishing, pretexting (fabricating a false scenario to extract information), and baiting (offering something enticing to lure a victim into compromising their own security). In the specific context of file based malware, social engineering is what makes a malicious attachment convincing in the first place, dressing it up as an invoice, a job offer, or an urgent notice from a trusted contact. Because this method exploits psychology rather than software, no purely technical detection system can eliminate the risk entirely, which is precisely why TrustFile's educational component explaining to users why a file was flagged, rather than simply blocking it, was treated as just as important as detection itself.

Spyware: A category of malware designed to covertly monitor a user's activity and collect information browsing habits, login credentials, financial details typically transmitting what it gathers back to an attacker without the user's knowledge or consent. Keyloggers, discussed elsewhere in this chapter, represent one specific and particularly common form of spyware. Because spyware is built specifically to remain unnoticed for as long as possible, often mimicking legitimate background processes, it can operate undetected for extended periods, quietly undermining every other security measure a user has in place.

Static Analysis: The examination of a file's code and structural properties without executing the file in a runtime environment.

Suspicious Level: The classification assigned to a file that displayed something genuinely unusual with an entropy value somewhat higher than typical for its type, metadata inconsistencies, or origin from an unfamiliar source without clearly qualifying as either safe or outright dangerous. A SUSPICIOUS classification prompted additional verification steps before the file was made accessible, reflecting the reality that assessing file safety is not always a clear cut determination, and that the system sometimes had to weigh genuinely ambiguous evidence rather than reach an immediately confident conclusion.

Threat Classification: TrustFile's overall system for automatically sorting every scanned file into one of five risk categories BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS based on the combined results of file hashing, entropy analysis, script detection, size verification, file type verification, and external threat intelligence lookups. Rather than relying on any single check, the AI analysis engine weighed all of these indicators together, alongside the heuristic analysis component, to arrive at a classification and an accompanying confidence level. This five tier structure allowed both individual users and organizations to respond proportionally to the actual severity of what was detected, rather than treating every flagged file identically.

Threat Intelligence: Evidence based knowledge about known and emerging cyber threats including malware families, attacker techniques, and known malicious file hashes collected, maintained, and continuously updated by security researchers and global threat monitoring networks. TrustFile integrated with external, API based threat intelligence services that maintained these databases at a scale no individual organization could realistically replicate on its own, allowing the system to identify emerging threats that might not yet be reflected in its local analysis alone. This combination of local analysis and external intelligence formed the hybrid detection model described throughout this chapter, balancing the speed and privacy of local processing with the broader coverage that only large-scale, continuously updated threat intelligence can provide.

Trojan Horse: A category of malware that disguises itself as legitimate, desirable software while secretly carrying out malicious activity once installed or executed. Unlike a virus, a Trojan horse does not self replicate by attaching to other files; instead, it relies entirely on tricking a user into installing it directly, typically by presenting itself as a useful program, a cracked piece of paid software, or an urgent system update. Once active, a Trojan can perform virtually any malicious function installing additional malware, creating backdoor access for an attacker, or stealing data outright making it, as introduced in the Background of the Study, one of the most versatile and commonly encountered categories of malware in circulation.

Virus: Malicious software that attaches itself to legitimate files or programs in order to spread from one system to another, in a manner conceptually similar to how a biological virus requires a host to replicate. A virus works by inserting its own code into a legitimate file, carefully avoiding anything that would make the file obviously corrupted or immediately trigger detection, and ensuring its malicious code runs once the host file is opened. Viruses commonly spread through shared files email attachments, removable media, or downloads from compromised websites and because they typically require some user action, such as opening an infected file, to begin spreading, they can, in some respects, be easier to trace back to their origin than malware capable of propagating entirely on its own.

Worm: A category of malware capable of copying itself and spreading across computers and networks entirely on its own, without needing to attach to a host file or rely on any action from a user, a key distinction from viruses, discussed elsewhere in this chapter. Worms exploit weaknesses in network configurations, outdated software, and weak security settings to move from one system to the next, replicating and continuing to spread automatically once inside a new environment. This capacity for fully autonomous spread has driven some of history's most disruptive cyberattacks, including the Morris Worm of 1988, which infected roughly 6,000 computers within a single day.

Zero Day Exploits: Attacks that target a software vulnerability unknown to the vendor or security researchers at the time of the attack, meaning no patch, signature, or specific defense yet exists to stop it. The term reflects the fact that developers have had zero days to address the vulnerability before it was already being exploited. Because signature based detection depends entirely on a threat having already been identified and catalogued, it is structurally incapable of recognizing zero day threats, leaving a genuine window of maximum vulnerability until a defense is developed. This limitation, discussed at several points throughout this chapter, is a central reason TrustFile combined signature based hash comparison with heuristic analysis, entropy analysis, and AI driven reasoning, rather than depending on any single detection method to catch threats of this kind.

REFERENCE

[1] Guo, Y. (2023). A review of Machine Learning-based zero-day attack detection: Challenges and future directions. Computer Communications, 198, 175–185. https://doi.org/10.1016/j.comcom.2022.11.001 Cited by: 388

[2] Sommestad, T., Holm, H., & Steinvall, D. (2021). Variables influencing the effectiveness of signature-based network intrusion detection systems. Information Security Journal: A Global Perspective, 31, 711–728. https://doi.org/10.1080/19393555.2021.1975853 Cited by: 44

[3] Gupta, R., et al. (2026). "Email Attachment Malware Scanning: Architecture, Implementation." International Journal of Virtual Reality and Applications.

[4] Gibert, D., Mateu, C., Planes, J., & Vicens, R. (2018). Classification of Malware by Using Structural Entropy on Convolutional Neural Networks. Proceedings of the AAAI Conference on Artificial Intelligence, 32. https://doi.org/10.1609/aaai.v32i1.11409 Cited by: 131

[5] Lavieille, P., & Atlas, I. A. H. (2023). IsoEx: an explainable unsupervised approach to process event logs cyber investigation. arXiv. https://doi.org/10.48550/arxiv.2306.09260 Cited by: 3

[6] MITRE Corporation, "Masquerading: Double File Extension," MITRE ATT&CK, T1036.007, Aug. 2021. [Online]. Available: https://attack.mitre.org/techniques/T1036/007/. (Accessed: Sep. 25, 2026).

[7] Z. Bazrafshan, H. Hashemi, S. M. H. Fard, and A. Hamzeh, "A survey on heuristic malware detection techniques," in Proc. 5th Conf. Inf. Knowl. Technol. (IKT), Shiraz, Iran, 2013, pp. 113-120.

[8] M. Sikorski and A. Honig, Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software. San Francisco, CA, USA: No Starch Press, 2012, p. 9.

[9] Khalid, O., Ullah, S., Ahmad, T., et al. (2023). An Insight into the Machine-Learning-Based Fileless Malware Detection. Sensors, 23, 612. https://doi.org/10.3390/s23020612 Cited by: 89

CHAPTER II

REVIEW OF RELATED LITERATURE

2.1 A Comprehensive Review on Malware Detection Approaches

Aslan and Samet [1] conducted a comprehensive review of malware detection approaches published in IEEE Access, examining how malicious software continues to grow at an alarming rate and frequently conceals itself within systems through various obfuscation techniques [1]. Their study found that while signature based and heuristic based detection approaches are fast and efficient for identifying known malware, signature based methods in particular have failed to detect unknown threats [1]. In contrast, behavior based, model checking based, and cloud based approaches perform well against unknown and more complex malware, and emerging methods such as deep learning based, mobile device based, and IoT based approaches also show promise in detecting both known and unknown variants [1].

The study's examination of malware growth rates quantifies the expanding threat environment that motivates Trustfile's development [1]. Industry reports consistently document exponential increases in new malware samples, with millions of new variants detected annually. This growth results from multiple factors: the low cost and high profitability of malware development, the availability of malware as a service platforms that democratize attack capabilities, the automation of malware generation through polymorphic and metamorphic

engines, and the expanding attack surface created by IoT devices, cloud services, and remote work arrangements.

The focus on obfuscation techniques addresses a critical challenge for detection systems. Malware developers employ diverse obfuscation strategies to evade signature based detection, including code encryption, packing, polymorphism, metamorphism, anti debugging, anti virtualization, and steganography [1]. These techniques transform malware appearance between generations or during execution while preserving malicious functionality. Aslan and Samet's examination of these techniques [1] provides essential context for understanding why signature based methods struggle and why alternative approaches such as machine learning classification and heuristic analysis are necessary. The comparative findings regarding detection methodologies inform Trustfile's hybrid architecture design. Signature based detection, while fast and efficient for known threats, exhibits fundamental limitations against novel malware that lacks matching signatures in detection databases [1].

The temporal gap between malware emergence and signature creation, sometimes hours or days, creates windows of vulnerability [1]. Heuristic based detection improves upon signatures by identifying suspicious patterns and behaviors, but basic heuristics can be circumvented by sophisticated obfuscation and may generate false positives that erode user trust [1]. The superior performance of behavior based approaches reflects their focus on what malware does rather than what it looks like. By monitoring runtime behaviors, file system modifications, registry changes, network communications, and process injections, behavior based systems can identify malicious activity even from previously unseen malware variants [1]. However, behavior based detection requires execution environments that introduce risks, performance impacts, and complexity [1].

Model checking based approaches apply formal verification techniques to analyze program behaviors against security specifications, identifying violations that indicate malicious intent [1]. These methods offer strong theoretical foundations but face scalability challenges with complex software [1]. Cloud based approaches leverage distributed computing resources and collective intelligence to analyze files at scale, correlating observations across large user populations to identify emerging threats [1]. The emerging methods identified by Aslan and Samet [1]—deep learning based, mobile device based, and IoT based approaches—represent the cutting edge of malware detection research. Deep learning techniques, including convolutional neural networks, recurrent neural networks, and autoencoders, can automatically learn complex feature representations from raw data, potentially capturing subtle malware indicators that elude manual feature engineering [1].

The study's conclusion that these emerging methods show promise for detecting both known and unknown variants [1] directly supports Trustfile's incorporation of advanced detection techniques alongside traditional methods. The hybrid approach advocated by the literature, combining signature, heuristic, behavioral, machine learning, and intelligence based detection, maximizes coverage across threat categories while mitigating the limitations of any single methodology [1].

Source: Ö. A. Aslan and R. Samet, "A Comprehensive Review on Malware Detection Approaches," in IEEE Access, vol. 8, pp. 6249 6271, 2020, doi: 10.1109/ACCESS.2019.2963724.
Published in: IEEE Access (Volume: 8)
Date of Publication: 03 January 2020
Publisher: IEEE
Electronic ISSN: 2169 3536

2.2 Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach

Kamdan et al. [2] conducted a comprehensive study on static malware detection and classification using machine learning classifiers, specifically focusing on Random Forest, Decision Tree, and Support Vector Machine (SVM) algorithms. Published in the Proceedings of the 7th International Global Conference Series on ICT Integration in Technical Education & Smart Society, this research addresses the critical limitations of traditional signature based detection techniques in identifying polymorphic and zero day malware variants [2]. The researchers collected their dataset from MalwareBazaar, a widely recognized repository for malware samples, and extracted static features including PE headers, entropy values, and API calls to train their classification models [2]. Their experimental methodology involved rigorous data preprocessing, feature encoding, normalization, and model evaluation using standard metrics including accuracy, F1 score, AUC ROC, true positive rate, and false positive rate [2].

The significance of this study for Trustfile lies in its systematic comparison of multiple machine learning algorithms for static analysis based malware detection [2], which parallels Trustfile's rule-based heuristic classification of files into five threat levels, since both combine multiple static indicators to categorize threats.

The researchers' approach to feature extraction from file headers, entropy analysis, and API call examination [2] mirrors the multi-layered detection strategy employed by Trustfile. Their finding that Random Forest provided balanced results across all metrics with the shortest training time of 0.23 seconds [2] highlights the algorithm's computational efficiency, making it particularly suitable for real time web based scanning applications where response time is critical for user experience. Although SVM achieved the highest accuracy at 53.2% and Decision Tree obtained the best F1 score at 61.1% in their specific dataset configuration [2], the researchers emphasized that Random Forest offers an optimal trade off between performance and efficiency [2], which aligns with Trustfile's design priority of balancing comprehensive detection with practical deployment constraints.

The study's methodology for static feature extraction provides a validated framework that Trustfile can reference for its own feature engineering processes [2]. The researchers' use of PE headers, imported functions, and entropy values as discriminative features demonstrates how structural file characteristics can effectively distinguish malicious from benign files without requiring execution [2]. This approach validates Trustfile's decision to implement entropy analysis and file type verification as core detection components. Furthermore, the

study's emphasis on data preprocessing, including handling missing values, encoding categorical features, and normalizing numerical features [2], provides methodological guidance for preparing file data before classification, ensuring that the data passed to Trustfile's heuristic engine is consistent and reliable before classification.

The research also highlights important challenges that Trustfile must address, including the difficulty of detecting malware classes with limited data representation and the presence of overlapping features between different malware families that can confuse classification models [2]. These findings suggest that Trustfile's heuristic engine should use carefully weighted rules and thresholds so that rare or subtle threat types are not overlooked, ensuring robust detection across diverse file types. The study's conclusion that future work should explore ensemble methods or hybrid static dynamic analysis to combine the advantages of different classifiers [2] provides a clear roadmap for Trustfile's potential enhancement, supporting the system's current hybrid approach of combining local static analysis with external API based intelligence.

Source: Kamdan; Pratama, Y.; Munzi, R.S.; Mustafa, A.B.; Kharisma, I.L. Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach. Eng. Proc. 2025, 107, 76. https://doi.org/10.3390/engproc2025107076
Published in: Engineering Proceedings (Volume 107)
Date of Publication: 9 September 2025
Publisher: MDPI
Electronic ISSN: 2673 4591

2.3 Enhanced Android Malware Detection Using Deep Learning and Ensemble Techniques

Museeb et al. [3] presented a comprehensive machine learning framework for malware detection that systematically compares deep neural networks with diverse ensemble methods including Voting Ensemble, Stacking Ensemble, XGBoost, and Random Forest. Published in the International Journal of Advanced Computer Science and Applications (IJACSA), this study addresses the evolving tactics of Android malware that frequently bypass traditional detection systems through obfuscation and novel variant creation [3]. The researchers trained their models using the CIC AndMal2017 dataset, incorporating a comprehensive set of static features including API calls, permissions, services, receivers, and activities [3]. Feature selection was performed to optimize model performance, reducing redundancy and improving detection accuracy through systematic elimination of irrelevant or correlated features [3].

The study's methodology provides valuable insights for the design of Trustfile's classification process. The researchers' use of multiple ensemble configurations demonstrates how combining diverse classifiers can achieve superior performance while maintaining computational efficiency [3]. Their finding that the Voting Ensemble model achieved the highest accuracy of 94.14%, outperforming all other approaches including the deep neural network [3], validates the effectiveness of ensemble methods in malware detection. This result is

particularly significant for Trustfile because it demonstrates that carefully constructed ensembles of diverse classifiers can improve detection accuracy while offering a more scalable, lightweight solution compared to complex deep learning models [3], which aligns with Trustfile's web based architecture that prioritizes accessibility and responsiveness.

The feature selection methodology employed by Museeb et al. [3] offers practical guidance for Trustfile's system design. Their approach to reducing feature redundancy while preserving discriminative power through systematic selection processes ensures that the classification model focuses on the most informative indicators of malicious behavior [3]. The study's evaluation across multiple classification metrics, including accuracy, F1 score, and confusion matrices [3], provides a comprehensive framework for assessing Trustfile's own classification performance. The researchers' conclusion that ensemble methods can improve detection accuracy while offering computational efficiency [3] directly supports Trustfile's design philosophy of balancing comprehensive threat detection with practical usability for non technical users.

The comparative analysis between deep learning and ensemble methods is particularly relevant for Trustfile's development trajectory. While deep learning models offer powerful feature learning capabilities, the study demonstrates that they may not always outperform well constructed ensemble methods, especially when computational resources are constrained [3]. This finding suggests that Trustfile's current approach of weighing several independent static indicators through a rule-based heuristic engine, much like an ensemble combines several classifiers into one verdict, provides a solid foundation that can be enhanced incrementally, for example by adding a trained classifier in future work, rather than requiring immediate adoption of complex deep learning architectures. Rather than requiring immediate adoption of complex deep learning architectures. The study's emphasis on maintaining computational efficiency while achieving high accuracy [3] aligns with Trustfile's requirement for fast, responsive web based scanning that does not overwhelm users with extended wait times.

Source: Abdul Museeb, Yaman Hamed, Rajalingam Sokkalingam, Anis Amazigh Hamza, Atta Ullah and Iliyas Karim Khan. "Enhanced Android Malware Detection Using Deep Learning and Ensemble Techniques". International Journal of Advanced Computer Science and Applications (ijacsa) 16.11 (2025).
Published in: International Journal of Advanced Computer Science and Applications (IJACSA), Volume 16, Issue 11
Date of Publication: 2025
Publisher: The Science and Information Organization (SAI)
Electronic ISSN: 2156 5570

2.4 Static Malware Analysis Using Optimal Machine Learning Algorithm for Malware Detection

Buradkar [4] conducted research on static malware analysis using optimal machine learning algorithms for malware detection, providing methodological insights that are particularly relevant for Philippine organizations with limited cybersecurity resources. Published in NeuroQuantology, this study aimed to find the optimal

machine learning algorithm to predict whether a file is malicious or not by implementing and comparing different machine learning models using Python libraries [4]. The research analyzed 4,386 records of portable executable attributes for malware classification, identifying critical file characteristics that could be used for effective detection [4]. Portable executable (PE) attributes are the structured header fields found in Windows executable files, describing how a program is laid out and loaded into memory. Because these fields are stored inside the file itself, they can be read and examined without ever running the program, which is exactly what makes them suitable for static analysis.

The study's findings have direct implications for Trustfile's implementation in the Philippine context. The research identified that Random Forest achieved the highest accuracy of 97.99% for malware detection among tested models, significantly outperforming Logistic Regression and Support Vector Machines [4]. Instead of relying on a single decision tree, a Random Forest combines the votes of many trees, which generally makes its predictions more stable and less sensitive to noise in the data. This result shows that high accuracy can be achieved with algorithms that are computationally efficient and relatively simple to implement, which supports Trustfile's decision to favor lightweight, efficient analysis and offers a possible model for future work on learned classifiers. For Philippine organizations with limited technical resources, this finding suggests that sophisticated detection capabilities need not require complex infrastructure or expensive commercial solutions.

The feature analysis conducted in this study provides specific guidance for Trustfile's detection methodology. The research highlighted critical file attributes for malware detection, including IMAGEBASE, SIZEOFSTACKRESERVE, and CHECKSUM, as particularly significant indicators [4]. IMAGEBASE refers to the preferred memory address at which an executable is loaded, SIZEOFSTACKRESERVE to the amount of memory reserved for the program's stack, and CHECKSUM to an integrity value stored in the file header. Malicious files often show unusual or inconsistent values in these fields because they are modified, packed, or assembled differently from ordinary software. These PE header features are exactly the type of static file characteristics that Trustfile examines through its file type verification and structural analysis components. The study's finding that combining machine learning with heuristic approaches improves malware detection accuracy [4] supports Trustfile's hybrid approach of combining multiple analysis layers with rule based heuristic analysis.

The dataset characteristics described in this research, 4,386 records balancing malicious and non malicious files [4], demonstrate the scale of data required for effective machine learning model training. This provides guidance for how Trustfile's own rules and thresholds are validated, suggesting that an adequate number and balance of benign and malicious sample files are essential for achieving reliable classification performance. A test set dominated by one type of file would make the results look better or worse than they truly are, so a balanced collection is needed to judge detection and false positive rates fairly. The study's use of Python libraries for implementation [4] aligns with Trustfile's technology stack, validating the feasibility of developing effective detection capabilities using accessible, open source tools.

The research's conclusion that the optimal machine learning model for static malware detection can serve as a layer in a larger neural network for dynamic malware analysis and attack detection [4] suggests a pathway for Trustfile's future enhancement. The current system focuses on static analysis for pre access scanning, but this study indicates that the same machine learning foundation can be extended to more sophisticated analysis as organizational needs and capabilities evolve. In this way, Buradkar's work supports both the present design of Trustfile, which relies on lightweight and explainable static checks, and its longer term direction, in which behavioral and learning based techniques may be added without discarding the existing foundation.

Source: Buradkar, Mrunalini U. "Static Malware Analysis Using Optimal Machine Learning Algorithm for Malware Detection." NeuroQuantology, Vol. 20, Issue 10, 2022, pp. 4128 4141.
Published in: NeuroQuantology, Volume 20, Issue 10
Date of Publication: August 2022
Publisher: NeuroQuantology
Electronic ISSN: 1303 5150

2.5 Analyzing and Comparing the Effectiveness of Malware Detection Using Machine Learning with Entropy Based Feature Selection

Azeem et al. [5] examined the effectiveness of malware detection using machine learning with entropy based feature selection, providing methodological insights that are directly applicable to the Philippine cybersecurity context. Published in Heliyon, this research addressed the challenge of feature selection in malware detection, where the goal is to choose a subset of features from the list of capabilities for dimensional characteristics to classify data as precisely and efficiently as possible [5]. In practice, a malware dataset can contain hundreds or thousands of measurable file characteristics, and many of them are redundant or add noise instead of useful information. Selecting only the informative ones makes a classifier faster and often more accurate.

The study employed Information Gain (IG) and Gain Ratio (GR) entropy based approaches to evaluate features and remove less useful features from the dataset, optimizing model performance while reducing computational complexity [5]. Information Gain measures how much uncertainty about a file's class (malicious or benign) is removed when a particular feature is known, while Gain Ratio adjusts this value so that features with many possible values are not unfairly favored. The study's methodology for entropy based feature selection has particular relevance for Trustfile's implementation in resource constrained Philippine environments. By employing entropy based techniques to identify the most discriminative features, the researchers demonstrated that machine learning models can achieve high detection accuracy without requiring extensive computational resources [5]. This approach aligns with Trustfile's design priority of creating a system that operates efficiently within the infrastructure limitations typical of Philippine organizations. The use of Information Gain and Gain Ratio for feature evaluation provides a validated methodology that Trustfile can draw on when deciding which file indicators deserve the most weight in its rules, ensuring that the classification engine focuses on the most informative indicators while minimizing processing overhead.

The research's emphasis on reducing dimensionality while maintaining classification accuracy [5] supports Trustfile's goal of efficient, real time scanning. The study's finding that optimal feature extraction eliminates unnecessary features and accelerates model performance [5] demonstrates that careful feature engineering can achieve both accuracy and efficiency. This is particularly important for web based systems like Trustfile, where user experience depends on rapid response times, since a scan that takes too long discourages users from checking files before opening them. The entropy based approaches described in this research, which calculate the reduction in total entropy when features are included [5], provide a mathematically rigorous framework for feature evaluation that Trustfile can incorporate into its rule development.

The study's integration of feature selection with machine learning classification [5] validates Trustfile's approach of combining multiple analytical techniques. By demonstrating that entropy based feature selection improves the performance of machine learning classifiers [5], this research supports Trustfile's hybrid methodology that combines entropy analysis with rule based heuristic analysis. The mathematical foundations of entropy calculation described in this study, which use Shannon entropy to measure information content and information gain to quantify feature importance [5], provide theoretical grounding for Trustfile's detection algorithms. Shannon entropy is also the same measure used to judge whether a file's contents look unusually random, a common sign of packed or encrypted malware, so the concept links the study's feature selection method directly to the entropy checks that Trustfile performs on uploaded files. The study's conclusion that Random Forest yielded the best accuracy of 97.68% using TFIDF based entropy feature selection [5] further supports the choice of entropy as a core indicator in Trustfile's classification engine.

Source: Azeem, M., Khan, D., Iftikhar, S., Bawazeer, S., & Alzahrani, M. (2023). Analyzing and comparing the effectiveness of malware detection using machine learning with entropy based feature selection. Heliyon, 9(12), e187275. https://doi.org/10.1016/j.heliyon.2023.e187275
Published in: Heliyon, Volume 9, Issue 12
Date of Publication: 2023
Publisher: Cell Press (Elsevier)
Electronic ISSN: 2405 8440

2.6 RATTATA: Ransomware Analysis via Automated Impact Assessment of Philippine Organizations

Abdon [6] conducted a study on ransomware experiences and impacts in Philippine organizations across different sectors, published in the International Journal of Intelligent Systems and Applications in Engineering. The research observes that the Philippines is particularly lacking in cybersecurity investment, with a misconception of its insignificance despite looming and ever growing threats [6]. This underinvestment has caused major data leakages and compromised services in recent years [6]. A study set in the Philippines is especially valuable for this review because it describes the conditions, constraints, and risks of the same environment in which Trustfile is intended to be used, instead of relying only on findings from organizations with much larger security budgets.

The focus on ransomware reflects this malware category's particular relevance to Philippine organizations and its alignment with Trustfile's protective objectives. Ransomware is malicious software that holds a victim's files or computer hostage until a ransom is paid, and it has emerged as one of the most financially devastating cyber threats globally, with attacks increasing in frequency, sophistication, and impact. The persistence and evolution characteristics described in the local study highlight the adaptive nature of ransomware operators, who continuously refine their techniques in response to defensive measures [6]. This arms race necessitates equally adaptive detection and prevention capabilities, such as those provided by Trustfile's rule based heuristic analysis, which looks at how a file is built and behaves instead of depending only on a list of previously known threats.

The dual attack mechanisms, system blocking and data encryption, represent the operational modalities through which ransomware achieves its objectives [6]. System blocking prevents users from accessing computing resources, displaying ransom demands and payment instructions. Data encryption renders files unreadable without decryption keys held by attackers, creating strong incentives for ransom payment. The combination of these mechanisms maximizes pressure on victims while demonstrating attacker capability. For a small office or school without a dedicated security team, either mechanism alone can halt daily operations, which is why stopping a malicious file before it is opened is far more practical than trying to recover afterward.

The organizational impact extends beyond immediate financial losses from ransom payments to encompass operational disruption, reputational damage, regulatory penalties, and long term recovery costs. Business interruption during ransomware incidents can last days or weeks, with critical systems offline and manual workarounds implemented. Data exfiltration by attackers, meaning the theft of sensitive information before encryption, adds extortion leverage through threats of public disclosure, transforming ransomware into data breach incidents with regulatory implications. The Philippine cybersecurity landscape assessment reveals systemic challenges that amplify ransomware and broader malware threats [6]. The identified investment deficiency reflects both public and private sector underfunding of cybersecurity infrastructure, personnel, training, and research [6]. This underinvestment manifests in outdated security technologies, insufficient security staffing, inadequate incident response capabilities, and limited threat intelligence resources.

The consequences of this underinvestment are tangible and severe. Major data leakages compromise citizen privacy, organizational secrets, and national security information. Compromised services disrupt government operations, business transactions, healthcare delivery, and educational activities. These incidents damage public trust in digital services, potentially slowing digital transformation initiatives that could otherwise drive economic and social development. Organizations that cannot afford enterprise security products are therefore left relying on basic antivirus software and individual user caution, which leaves a gap that low cost and accessible tools can help to fill.

The identified opportunity for deeper investigation supports the research motivation for Trustfile [6]. Ransomware commonly arrives as a downloaded file or an email attachment that a user opens without

suspecting anything, so the moment before a file is opened is one of the most effective points at which to intervene. Trustfile's static analysis checks, such as hashing, entropy analysis, and file type verification, are designed to work at this point, flagging files that appear packed, disguised, or structurally inconsistent before they are ever executed. Its plain language explanations also give users the knowledge to understand why a file is risky, addressing the knowledge deficits that the local study associates with the Philippine landscape.

By developing and evaluating a practical file security solution within the Philippine context, this study contributes to addressing the documented cybersecurity gaps [6]. The system's focus on pre-access scanning, user education, rule based heuristic analysis, and accessible design specifically addresses the resource constraints and knowledge deficits identified in the local landscape.

Source: Aaron Lensmer Abdon. (2024). RATTATA: Ransomware Analysis via Automated Impact Assessment of Philippine Organizations. International Journal of Intelligent Systems and Applications in Engineering, 12(4), 3430. https://ijisae.org/index.php/IJISAE/article/view/6858
Published in: International Journal of Intelligent Systems and Applications in Engineering, Volume 12, Issue 4
Date of Publication: 2024
Publisher: IJISAE
Electronic ISSN: 2147 6799

2.7 Harnessing AI and Analytics to Enhance Cybersecurity and Privacy for Collective Intelligence Systems

Naeem et al. [7] demonstrate a novel application of deep learning to malware detection through image representation techniques. Published in Big Data and Cognitive Computing, this approach transforms the malware detection problem from binary analysis into image classification, leveraging the powerful capabilities of convolutional neural networks (CNNs) developed for computer vision applications [7]. The idea behind this method is that files from the same malware family tend to share a similar internal structure, so when they are converted into pictures they produce similar visual textures and patterns. A CNN, which is designed to recognize such patterns in images, can then learn to separate malicious files from benign ones without any need to execute them.

The grayscale image construction process involves interpreting binary file contents as pixel intensity values, creating two dimensional image representations that encode structural and statistical features of the original files [7]. Each byte in a file holds a value from 0 to 255, which maps naturally to a shade of gray from black to white. This transformation is non trivial and requires careful consideration of image dimensions, byte to pixel mapping strategies, and normalization techniques. Common approaches include reading file bytes sequentially to fill image rows, with image width determined by file size or fixed parameters. The resulting images capture patterns including code sections, data segments, padding, and structural regularities that may distinguish malware from benign software. For example, long runs of padding appear as flat uniform areas, while compressed or encrypted sections appear as dense, noisy textures.

Feature extraction through deep learning automates the identification of discriminative patterns without manual feature engineering [7]. Traditional machine learning approaches require security experts to identify and quantify relevant features, such as byte frequencies, instruction sequences, API calls, and string patterns, creating features that may miss novel indicators or incorporate researcher biases. Deep learning models learn hierarchical feature representations directly from raw image data, potentially capturing subtle patterns invisible to human analysts. In a CNN, the early layers typically detect simple local patterns, while deeper layers combine them into more complex shapes that characterize an entire malware family. The trade off is that these learned features are difficult for a person to inspect, which makes it harder to explain to a user why a particular file was flagged.

The dataset scale, 6,414 malware variants and 2,050 benign files from the MalImg collection [7], provides substantial training and validation material. The MalImg dataset is a well known benchmark in malware image analysis research, containing grayscale images of malware files categorized by family. This dataset enables reproducible research and comparative evaluation across studies. The class distribution, with malware outnumbering benign files, reflects real world prevalence while requiring careful handling to prevent class imbalance from biasing model performance. A model trained on mostly malicious samples may learn to label nearly everything as malicious unless this imbalance is corrected, which would produce many false alarms in practical use.

The reported 99.86 percent validation accuracy [7] represents exceptional performance, calculated on held out validation data not used during training, indicating strong generalization capability. However, accuracy alone provides incomplete performance assessment, as it can be misleading in imbalanced datasets or when false negative costs exceed false positive costs. Comprehensive evaluation would additionally report precision, recall, F1 score, and area under the ROC curve. Results from a single benchmark dataset should also be interpreted with caution, because real files encountered by users may differ from those in a curated collection and because malware authors can modify their files to change how they appear when converted to images.

The implications for Trustfile are significant. While Trustfile’s current scope focuses on static analysis techniques including hashing, entropy analysis, script detection, and rule based heuristic analysis, the demonstrated effectiveness of deep learning image analysis suggests a promising enhancement path [7]. Future iterations could incorporate CNN based classification as an additional detection layer, potentially improving accuracy against novel threats while providing intuitive visualization capabilities. A visual representation of a file could also help users and analysts see where unusual or suspicious regions are located, supporting Trustfile’s goal of educating users about the risks they face. At the same time, CNN models require substantial training data and more computing resources than rule based checks, so any such addition would need to be weighed against Trustfile’s priority of fast, accessible, web based scanning.

The image based approach is also conceptually close to the entropy analysis that Trustfile already performs. Both treat a file as a sequence of bytes whose patterns reveal whether it has been packed, encrypted, or

otherwise altered, and the dense, noisy regions seen in a grayscale image correspond to the high entropy regions that entropy analysis measures numerically. The high accuracy achieved in the study demonstrates that sophisticated detection is feasible and effective [7], motivating continued investment in advanced detection capabilities, including machine learning and the continued refinement of Trustfile's heuristic analysis.

Source: Naeem, M. R., Amin, R., Farhan, M., Alotaibi, F. A., Alnfiai, M. M., Sampedro, G. A., & Karovič, V. (2024). Harnessing AI and analytics to enhance cybersecurity and privacy for collective intelligence systems. Journal of Big Data and Cognitive Computing, 8(2). https://doi.org/10.3390/jbdcc8020042
Published in: Journal of Big Data and Cognitive Computing, Volume 8, Issue 2
Date of Publication: 2024
Publisher: MDPI
Electronic ISSN: 2504 2289

2.8 Leveraging CNNs and Entropy Based Feature Selection to Identify Potential Malware Artifacts of Interest

Wee Ki Joon [8] conducted pioneering research on integrating entropy driven feature selection with specialized convolutional neural network architectures for multi class malware classification, published as a guest diary through the SANS Internet Storm Center. This research addresses the increasing obfuscation tactics employed by modern malware authors, focusing on capturing high entropy segments within files that are most likely to harbor malicious functionality [8]. The methodology employs a sliding window mechanism that computes entropy in overlapping chunks, locating suspicious high entropy hotspots without being limited by fixed size segments, then feeds these distinct byte patterns into a multi layer CNN for classification [8]. Multi class classification means that the model does not only decide whether a file is malicious, but also assigns it to a specific type of malware, which gives analysts more useful information than a simple yes or no verdict.

The study's findings have direct implications for Trustfile's entropy analysis implementation. The researchers achieved approximately 91% overall accuracy on a held out test set of 11,442 malware samples from the BODMAS dataset, with the model maintaining strong performance across varied malware families including trojans, worms, ransomware, backdoors, downloaders, droppers, and information stealers [8]. Notably, the model achieved 96.6% accuracy for backdoors, 95.4% for worms, 94.7% for viruses, and 88.4% for ransomware [8], demonstrating that entropy based feature selection effectively identifies diverse threat categories. A held out test set consists of samples that the model never saw during training, so these figures indicate how well the approach is likely to perform on new files. The lower result for ransomware also shows that detection is not equally easy for every malware type, which is why relying on a single indicator carries risk.

The entropy based sliding window extraction methodology provides specific technical guidance for Trustfile's entropy analysis module. Rather than calculating a single entropy value for an entire file, the sliding window approach computes entropy across overlapping segments, identifying localized high entropy regions that may indicate packed, encrypted, or obfuscated code [8]. This approach is particularly effective against modern

malware that may mix low entropy legitimate code with high entropy malicious payloads, as it locates suspicious regions without requiring manual segmentation. To illustrate, a file that is mostly ordinary, low entropy code but contains a small encrypted payload would produce an unremarkable average entropy if measured as a whole, since the payload is diluted by the rest of the file. Examining the file in overlapping windows exposes the payload as a distinct peak. The dynamic thresholding of high entropy regions ensures the model focuses on byte level segments most indicative of malicious behavior [8], a technique that Trustfile can adapt for its own entropy based threat detection.

The study's discussion of misclassification challenges offers important insights for Trustfile's classification system design. The researchers identified that categories like droppers and downloaders showed overlapping features, with both exhibiting high recall and low precision, indicating that the model misclassifies other samples as these categories [8]. High recall means that most files truly belonging to a category are found, while low precision means that many files placed in that category do not actually belong to it. This observation highlights the importance of feature distinctiveness in classification performance and suggests that Trustfile's multi indicator approach, combining entropy with hashing, script detection, heuristic analysis, and file type verification, provides robustness against the confusion that single feature systems may experience. When one indicator is ambiguous, the others can confirm or contradict it, reducing the chance of a wrong verdict. The study's conclusion that targeted misclassification challenges emphasize the need for continuous refinement of feature engineering [8] directly supports Trustfile's iterative development approach.

The practical implementation details, including the use of batch normalization, early stopping, and class weighting to ensure balanced learning [8], provide methodological guidance for how Trustfile's own rules and thresholds are tuned and validated. Batch normalization stabilizes the training of a neural network, early stopping halts training before the model begins to memorize its training data, and class weighting prevents categories with many samples from overwhelming those with few. The researchers' emphasis on rigorous data preprocessing, including filtering out spurious files and employing stratified sampling with class weight adjustments [8], demonstrates the importance of data quality in achieving reliable detection performance. These practices inform Trustfile's dataset construction and validation procedures, because any evaluation of its detection rules is only as trustworthy as the sample files used to test them.

Taken together, the study reinforces the central role of entropy in Trustfile's design. It shows that measuring randomness locally, and not only across an entire file, is an effective way to expose hidden or packed content, and it cautions that no single indicator is sufficient for every malware type. Both lessons are consistent with Trustfile's combination of entropy analysis with complementary static checks.

Source: Wee Ki Joon. "Leveraging CNNs and Entropy Based Feature Selection to Identify Potential Malware Artifacts of Interest." SANS Internet Storm Center Guest Diary, March 26, 2025. https://isc.sans.edu/diary/31790
Published in: SANS Internet Storm Center Guest Diary
Date of Publication: March 26, 2025

Publisher: SANS Institute
Electronic ISSN: N/A (Web based technical publication)

2.9 Entropy Analysis: A Critical Test for Malware Detection

Mheboobkhan [9] published a comprehensive analysis of entropy as a critical test for malware detection, explaining why entropy analysis serves as an essential complement to traditional indicator based detection methods. This research addresses the fundamental question of why entropy analysis is necessary even when Indicators of Compromise (IOCs) and hash values are available, demonstrating that entropy provides proactive threat identification capabilities that reactive signature based methods cannot achieve [9]. IOCs are pieces of forensic evidence, such as file hashes, IP addresses, or domain names, that show a system may have been compromised. A hash value is a fixed length fingerprint of a file's contents, and changing even a single byte of the file produces a completely different hash, which is why hash based detection can be avoided by malware that is modified slightly.

The study's central argument directly supports Trustfile's inclusion of entropy analysis as a core detection component. The author explains that while hash values and IOCs can identify known threats, they are inherently reactive, only useful for threats that have already been identified, analyzed, and documented [9]. Entropy analysis, in contrast, can prevent threats in advance by identifying and studying entropy patterns associated with malware behavior [9]. This proactive capability is particularly valuable against zero day threats and novel variants that have not yet been cataloged in threat databases. The study notes that this technique is widely used in machine learning antivirus detection [9], validating its effectiveness in modern security architectures. For Trustfile, whose users may not have access to up to date commercial threat feeds, a technique that does not depend on prior knowledge of a specific threat is especially important.

The practical examples provided in this research demonstrate entropy's diagnostic value for analysts. The author compares entropy values of PDF files before and after compression, noting that entropy increases after compression [9]. This happens because compression removes redundancy and predictable patterns from the data, so the remaining bytes appear more random. This example illustrates how entropy changes reflect file transformation processes, with elevated entropy indicating compression, encryption, or other operations that reduce data predictability. For malware detection, similar entropy elevations indicate packing, encryption, or obfuscation techniques commonly employed to evade signature detection. Packing, for instance, compresses or encrypts the real program inside a wrapper so that its true contents cannot be matched against known signatures, and the resulting file shows unusually high entropy.

The study's explanation of entropy as a time saving and methodology guiding tool for analysts resonates with Trustfile's educational objectives. The author notes that analysts can quickly check entropy patterns to determine whether a file is obfuscated, saving time and helping them choose appropriate analysis methodologies [9]. This practical application aligns with Trustfile's goal of helping users understand why a file is risky, since suspicious entropy values provide concrete, interpretable indicators that can be explained to non-technical users. A

statement such as “this file’s contents look unusually random, which is common in packed or encrypted programs” is far easier for an ordinary user to understand than a technical signature name. The study’s emphasis on entropy as a decision support tool rather than merely a detection mechanism [9] supports Trustfile’s approach of using technical indicators for user education.

The integration of entropy analysis with machine learning described in this research suggests a path for strengthening Trustfile’s classification over time. The author notes that entropy techniques are widely used in machine learning antivirus detection [9], indicating that entropy values can serve as effective features for automated classification models. This observation supports Trustfile’s use of entropy as a direct detection indicator within its rule based heuristic analysis, where entropy values contribute to the overall threat assessment alongside other indicators. It also suggests that, in future work, the entropy values that Trustfile already computes could be used as input features for a trained classifier without changing how the files are analyzed.

It is also important to recognize the limits of entropy as an indicator. High entropy by itself does not prove that a file is malicious, because many legitimate files, such as ZIP archives, JPEG images, video files, and software installers, are compressed by design and naturally show high entropy. For this reason, entropy is best treated as a prompt for closer inspection and not as a final verdict. This is the reason Trustfile combines entropy with hashing, script detection, file type verification, and heuristic analysis, so that a high entropy value is judged in the context of the file’s type, structure, and other characteristics before it is reported to the user as a threat.

Source: Mheboobkhan. "Entropy Analysis: A Critical Test for Malware's." Medium, February 13, 2019. https://whiteheart0.medium.com/entropy analysis a critical test for malwares 69939f5b8b1
Published in: Medium
Date of Publication: February 13, 2019
Publisher: Medium
Electronic ISSN: N/A (Online blog article)

2.10 Cybersecurity for All: Why Every Employee Matters in Data Protection

TechClass [10] published comprehensive research on cybersecurity awareness for non technical teams, emphasizing that human error is the root cause in 95% of cybersecurity breaches. This study addresses the critical need for security education that extends beyond technical teams to encompass all organizational personnel who interact with digital systems and files [10]. The research identifies common cyber threats for non technical employees including phishing, weak passwords, insider mistakes, business email compromise, and physical security lapses, demonstrating that these threats exploit human behavior rather than technical vulnerabilities [10]. This perspective matters for a file security tool, because the most advanced detection technology can be defeated if the person using it is persuaded to ignore a warning, open an attachment, or disable a protection.

The study's findings regarding non-technical user vulnerabilities directly support Trustfile's educational design philosophy. The researchers identify that phishing emails or fraudulent messages target employees across all departments, attempting to trick them into divulging credentials or clicking malicious links [10]. Phishing alone was responsible for approximately 12% of breaches in one study, making it the second most common entry method after stolen logins [10]. Social engineers pose as bosses, vendors, or IT support personnel to manipulate unwary staff, exploiting trust relationships that non technical users may not scrutinize critically [10]. Social engineering works because it targets natural tendencies such as the wish to be helpful, to obey authority, or to respond quickly to something that appears urgent. These findings validate Trustfile's emphasis on explaining file threats clearly, as users who understand what makes files dangerous are better equipped to recognize and resist social engineering attacks.

The research on business email compromise and fraud highlights specific file related risks that Trustfile addresses. Cybercriminals target finance departments and executives by impersonating senior leaders or suppliers, sending convincingly forged emails instructing wire transfers or requesting sensitive documents [10]. Employees in accounting, HR, or executive assistance roles receive these fake requests, and without verification protocols, companies have been defrauded of millions through BEC scams that started with an employee trusting a spoofed email [10]. These messages often include an attachment, such as a fake invoice, purchase order, or payment form, which may carry malicious code or lead to a credential stealing page. Trustfile's pre access scanning and clear result explanations provide a verification mechanism that can interrupt such attack chains by identifying malicious attachments before users act on fraudulent instructions.

The study's recommendations for building security aware cultures inform Trustfile's user education approach. The researchers recommend providing regular, engaging training; encouraging open incident reporting; leading by example; integrating security into daily processes; and running creative awareness initiatives [10]. These principles translate into Trustfile's design through embedded educational content that transforms each scanning interaction into a micro learning opportunity, clear explanations that build user knowledge incrementally, and actionable guidance that users can apply to future file encounters. Because the explanation appears at the exact moment a user is deciding whether to trust a file, the lesson is immediately relevant and more likely to be remembered than a lecture delivered at an unrelated time. Integrating security into the daily act of checking a file also fits the recommendation that security should be part of ordinary work and not a separate, occasional activity.

The emphasis on gamification and interactive simulations for security training suggests enhancement opportunities for Trustfile. The study notes that when training feels like a chore, employees are less likely to retain critical information [10]. Features like gamification and AI driven insights allow leadership to track real time progress and identify potential security gaps [10]. While Trustfile's current scope focuses on functional scanning and education, these findings suggest future enhancements could include progress tracking, achievement systems, and personalized learning paths that adapt to individual user knowledge levels. For example, a user who repeatedly encounters a certain type of risky file could be shown short, targeted tips about

that threat, and an organization could use summary progress information to see which topics need more attention.

Taken together, the study shows that technical detection and user awareness are complementary and not interchangeable. A scanner can identify and explain a threat, but only an informed user will act on that information, and an informed user still benefits from a tool that catches what human judgment misses. This supports the dual nature of Trustfile as both a detection system and an educational resource. It also indicates the boundary of the system's contribution, since Trustfile addresses risks at the level of individual files and does not replace broader organizational measures such as password policies, incident reporting procedures, and regular security training.

Source: "Cybersecurity for All: Why Every Employee Matters in Data Protection." TechClass Resources, April 25, 2025. https://www.techclass.com/resources/learning and development articles/cybersecurity for non tech teams why everyone plays a role in data protection
Published in: TechClass Resources
Date of Publication: April 25, 2025
Publisher: TechClass
Electronic ISSN: N/A (Online resource article)

2.11 Synthesis of the Study

The reviewed literature and studies collectively illuminate a profound and accelerating paradigm shift in malware detection, moving decisively from traditional, reactive signature-based methods toward proactive, data-driven intelligence architectures that integrate multiple analytical techniques, machine learning classification, and comprehensive user education.

This transformation is not merely technological but represents a fundamental reconceptualization of how cybersecurity systems should be designed, deployed, and experienced by end users. The convergence of findings across foreign studies, local Philippine research, and specialized technical literature establishes a comprehensive theoretical and empirical foundation for the development of Trustfile as a web-based file security platform that addresses documented gaps in current detection methodologies while prioritizing accessibility, transparency, and user empowerment.

The Foundational Limitations of Traditional Detection and the Imperative for Hybrid Architectures.[1] Aslan and Samet establish the foundational problem that motivates this entire body of research: the systematic inadequacy of signature-based detection approaches in confronting modern malware threats. Their comprehensive review documents how malware continues to proliferate at an alarming rate, with millions of new variants emerging annually through polymorphic engines, malware-as-a-service platforms, and increasingly sophisticated obfuscation techniques including code encryption, packing, metamorphism, and steganography. The critical insight from this study is that signature-based methods, while fast and efficient for known threats,

possess an inherent temporal vulnerability—the window between malware emergence and signature database updates creates periods of maximum exposure during which systems remain completely unprotected against novel threats.

The significance of Aslan and Samet's findings for Trustfile is multifaceted. First, their documentation of detection approach evolution—from signature-based through heuristic, behavioral, model checking, and cloud-based methods to emerging deep learning, mobile, and IoT approaches—establishes a clear trajectory that Trustfile follows. The study's conclusion that no single approach can detect all malware in the wild, combined with its observation that different methods excel in different contexts, provides direct justification for Trustfile's hybrid architecture that combines multiple detection paradigms. Second, their identification of deep learning as a promising emerging method supports Trustfile's incorporation of machine learning classification alongside traditional static analysis techniques. Third, their emphasis on the ongoing arms race between malware sophistication and detection capability establishes the need for systems that can adapt and evolve, a requirement addressed by Trustfile's API integration with continuously updated external threat intelligence.

The local Philippine context, as documented by Abdon [6], amplifies the urgency of addressing these foundational limitations. The Philippines faces a particularly severe manifestation of the global malware threat due to the convergence of widespread internet adoption, limited cybersecurity investment, and a cultural misconception of security insignificance that leaves organizations systematically underprepared. The documented surge in ransomware attacks targeting Philippine finance, government, healthcare, education, and retail sectors demonstrates that the theoretical vulnerabilities identified by Aslan and Samet manifest as concrete, devastating harms in the local context. This empirical evidence of active, ongoing targeting validates Trustfile's focus on pre-access file scanning as a preventive measure, particularly for resource-constrained Philippine organizations that cannot afford expensive commercial security suites or dedicated security teams.

Machine Learning Classification: From Theoretical Promise to Practical Implementation. The reviewed literature demonstrates a clear evolution from acknowledging machine learning's potential to achieving validated, deployable classification systems. Kamdan et al [2]. Provide the most direct methodological guidance for Trustfile's supervised classification implementation through their systematic comparison of Random Forest, Decision Tree, and Support Vector Machine algorithms for static malware detection. Their finding that Random Forest achieved balanced performance across all metrics with the shortest training time of 0.23 seconds is particularly significant for Trustfile's web-based architecture, where computational efficiency directly impacts user experience and operational viability. The study's use of PE headers, entropy values, and API calls as discriminative features validates Trustfile's multi-layered detection strategy, while their emphasis on data preprocessing provides methodological standards for preparing file inputs before classification.

Museeb et al [3]. extend this validation by demonstrating that ensemble methods can outperform individual deep neural networks while maintaining superior computational efficiency. Their Voting Ensemble achieving 94.14% accuracy, surpassing all other approaches including complex deep learning models, provides compelling evidence that carefully constructed ensembles of diverse classifiers offer an optimal balance of performance and

practicality. This finding is transformative for Trustfile because it suggests that sophisticated detection need not require expensive infrastructure or specialized hardware—ensemble methods can operate effectively on standard web servers, making advanced protection accessible to organizations with limited resources.

Buradkar [4] provides additional validation from a resource-constrained perspective, achieving 97.99% accuracy with Random Forest on a dataset of 4,386 portable executable records. The identification of IMAGEBASE, SIZEOFSTACKRESERVE, and CHECKSUM as particularly significant features offers specific technical guidance for Trustfile's file type verification module, while the study's conclusion that machine learning models can serve as layers in larger neural networks suggests a clear enhancement path for future system iterations.

Azeem et al [5]. add mathematical rigor to this foundation, demonstrating that Information Gain and Gain Ratio entropy-based approaches can reduce dimensionality while maintaining classification accuracy. The achievement of 97.68% accuracy with Random Forest using TFIDF-based entropy feature selection validates Trustfile's strategy of using entropy both as a direct detection indicator and as a feature selection mechanism for machine learning models.

The convergence of these machine learning studies around Random Forest and ensemble methods is not coincidental but reflects the algorithm's particular suitability for malware detection tasks. Random Forest's inherent resistance to overfitting, its ability to handle high-dimensional feature spaces, its provision of feature importance rankings, and its computational efficiency make it ideally suited for web-based deployment where models must generalize across diverse file types, operate under resource constraints, and provide interpretable results. The consistent high accuracy achieved across independent studies—97.99% by Buradkar, 97.68% by Azeem et al., and balanced strong performance by Kamdan et al.—establishes confidence that Trustfile's classification foundation is built upon validated, reproducible methodology.

Entropy Analysis: From Simple Heuristic to Sophisticated Detection Foundation. The reviewed literature elevates entropy analysis from a supplementary diagnostic tool to a core detection methodology with demonstrated effectiveness against modern threats. Wee Ki Joon [8] represents the cutting edge of entropy-based detection through the integration of sliding window entropy extraction with convolutional neural networks, achieving approximately 91% overall accuracy across 11,442 malware samples from the BODMAS dataset. The sliding window methodology—computing entropy across overlapping segments rather than calculating a single value for entire files—provides specific technical guidance for Trustfile's entropy module. This approach identifies localized high-entropy hotspots that may indicate packed, encrypted, or obfuscated code regions, addressing a critical limitation of whole-file entropy calculation which can miss mixed-content files where legitimate low-entropy code conceals malicious high-entropy payloads.

Mheboobkhan [9] provides the foundational conceptual framework that justifies entropy's inclusion as a core Trustfile component. This study clarifies the critical distinction between reactive and proactive detection: while hash values and IOCs identify known threats, entropy analysis prevents threats in advance by identifying obfuscation patterns associated with malware behavior. This proactive capability is essential for zero-day protection—the period before signatures exist when systems are maximally vulnerable. The study's explanation

of entropy as a decision-support tool that guides analyst methodology aligns with Trustfile's educational objectives, demonstrating that entropy values can be translated into understandable indicators that help non-technical users comprehend why files are suspicious.

The synthesis of entropy research reveals a progression from basic threshold-based detection to sophisticated feature extraction for machine learning integration. This progression mirrors Trustfile's own architecture, where entropy analysis operates at multiple levels: as a direct heuristic indicator flagging files with anomalous randomness profiles, as a feature input for the supervised classification system, and as an educational element that helps users understand packing, encryption, and obfuscation concepts. The consistency of entropy's effectiveness across independent studies—from foundational analysis through CNN integration to large-scale dataset validation—establishes it as a reliable, validated detection modality that complements rather than replaces other analytical techniques.

Web-Based Deployment and API Integration: Democratizing Advanced Detection. The reviewed literature provides compelling evidence that sophisticated malware detection can be effectively delivered through accessible web interfaces, directly supporting Trustfile's architectural decisions. The hybrid detection architecture combining local machine learning with external threat intelligence, as demonstrated across multiple studies in the literature, establishes the technical viability of Trustfile's core design philosophy. These studies demonstrate that local static analysis—examining file headers, executable sections, API calls, and byte sequences—can be effectively complemented by real-time threat intelligence queries that provide global context for locally observed file characteristics. The combination of local heuristic analysis with external intelligence validation creates a hybrid model that balances speed and privacy with comprehensive coverage.

The democratization aspect of web-based deployment is particularly significant for the Philippine context. The reviewed literature consistently emphasizes that advanced detection capabilities need not require expensive infrastructure, specialized expertise, or complex software installation.[2] Kamdan's efficient ensemble methods, Buradkar's Python-based implementation, and Museeb's scalable frameworks all demonstrate that effective security can be built using accessible, open-source tools and deployed through standard web browsers. This accessibility directly addresses the resource constraints and knowledge deficits identified in Philippine cybersecurity assessments, where organizations lack budget for commercial security suites and personnel with specialized technical skills.

User Education and Communication: The Human-Centered Security Imperative. The reviewed literature establishes that technical detection capability alone is insufficient for effective cybersecurity—systems must also address the human factors that determine whether security measures achieve their intended protective outcomes. TechClass (2025) provides compelling empirical evidence for this imperative, documenting that human error is the root cause in 95% of cybersecurity breaches. This finding transforms user education from a peripheral concern to a core security requirement, validating Trustfile's emphasis on clear result explanations, actionable guidance, and embedded learning opportunities. The study's identification of phishing, business email

compromise, and insider mistakes as primary non-technical threats demonstrates that file-based attacks frequently exploit human vulnerabilities that technical controls alone cannot address.

The research on business email compromise highlights specific attack vectors that Trustfile's pre-access scanning directly interrupts. Cybercriminals impersonating senior leaders or suppliers send convincingly forged emails with malicious attachments, and employees acting on these requests without verification have caused millions in losses. Trustfile's scanning and clear result explanations provide a verification mechanism that can break this attack chain by identifying malicious attachments before users act on fraudulent requests. The study's recommendations for building security-aware cultures—regular engaging training, open incident reporting, leadership by example, security integration into daily processes—translate into Trustfile's design through embedded educational content that transforms each scanning interaction into a micro-learning opportunity.

The synthesis of user education research reveals that Trustfile's approach addresses a documented, critical gap in current security tools. Most existing systems prioritize threat neutralization over user enlightenment, presenting minimal information beyond generic threat names when detections occur. This opacity prevents users from developing independent threat recognition capabilities, creates dependency on automated tools, and may actually increase vulnerability by fostering distrust that leads to security circumvention. Trustfile's transparent detection process—explaining specific indicators, their significance, and recommended actions—transforms each security event into a learning opportunity that gradually builds user knowledge and threat awareness. This educational dimension distinguishes Trustfile from purely automated solutions and contributes to long-term organizational security resilience.

Integration and Convergence: Toward a Coherent Detection Architecture. The synthesis of all reviewed literature reveals a remarkable convergence around the architectural principles embodied by Trustfile. The foreign studies establish that hybrid detection combining multiple methodologies outperforms any single approach. The machine learning research validates that ensemble classification achieves high accuracy with practical efficiency. The entropy analysis literature demonstrates proactive threat identification capabilities. The user education research establishes that human factors determine whether technical controls achieve protective outcomes.

This convergence is not merely coincidental but reflects an emerging consensus in cybersecurity research: effective protection requires systems that are comprehensive in detection, efficient in operation, accessible in deployment, and empowering in education. Trustfile's architecture—integrating file hashing for known threat identification, entropy analysis for obfuscation detection, script scanning for embedded executable identification, file type verification for spoofing detection, external API integration for global threat intelligence, supervised classification for threat level assignment, and clear result explanation for user education—addresses each dimension of this consensus.

The Philippine context amplifies the relevance of this integrated approach. Documented underinvestment in cybersecurity infrastructure, limited technical expertise, widespread targeting by ransomware and other threat

actors, and the documented misconception of security insignificance create an environment where traditional security solutions fail to achieve adoption or effectiveness. Trustfile's web-based accessibility eliminates installation barriers, its user-friendly design reduces training requirements, its educational components build local capacity, and its hybrid detection provides comprehensive protection without expensive infrastructure. The academic research framework ensures rigorous evaluation and documentation, creating knowledge resources that can inform broader cybersecurity improvement efforts.

Future Trajectories and Enhancement Pathways. The reviewed literature also illuminates clear pathways for Trustfile's future evolution. The deep learning image analysis demonstrated by Naeem et al[7]. Achieving 99.86% validation accuracy through grayscale image representation and CNN classification—suggests a promising enhancement layer that could improve detection against novel threats while providing intuitive visualizations. The entropy-based CNN feature selection by Wee Ki Joon demonstrates how entropy analysis can be extended beyond simple threshold detection to sophisticated pattern recognition, suggesting that Trustfile's entropy module could evolve from heuristic indicator to deep learning feature extractor [8]. The gamification and interactive simulation recommendations from TechClass indicate that user engagement strategies could enhance educational effectiveness, potentially including progress tracking, achievement systems, and personalized learning paths in future iterations [10].

The reviewed literature collectively establishes that Trustfile represents a timely, evidence-based contribution to cybersecurity that addresses documented gaps across multiple dimensions. The foundational research by Aslan and Samet establishes the imperative for moving beyond signature-based detection [1]. The machine learning studies by Kamdan et al. [2], Museeb et al. [3], Buradkar [4], and Azeem et al. [5] validate the effectiveness of ensemble classification for static malware detection. The entropy analysis research by Wee Ki Joon [8] and Mheboobkhan [9] establishes entropy as a proactive, validated detection modality. The user education research by TechClass [10] establishes the critical importance of human-centered security communication.

The convergence of these diverse research streams around Trustfile's core principles—hybrid detection, machine learning classification, entropy analysis, web-based accessibility, API integration, and user education—demonstrates that the system is positioned within the mainstream of contemporary cybersecurity research while addressing specific, documented needs in the Philippine context. The empirical validation of each component across independent studies provides confidence in the system's potential effectiveness, while the identified enhancement pathways suggest a clear trajectory for continued improvement. Ultimately, the reviewed literature establishes that Trustfile's development is not merely an academic exercise but a practical response to urgent, validated cybersecurity challenges that affect organizations and individuals in the Philippines and globally.

REFERENCE

[1] Aslan, Ö. A., & Samet, R. (2020). A comprehensive review on malware detection approaches. IEEE Access, vol. 8,pp. 6249–6271. https://doi.org/10.1109/ACCESS.2019.2963724

[2] Kamdan, Y. Pratama, R. S. Munzi, A. B. Mustafa, and I. L. Kharisma (2025). Static malware detection and classification using machine learning: A random forest approach.Eng. Proc., vol. 107, Art. no. 76, Sep. 2025 https://doi.org/10.3390/engproc2025107076

[3] A. Museeb, Y. Hamed, R. Sokkalingam, A. A. Hamza, A. Ullah, and I. K. Khan. Enhanced Android malware detection using deep learning and ensemble techniques. Int. J. Adv. Comput. Sci. Appl. (IJACSA), vol. 16, no. 11, 2025 https://doi.org/10.14569/IJACSA.2025.0161108

[4] M. U. Buradkar, "Static malware analysis using optimal machine learning algorithm for malware detection," NeuroQuantology, vol. 20, no. 10, pp. 4128–4141, Aug. 2022.

[5] M. Azeem, D. Khan, S. Iftikhar, S. Bawazeer, and M. Alzahrani. Analyzing and comparing the effectiveness of malware detection using machine learning with entropy based feature selection. Heliyon, vol. 9, no. 12, Art. no. e187275, 2023. https://doi.org/10.1016/j.heliyon.2023.e187275

[6] A. L. Abdon, "RATTATA: Ransomware analysis via automated impact assessment of Philippine organizations," Int. J. Intell. Syst. Appl. Eng., vol. 12, no. 4, p. 3430, 2024. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/6858

[7] M. R. Naeem, R. Amin, M. Farhan, F. A. Alotaibi, M. M. Alnfiai, G. A. Sampedro, and V. Karovič, "Harnessing AI and analytics to enhance cybersecurity and privacy for collective intelligence systems," Big Data Cogn. Comput., vol. 8, no. 2, 2024, doi: 10.3390/jbdcc8020042.

[8] Wee Ki Joon, "Leveraging CNNs and entropy based feature selection to identify potential malware artifacts of interest," SANS Internet Storm Center Guest Diary, Mar. 26, 2025. [Online]. Available: https://isc.sans.edu/diary/31790

[9] Mheboobkhan, "Entropy analysis: A critical test for malware's," Medium, Feb. 13, 2019. [Online]. Available: https://whiteheart0.medium.com/entropy-analysis-a-critical-test-for-malwares-69939f5b8b1

[10] TechClass, "Cybersecurity for all: Why every employee matters in data protection," TechClass Resources, Apr.25,2025.[Online].Available:https://www.techclass.com/resources/learning-and-development-articles/cybersecurity-for-non-tech-teams-why-everyone-plays-a-role-in-data-protection

PHINMA UNIVERSITY OF ILOILO
College of Information Technology Education

CHAPTER III

METHODOLOGY

3.1 Research Design

This study employed a developmental research design, focused on designing, developing, and testing TrustFile as a web-based file security platform. Developmental research, also known as Research and Development (R&D) methodology, was particularly well suited to a study of this kind, since it emphasized the systematic process of identifying a real need, designing a solution around it, building a working prototype, and testing that prototype's effectiveness rather than simply describing or explaining an existing phenomenon, as purely descriptive or explanatory research would [1], [2]. Because this was a developmental study, the system itself constituted a primary research output, and TrustFile's actual capabilities directly reflected whether the study's stated objectives had been met [3].

The design dimension of the study encompassed architectural planning, interface specification, and the selection of the five core static analysis techniques file hashing, entropy analysis, script detection, abnormal size verification, and file type verification alongside the overarching AI analysis engine and the heuristic analysis layer that interpreted their combined results [4], [5]. Design activities in this phase were what turned abstract requirements into concrete technical specifications capable of guiding actual implementation. This process involved weighing genuine trade-offs between competing objectives: security thoroughness against usability, comprehensiveness against processing speed, and reliance on local server-side analysis against dependency on an external threat-intelligence API [4]. Every design decision made during this phase was documented, both to allow later evaluation and to give future researchers a clear basis for replicating or extending the work.

The development component involved the actual construction of the system according to the specifications established during design. This included writing the server-side scanning logic, configuring the PythonAnywhere hosting environment, integrating the external malware-intelligence API, and assembling all of these pieces into a single working web application. Development followed conventional software engineering practices, including incremental version control, informal code review between the researchers, iterative testing, and running documentation of design decisions as they were made [6].

Testing, the final component of this design, validated that the developed system actually met the requirements identified earlier in the study and functioned as intended. Testing was carried out at multiple levels beginning with the verification of individual components such as the hashing module or the entropy calculator in isolation, moving through integration testing of the complete scanning pipeline, and ending with an overall assessment of whether the system's classifications matched expectations for the sample files used [6]. The results generated

during this phase provided the empirical basis for the conclusions presented later in this study, highlighting both the system's strengths and the areas that still required refinement.

The decision to build TrustFile as a web-based platform, rather than a locally installed application, constrained the design space in some ways while enabling others. Web architectures like this one supported universal access across operating systems, simplified deployment, and centralized management updates to the scanning logic could be applied once, centrally, without requiring anything from the end user. At the same time, this approach meant the researchers had to design around browser sandboxing restrictions, network dependency, and the resource ceilings imposed by the specific PythonAnywhere hosting tier used, all of which shaped decisions made later in the developmental and experimental phases.

The research was divided into the following three phases.
Descriptive Phase - In this phase, the researchers studied common problems related to file security including the risks of opening unscanned files and the broader lack of accessible tools for checking a file's safety before opening it. Based on these problems, the researchers identified the features the system would ultimately need: file upload and scanning, threat detection through five static analysis techniques, an AI analysis engine capable of reasoning across the combined results of those techniques, a parallel heuristic analysis layer, and a results interface that displayed clear, understandable explanations to the user.

This phase relied primarily on descriptive research methods grounded in a review of existing literature concerning malware delivery methods [7], antivirus limitations [8], [9], and end-user security behavior [10] to characterize the current state of file security practices among the study's intended end users: offices, private businesses, government agencies, and other organizations that regularly received large volumes of incoming files [11]. This approach ensured that the eventual system requirements were grounded in documented, real-world problems rather than assumptions about what users might need.

The risks associated with opening unscanned files, examined during this phase, spanned the full range of potential consequences system compromise, data theft, ransomware encryption, credential harvesting, and the broader financial and operational costs that these incidents impose on organizations [11], [12], as discussed in the Background of the Study. Understanding the scope of these risks gave the researchers a clearer basis for prioritizing which detection techniques mattered most, and for framing the five-tier BENIGN-to-MALICIOUS classification scale that the system would eventually use to communicate risk to non-technical users.

Developmental Phase - In this phase, the researchers designed and developed the system itself. This included building the file upload and scanning workflow, implementing the five static analysis techniques, building the AI analysis engine that synthesized the combined results of those techniques into a single risk assessment, developing the heuristic analysis layer that ran in parallel to catch novel or obfuscated threats, and integrating an external API for malware-intelligence lookups. All of these components were ultimately combined into a working prototype of the system.

Building the file scanning workflow required the researchers to define the entire path a file took, from the moment a user submitted it through the browser to the moment a final classification was displayed. This included deciding which file extensions and sizes the system would accept ultimately, any file size and a broad range of common extensions, including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, and RAR designing a secure upload mechanism over HTTPS [13], and determining the order in which the five analysis techniques and the external API lookup would run before their results were passed to the AI analysis engine and the heuristic layer.

Implementing the five analysis techniques required separate, modular development work for each:

- Hashing required the researchers to select a cryptographic hash algorithm MD5 [14] and SHA-256 [15] were used to compute a digest for each uploaded file and compare that digest against known-malware records retrieved through the external API [16].
- Entropy analysis required implementing Shannon entropy calculations [17] across each file's content and establishing baseline entropy ranges for common file types, so that unusually random-looking files often a sign of packing or encryption could be flagged for further scrutiny [18], [19].
- Script detection required building logic capable of identifying embedded JavaScript, VBScript, PowerShell, or macro code hidden inside files that would not normally be expected to contain executable code [20].
- Abnormal size verification required compiling reference ranges for typical file sizes by format, so that files padded with an appended malicious payload, or suspiciously small stub loaders, could be identified [21].
- File type verification required checking each file's internal structure and magic numbers against its stated extension, catching cases where an executable had been disguised as an ordinary document [7].

Rather than treating each of these five techniques as an isolated pass-or-fail check, the researchers built the heuristic engine to weigh all five sets of results together, along with the outcome of the external API lookup, through a defined set of rules and thresholds, producing a single classification along the five-tier BENIGN-to-MALICIOUS scale [4], [5]. The AI explanation layer was developed as a separate component that took this finished classification and generated the plain-language explanation accompanying it describing which specific indicators had contributed to the result and why they mattered without participating in the classification decision itself.

Integrating an external API for malware detection required the researchers to select a suitable threat-intelligence service, implement a secure client that queried the service using only a file's cryptographic hash rather than its full contents, and design handling for cases where the service was slow to respond or temporarily unavailable [16]. This hash-based approach, discussed further in the System Architecture section, was chosen specifically to preserve the privacy of file contents while still benefiting from externally maintained malware databases.

By the end of this phase, all of these components had been combined into a working prototype demonstrating the complete scanning pipeline: file upload, execution of the five static analysis techniques, an external API

lookup, synthesis by the AI analysis engine, a parallel heuristic pass, and a final five-tier classification presented to the user along with a plain-language explanation.

Experimental Phase - In this phase, the system was tested using different types of files, including normal files, suspicious files, and the safe EICAR test file [22]. The purpose of this testing was to check whether the system could properly detect and classify threats using the AI analysis engine and the heuristic analysis layer working together, and to evaluate the accuracy of the results produced and the overall performance of the system.

Testing with different file types allowed the researchers to evaluate the system across the range of files it was likely to encounter in practice. Normal, legitimate files ordinary documents, images, and archives were used to check whether the system correctly avoided false positives. Files exhibiting suspicious but not definitively malicious characteristics, such as elevated entropy resulting from ordinary compression [18], were used to check whether the system correctly assigned intermediate classifications, such as LOW RISK or SUSPICIOUS, rather than jumping straight to an extreme verdict in either direction. The EICAR test file a standardized, harmless string universally recognized by security software as a test signature [22] was used to validate the system's ability to detect and flag known-malicious content without introducing any actual malware into the testing environment.

Beyond simple detection, the purpose of this testing extended to validating classification accuracy: confirming that files were sorted into the correct tier along the five-level BENIGN-to-MALICIOUS scale, and that the AI analysis engine's synthesis of the five underlying indicators, together with the heuristic layer's independent findings, produced classifications that matched the researchers' expectations for each test file. Each test file's actual classification was compared against its known or expected status, with standard evaluative concepts such as detection accuracy and error rate the same false positive and false negative concepts discussed in the Definition of Terms used as the interpretive framework for assessing how well the system performed [23]. The researchers also recorded processing times and resource usage during these tests to assess whether the system's performance remained acceptable under the conditions available through the PythonAnywhere hosting tier used for this study.

3.2 System Development Methodology

This study employed the Agile Development Model in developing TrustFile, an approach that allowed the researchers to improve the system step by step through continuous testing and adjustment, which proved especially useful given how easily changes could be applied throughout the development process rather than being locked in from the start [24].

The Agile Development Model was selected specifically because of how closely it matched the exploratory, iterative nature of this academic project. Agile methodologies are built on two complementary principles iterative development, in which each piece of the system is refined and improved through repeated cycles, and incremental development, in which usable, working pieces of software are delivered progressively throughout

the project rather than all at once [25]. Agile approaches, more broadly, prioritize working software over exhaustive up-front documentation and treat responsiveness to new findings as more valuable than strict adherence to an original plan [24] priorities that suited a project where the researchers' understanding of file security requirements deepened considerably as the descriptive and developmental phases progressed.

A sequential, plan-driven model such as the Waterfall model would have required the researchers to fix every requirement before any code was written, with testing deferred until the end of the process [26]. Such an approach is best suited to projects whose requirements are stable and well understood from the start [6]. TrustFile did not fit that profile: the interaction between the five static analysis techniques, the heuristic engine, the external threat-intelligence lookup, and the AI explanation layer could only be fully understood by building and observing them working together. Agile development accommodated this uncertainty by allowing earlier design decisions to be revisited as new findings emerged, rather than treating them as fixed commitments.

Agile development was also a practical fit for the researchers' circumstances. Agile methods emphasize close collaboration within small teams, frequent communication, and rapid adaptation to change [27], all of which matched a small group of student researchers working within a limited academic timeframe. Because the system was developed by a small team without a dedicated customer or external client, the researchers themselves acted as both developers and the primary reviewers of each increment, which shortened the feedback loop between building a component and judging whether it met the study's objectives.

Development was broken into a series of small, manageable increments, each of which added one testable piece of functionality: first the upload mechanism, then each of the five static analysis techniques individually, then the AI analysis engine that synthesized their results, then the heuristic layer, and finally the external API integration and the results interface. Each increment was tested as soon as it was completed, rather than waiting until the entire system had been assembled an approach consistent with how Agile teams typically operate in short cycles of one to four weeks, continuously demonstrating working software and incorporating feedback before moving to the next increment [28]. This allowed the researchers to catch integration problems early and adjust the design of later components based on what had been learned from earlier ones, rather than discovering fundamental design flaws only after the full system had already been built.

The phases described below correspond directly to the three research phases presented in the Research Design: Requirements Analysis aligns with the Descriptive Phase, System Design, Development, and Testing together make up the Developmental Phase, and the Testing and Deployment (Prototype) stages feed into the Experimental Phase. Presenting the methodology this way keeps the research design and the development process consistent with one another, since both describe the same work from two perspectives: what the study set out to learn, and how the system was actually built.

Requirements Analysis - In this phase, the researchers identified the common problems related to file security that motivated the study. Many users did not scan files before opening them, which created the possibility of malware infection. The researchers listed the features the system would need: file upload and scanning, the five

static analysis techniques, the AI analysis engine, the heuristic analysis layer, external API integration, and a results display capable of communicating findings clearly. The goal set during this phase was to create a system simple enough for offices and organizations that constantly received files to use without requiring any specialized technical staff.

Identifying these problems drew on prior research concerning user behavior [10], and the shortcomings of existing antivirus software's black-box detection model [8], [9], as discussed at length in the Statement of the Problem. The researchers used this understanding to prioritize which features the prototype needed to include from the very outset most notably, the emphasis on transparent, explained results rather than an opaque safe/unsafe verdict, since this transparency gap had already been identified as one of the central problems the study set out to address. This early prioritization mattered specifically because Agile development, by design, does not attempt to specify every feature in exhaustive detail before development begins; instead, it relies on a clear sense of what matters most so that the highest-priority features are built and tested first, with less critical refinements following in later increments [29].

The requirements gathered in this phase were treated as a working baseline rather than a final specification. Both functional requirements, such as accepting uploads, running the analysis techniques, and displaying a classification, and non-functional requirements, such as usability for non-technical users, privacy of file contents, and acceptable processing time, were recorded so that later increments could be checked against them. Because the Agile model expects requirements to be refined as understanding grows, any requirement that proved unclear or incomplete during later phases could be revisited without disrupting the overall development process.

System Design - In this phase, the researchers planned how the system would function. The design covered the process of scanning files using the five techniques already identified file hashing, entropy analysis, script detection, abnormal size verification, and file type verification along with the AI analysis engine that synthesized these results and the heuristic analysis layer that operated alongside it [4], [5]. The researchers also planned the integration of an external API for malware-intelligence lookups. The interface itself was designed to remain simple, so that users could easily upload and scan files of any size and any extension without needing guidance.

Designing the scanning process required specifying, for each of the five techniques, exactly which algorithm or comparison method would be used and how its output would be represented before being passed on to the AI analysis engine. For hashing, this meant selecting a cryptographic hash algorithm and defining how a match against the external threat database would be handled [16]. For entropy analysis, this meant defining the calculation method and the baseline ranges used to flag deviations [17], [18]. For script detection, this meant specifying which script types and embedding patterns would trigger a flag. For size verification, this meant compiling reference size ranges by file type. For file type verification, this meant specifying how a file's internal structure and magic numbers would be compared against its claimed extension [7].

Designing the classification logic itself required the researchers to plan how the five sets of indicators, together with the VirusTotal result, would be weighed against one another through the heuristic engine's rule-based logic to produce a single classification along the five-tier BENIGN-to-MALICIOUS scale [5]. Separately, the researchers planned how the AI explanation layer would take that finished classification and its contributing indicators and generate the plain-language explanation accompanying each result describing which specific indicators had contributed to the classification and why they mattered, consistent with the transparency goal established during Requirements Analysis. Keeping classification and explanation as two separate components also meant that each could be developed, tested, and adjusted independently, which supported the incremental approach adopted for the study.

The interface design process produced simple wireframes for the upload screen, the in-progress scanning screen, and the results screen. Simplicity guided every decision made at this stage: minimizing the number of steps required to submit a file, using plain language rather than technical terminology, and ensuring that the five-tier classification was communicated clearly through color and short explanatory text before any more detailed technical information was shown to the user.

Development - In this phase, the system was built step by step. The researchers first created the file upload and scanning features, and afterward implemented the file analysis methods hashing, entropy analysis, script detection, abnormal size verification, and file type verification followed by the AI analysis engine and the heuristic analysis layer. The system was able to classify files into the five threat levels defined earlier in the study: BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS.

Building the file upload feature involved creating a server-side endpoint that received submitted files over an encrypted connection [13], validated them, stored them temporarily for analysis, and managed the upload session for each user. Because the system was designed to accept files of any size and any extension, particular care was taken to validate submissions safely without rejecting legitimate files simply because of their format or size.

Building the scanning feature involved creating the workflow that coordinated the five static analysis techniques, invoked the VirusTotal API for a hash-based lookup [16], and passed all of these results into the heuristic engine, where rule-based logic weighed them together to reach the final five-tier classification. That finished classification was then passed to the AI explanation layer, built on the NVIDIA NIM API and the Llama 3.1-70B-Instruct model [30], which generated the plain-language explanation shown to the user alongside the result. Developing each of these pieces as a separate module meant that a change to one component, such as adjusting an entropy threshold, could be made and retested without rebuilding the rest of the pipeline.

Building the classification logic required the researchers to define the specific criteria distinguishing each of the five tiers. A file was classified as BENIGN when it passed all five checks cleanly with no anomalies detected. A file was classified as LOW RISK when it showed minor deviations, such as slightly elevated entropy attributable to ordinary compression [18], that the AI analysis engine judged unlikely to indicate malicious

intent. A file was classified as SUSPICIOUS when it exhibited anomalies that did not conclusively indicate malicious content but warranted closer review. A file was classified as HIGH RISK when several indicators converged, or when a single strong indicator was present, such as an external API match or a markedly abnormal entropy value. A file was classified as MALICIOUS when it produced a confirmed indicator of compromise, most commonly a direct hash match against a known-malicious file in the external threat database. Establishing these criteria required the researchers to calibrate thresholds and assign relative weight to each of the five indicators, so that the AI analysis engine's synthesized output remained consistent across the range of test files used later in the experimental phase.

Testing - The system was tested using different sample files to check whether it worked properly. The researchers used safe testing files, such as the EICAR test file [22], to simulate malware detection without causing any actual harm. This testing helped confirm whether the system could detect and classify suspicious files correctly using the AI analysis engine and the heuristic analysis layer. Some errors were found during testing and were corrected to improve the system.

Testing with different sample files was intended to give the researchers comprehensive coverage of the file types and threat categories the finished system needed to handle correctly. This included ordinary, legitimate files, to check that the system did not produce false positives [23]; files with mildly suspicious characteristics, to check that the AI analysis engine correctly assigned intermediate classifications such as LOW RISK or SUSPICIOUS rather than an extreme verdict; and the EICAR test file, to confirm that a known threat indicator reliably produced a MALICIOUS classification.

During this phase, the researchers identified several issues including cases where the AI analysis engine's classification did not match the researchers' expectations, and cases where the heuristic analysis layer produced results that conflicted with those of the AI analysis engine and corrected these issues before proceeding to the next round of testing. This iterative cycle of testing, identifying discrepancies, and correcting the underlying logic continued throughout this phase, consistent with the Agile approach adopted for the study: rather than treating testing as a single, final gate at the end of development, each increment was tested and refined in its own right, with lessons from one round of testing directly informing adjustments made before the next [6], [29].

Deployment (Prototype) - After testing, the system was presented as a working prototype. It demonstrated how the system could scan and analyze files before they were opened, and display the results to users in a way intended to help them become more aware and more careful when handling files going forward.

The completed prototype demonstrated the full range of intended features: file upload through a simple browser interface, execution of the five static analysis techniques, a hash-based external API lookup, synthesis of all results by the AI analysis engine, a parallel heuristic analysis pass, and a clear, explained five-tier classification presented to the user. This demonstration confirmed that the concept behind the study's objectives could be realized as a functioning system, even though, as an academic prototype, it was tested against a curated set of sample files rather than deployed for unrestricted public use.

Beyond the technical demonstration, the researchers also observed that the explanatory results produced by the AI analysis engine appeared to build user understanding of why a given file had been flagged, rather than presenting a bare verdict. This supported the study's broader educational goal helping users gradually develop their own threat-recognition awareness rather than remaining entirely dependent on the system's automated judgment, a theme discussed at length in the Significance of the Study.

3.3 System Architecture

TrustFile functioned much like a security checkpoint for files before users ever opened them. The system offered a simple web interface through which users uploaded files from their computer for scanning, designed specifically so that even users without technical knowledge could use it confidently. The system was built to scan files of any size and virtually any extension, including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, RAR, and other common formats. This section describes the high-level structure and operational model of TrustFile, explaining how its components interacted to deliver file security services giving readers a conceptual understanding of how the system worked without requiring a deep technical background.

The security checkpoint analogy captured TrustFile's core function well: intercepting files before they reached the user, examining them for threats, and either permitting or blocking their passage based on what that examination found. In the same way a physical security checkpoint inspects luggage and verifies identification before granting entry, TrustFile inspected digital files for malicious content before ever granting the user access to them. This preventive model stood in direct contrast to reactive approaches that only respond once an infection has already occurred or a known signature has already been matched [8], [9], a distinction discussed at length in the Objectives of the Study.

The web interface served as the system's primary point of interaction, designed for accessibility across a wide range of technical skill levels. Web interfaces offer interaction patterns, forms, buttons, progress indicators, result displays that most users already encounter daily in other applications, which reduced the learning curve considerably and encouraged genuine adoption rather than avoidance [31]. The interface design consistently prioritized clarity over complexity, keeping essential functions prominent while still making more advanced technical detail available for users who wanted to dig deeper, without letting that detail overwhelm the primary experience.

File upload functionality allowed users to submit files from their local computers for remote analysis. This involved browser based file selection, secure transmission to the server [13], temporary storage during analysis, and eventual result presentation. The upload process included validation to confirm that submitted files were scannable and appropriately received, while the system's support for any file size and any extension ensured that comprehensive protection was never limited by a file's particular format or size. This design choice reflected a deliberate recognition that the users most in need of file security assistance were often precisely the users with

the most limited technical knowledge which is why interface elements throughout the system relied on plain language rather than technical jargon, why workflows were kept to a minimal number of steps, why results were always presented with clear explanations rather than raw technical data, and why guidance was made readily available rather than assumed. This inclusive design approach ensured that meaningful security protection extended to all users, not solely those with existing technical expertise.

The system also connected to an external threat intelligence API to check whether an uploaded file was already known to be malicious, comparing it against existing malware databases to strengthen detection beyond what local analysis alone could achieve [4]. This external API integration formed a critical architectural component precisely because it connected TrustFile to global threat intelligence networks maintaining databases of known malware that were updated continuously through automated collection, researcher submissions, and observations gathered from sensor networks worldwide resources no single local system could realistically replicate [16].

The specific mechanism used for this integration was a hash based query, chosen deliberately to preserve privacy while still maximizing threat coverage. Rather than transmitting a file's complete contents to an external service, TrustFile computed a cryptographic hash of the file [15] and sent only that hash for lookup [16]. The external service then returned information about whether the hash matched a known malicious file, along with any associated threat intelligence or reputation data. This approach protected sensitive file content from ever being exposed externally, while still allowing the system to benefit fully from comprehensive, continuously updated threat databases maintained well beyond what this study alone could build.

This external comparison addressed a specific limitation of local analysis on its own: local techniques such as heuristic and anomaly detection excel at catching genuinely novel threats [5], but can still miss known threats that would otherwise match a specific signature already documented in an external database. Combining local and external detection gave the system defense in depth, in the sense discussed in the Definition of Terms, each approach compensating for gaps the other inevitably left open [4].

After scanning, the system presented its findings by classifying the file into one of the five threat levels established throughout this study: BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, or MALICIOUS. This classification was determined by TrustFile's heuristic engine, which weighed the results of the system's five static analysis techniques together with the external VirusTotal lookup to reach a single, rule based verdict [5]. This result was displayed clearly enough that a user could immediately understand whether a file was safe, and the system explained why a file received a given classification through a separate AI generated explanation layer built on the NVIDIA NIM API and the Llama 3.1 70B Instruct model [30] which took the heuristic engine's finished classification and its supporting indicators and translated them into clear, understandable language, rather than a bare technical readout [32]. This result presentation architecture existed specifically to transform an already decided technical outcome into information a non technical user could actually act on. Each of the five levels was paired with a distinct color and short explanatory message reinforcing the assessment at a glance following conventional visual associations such as green for safe and red for confirmed danger [33],

with intermediate shades distinguishing the levels in between. Clear presentation meant organizing information hierarchically: the most critical information, the threat level itself and any recommended action appeared prominently, with supporting technical detail available through progressive disclosure for users who wanted it. Explanatory text consistently described what had been found, why it mattered, and what the user should do next.

The AI explanation layer analyzed the specific combination of indicators the heuristic engine had already flagged in each individual file and generated a customized explanation reflecting that file's particular threat profile, rather than pulling from a generic, one size fits all template. This ensured that users received precise, relevant information about why their specific file had been classified the way it was. For example, rather than simply stating "Threat detected," the system might explain that a file contained an unusually high entropy value suggestive of packing or encryption [18], that embedded JavaScript attempting to contact an external server had also been found a behavior commonly associated with Trojan malware [21] and that blocking the file and running an updated antivirus scan was recommended as a next step. This kind of explanation avoided technical jargon, provided context for why each indicator mattered, and offered guidance the user could actually act on, gradually building the user's own threat recognition capability alongside the immediate protection the classification decision provided.

The overall architectural philosophy behind TrustFile balanced simplicity with security effectiveness. Simplicity mattered because a security tool that goes unused, out of confusion or frustration, provides no protection at all, accessibility and adoption were treated as prerequisites for effectiveness, not separate concerns from it [31]. At the same time, robust detection capability mattered because simplicity alone cannot protect against genuinely diverse threats. This balance was achieved by deliberately hiding technical complexity behind a simple interface, while preserving the full analytical depth of the five static analysis techniques, the AI analysis engine, the heuristic analysis layer, and the external threat intelligence integration underneath it.

The behavioral impact of this design extended beyond any single scan. By consistently showing users what made a specific file dangerous, explaining how the AI analysis engine and heuristic analysis had identified a given threat, and reinforcing safe practices with every result, the system gradually built a level of security awareness that carried over into how users handled files even outside the scanning process itself, making them more cautious about file sources, more attentive to suspicious characteristics, and more likely to seek verification before opening an unexpected attachment [10]. For offices and organizations that constantly received and downloaded files, this behavioral shift carried real organizational weight: employees who genuinely understood file security contributed to a stronger overall security posture, reducing the likelihood of successful attacks and minimizing security incidents across the organization as a whole [10], [11].

3.4 Tools and Technologies Used

TrustFile was built from a small, focused set of technologies, each chosen for a specific role in the system. The frontend technologies handled what the user saw and touched. The backend technology carried the scanning logic and the heuristic engine. The database held the information the system needed to keep. Two external

services supplied threat intelligence and plain-language explanations, and a single development environment tied the work together. The tools are described below in that order.

Frontend: HTML, CSS, and JavaScript. The frontend was built with HyperText Markup Language (HTML) and Cascading Style Sheets (CSS), the two standard technologies that every web browser understands. HTML provided the structure of every page in the system. It defined the forms, buttons, headings, tables, and containers used by the login and registration pages, the file upload page, the scan result page, the dashboard, and the scan history. The upload form, for example, was an HTML form that let users select a file from their computer and submit it to the server for scanning. Without this structure, the browser would have had nothing meaningful to display.

CSS controlled how that structure looked. It defined the colors, fonts, spacing, alignment, and overall layout of the interface, which gave TrustFile a consistent and professional appearance across all of its pages. CSS was also used to communicate risk visually. Each of the five threat levels (BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS) was assigned its own color, so a user could understand the severity of a result at a glance before reading any explanation. Responsive styling rules allowed the pages to adapt to different screen sizes, so the system remained usable on desktops, laptops, and smaller devices. Because HTML and CSS run natively in every modern browser, users did not have to install anything to use TrustFile, which supported the study's aim of a web-based platform that works on any operating system.

Backend: Python was the programming language used to build the server-side of TrustFile. It handled everything that happened between the moment a file was submitted and the moment a result was returned. Python received the uploaded file, validated it, and then ran the static analysis techniques: computing the cryptographic hashes, calculating the Shannon entropy, verifying the file type through its magic bytes, detecting embedded scripts, and checking the file size. It also housed the heuristic engine, which combined those results with the VirusTotal result and applied a defined set of rules and thresholds to reach a single classification along the five-tier scale. Finally, Python communicated with the external services and the database, and it passed the finished results back to the interface.

Python was chosen for several reasons. Its clear, readable syntax made the code easier to write, review, and maintain, which mattered for a small team of student researchers working within a limited academic timeframe. It has strong built-in support for reading files as raw bytes, which is essential for hashing, entropy calculation, and magic byte inspection. It also has mature support for numerical computation, text and pattern matching, and communication with web services. Because Python could carry every stage of the scanning pipeline, the whole analysis logic lived in one language, which kept the system consistent and easier to test module by module.

Database: Firebase was the cloud database platform used to store the data TrustFile needed to keep beyond a single request. This included registered user accounts, the record of every uploaded file, the results of each scan with its classification and supporting indicators, and cached VirusTotal results. Firebase stores data as flexible, hierarchical records instead of fixed tables with rows and columns, which suited TrustFile well. A scan result is

naturally a nested piece of information containing a file name, a hash, several indicator values, a classification, and an explanation, and it could be stored and retrieved as one unit without complicated joins.

Because Firebase is hosted in the cloud, the researchers did not have to install, configure, or maintain a database server. The same stored records powered the dashboard and scan history, which showed each user only their own files and past results. Firebase also kept the cache of VirusTotal results, which allowed the system to reuse a recent result when the same file was scanned again, instead of sending it to VirusTotal a second time. This saved time and reduced the number of requests made to the external service.

Virus Scanner: VirusTotal API v3. VirusTotal was TrustFile's external source of threat intelligence. It is a cloud service that combines the verdicts of dozens of independent antivirus engines and security vendors into one report. Using it gave TrustFile access to malware databases and detection knowledge that are updated continuously from submissions and sensor networks around the world. That knowledge goes far beyond what a locally built system could hold on its own. Version 3 of its API is a web interface that lets applications send requests and receive structured, machine-readable responses, which allows TrustFile to ask for a scan and read the results directly in its code.

In TrustFile, the files users submitted were uploaded to VirusTotal in full so that the engines could examine the actual contents. Before that upload, the system computed the file's hash and used it to check whether a stored result already existed, which avoided scanning the same file twice. Once the analysis was complete, TrustFile read how many engines flagged the file, along with the individual engine results, and passed these findings to the heuristic engine. The VirusTotal result was only one of the inputs the heuristic engine weighed. It sat alongside the local static analysis results instead of acting as the only verdict, so the two kinds of detection covered each other's gaps.

AI Analysis: Google Gemini 2.0. The explanation half of TrustFile was powered by Google Gemini 2.0, a large language model that can read structured information and write clear, natural text. The role of the AI was deliberately limited. It did not decide whether a file was safe or dangerous. That decision belonged to the heuristic engine, which had already assigned the classification using fixed rules and thresholds. Only then were the finished classification and the specific indicators behind it sent to Gemini, with the instruction to explain them in plain language for a reader with no technical background.

This separation was an intentional design choice. A rule-based engine always gives the same result for the same file, which makes the classification consistent and open to inspection. A language model can word its answers slightly differently from one run to the next, and that variability would be unacceptable in a security decision. By keeping the verdict in rules and using the AI only to explain it, TrustFile combined a result users could trust with an explanation they could understand. The explanations were written for each file from its own indicators instead of from generic templates. For example, they could describe what an unusually high entropy value usually means, why an embedded script was suspicious in that type of file, and what the user should do next.

This supported the study's educational goal of helping users learn why a file was flagged. If the AI service was unavailable, the system could fall back to a rule-based explanation, so a result was always shown.

Development Tools: Visual Studio Code. Visual Studio Code (VS Code) was the environment in which the researchers wrote and reviewed all of TrustFile's code. It is a free, widely used code editor that supports Python, HTML, and CSS in a single workspace, so the same tool covered both the backend and the frontend. Its syntax highlighting, code completion, built-in debugging, and version control integration made it practical to write the scanning modules, test them one at a time, and fix errors as they appeared. This fit the incremental approach of the Agile development model described in Section 3.2, in which each piece of the system was built, tested, and improved in repeated cycles.

Hosting and Deployment: Vercel. TrustFile was deployed on Vercel, a cloud platform for hosting and publishing web applications. Hosting the system there meant anyone with an internet connection could reach it through an ordinary browser, without the researchers or the users configuring or maintaining a server. Vercel supplied the environment where the frontend pages were served and where the Python backend ran, and it provided an HTTPS endpoint, so files uploaded by users traveled over an encrypted connection. Because the system was deployed in one central place, any improvement to the scanning logic, the heuristic rules, or the interface could be published once and reach every user immediately, with nothing to install or update on the user's side.

The platform also placed limits that shaped the design. The backend ran as on-demand cloud functions that were allowed a limited amount of time to complete each request, and each request could carry only a limited amount of data. These limits influenced decisions such as keeping each analysis module lightweight, caching external results to reduce repeated network calls, and handling the wait for VirusTotal in a way that stayed within the platform's time limits.

3.5 Data Gathering Techniques

The researchers relied on the following methods to gather the information that shaped TrustFile's design and eventual feature set:

Observation - The researchers observed that many companies received large volumes of files every day, a pattern that left them more vulnerable to malware capable of harming their computers and networks. Given this overwhelming volume, these organizations often struggled to identify and separate which incoming files might actually cause harm. This is precisely where TrustFile was positioned to help scanning files and explaining clearly why a given file was harmful and should not be opened. This observation was especially relevant for offices, private sector organizations, and government agencies, all of which constantly received and downloaded files as a routine part of their daily operations.

This observation of high-volume file reception revealed a genuine operational challenge that directly amplified malware risk. Organizations across sectors such as government, education, business services, and private

enterprise routinely received hundreds or even thousands of files daily, arriving through email, web portals, file-sharing services, and removable media. Every one of these files represented a potential threat vector, yet the sheer volume made individual manual scrutiny practically impossible. Security personnel could not realistically examine every file by hand, which created a structural reliance on automated tools that, as discussed in the Statement of the Problem, often missed sophisticated or previously unseen threats.

This volume-driven vulnerability extended beyond detection difficulty alone. It also created a real risk of alert fatigue, where security personnel, overwhelmed by a high number of warnings, gradually began to ignore or dismiss alerts altogether a pattern that could cause genuine threats to slip through unnoticed among a backlog of false positives. Automated tools lacking contextual understanding compounded this problem further by flagging legitimate files as suspicious, consuming investigation time and creating friction between security teams and the rest of an organization trying to get ordinary work done.

TrustFile's design directly addressed this observed need by providing automated scanning capable of processing files quickly, classifying threats through the system's heuristic engine, and then explaining those findings clearly through its AI-generated explanation layer. The "why they are harmful" component of this observation addressed the underlying educational gap identified throughout this study, ensuring that users came to understand a given threat rather than simply accepting an automated verdict at face value. The accompanying "should never be opened" guidance, in turn, gave users clear, actionable direction, reducing ambiguity and supporting more consistent security practices across an organization, rather than leaving each individual user to interpret an ambiguous warning on their own.

For offices and organizations that constantly received files, this combination of automation and explanation was not merely convenient but essential. The sheer volume of incoming files made manual analysis impossible from the outset, while the diversity of file sources and formats increased exposure to a correspondingly wide range of threat vectors. TrustFile's comprehensive scanning of files of any size and virtually any extension was designed specifically to ensure that no potential threat escaped analysis simply because of its format or size.

Analysis of Existing Systems - The researchers analyzed existing platforms and systems that offered file scanning, either online or through installed applications, in order to observe their features and identify their limitations. This analysis helped develop a clearer understanding that file scanning alone was not sufficient as a standalone feature the system also needed to explain the problems associated with a flagged file, so that users could genuinely understand what made a file harmful rather than simply being told that it was. The researchers also examined how existing solutions implemented AI-assisted communication and heuristic-based detection to better understand where TrustFile could meaningfully improve on current practice.

This analysis of online scanning platforms examined services such as VirusTotal, MetaDefender, Jotti, and similar tools offering web-based file analysis. These platforms generally offered comprehensive multi-engine scanning, extensive threat databases, and detailed technical reports genuine strengths that TrustFile itself drew on directly, most notably through its own integration with the VirusTotal API. However, the researchers observed that these platforms tended to present their results in formats accessible primarily to security

professionals dense technical terminology, raw detection output, and minimal explanatory content that a non-technical user would likely find confusing or simply overwhelming. None of the platforms examined offered the kind of accessible, AI-generated explanation layer that TrustFile was built around.

Application-based scanning tools, including conventional antivirus software, endpoint protection platforms, and standalone file analysis utilities, were also evaluated as part of this analysis. These tools generally offered deeper system integration and a broader overall protection scope than a browser-based tool like TrustFile could provide, but they frequently came with complex interfaces, intrusive notifications, noticeable performance impacts on the host device, and most relevant to this study, the same opaque, black-box detection process discussed at length in the Statement of the Problem. Non-technical users were often alienated by exactly this opacity, left with a bare verdict and no real understanding of the reasoning behind it.

The key insight to emerge from this analysis was that detection alone is insufficient for genuinely effective security. A user who does not understand why a file is dangerous cannot apply that understanding to similar files encountered in the future, cannot meaningfully verify whether a given detection was accurate, and may come to distrust or even actively circumvent a security tool that feels arbitrary or opaque in its judgments. The educational component identified through this analysis, explaining the specific problems associated with a flagged file was what transformed security scanning from a black-box service into a genuine learning opportunity, building a user's own capability over time rather than leaving them permanently dependent on the tool's verdict. TrustFile's AI-generated explanation layer was built specifically to close this gap: taking the heuristic engine's finished classification and translating it into contextual, understandable guidance a non-technical user could actually act on.

This understanding directly shaped TrustFile's design priorities going forward, ensuring that explanatory content, educational context, and clear communication of both the heuristic engine's classification and the reasoning behind it were treated as integral features from the outset, rather than as afterthoughts added late in development. Every scan result was designed to include an explanation of what had been found, why it mattered, how the heuristic engine's combination of indicators had contributed to the classification, and what the user could do to protect themselves going forward directly addressing the educational gap this analysis of existing systems had identified.

3.6 Testing Procedures

In order to make TrustFile work properly as intended, the researchers conducted the following types of testing:

Unit Testing - This phase focused on the functionality of individual components in isolation the file hashing module, the entropy analysis module, the script scanner, the file type verifier, the heuristic classification engine, and the AI explanation layer using both valid and malicious test samples to confirm accurate performance at the component level before any of these pieces were tested together.

File hashing module testing validated that hash computation produced correct, consistent results for known inputs. This included verifying that identical files produced identical hashes, that different files produced different hashes, that the hashing algorithm executed correctly across a range of file sizes, and that hash comparison against the VirusTotal database functioned properly. Test samples included files with already-known hashes, used specifically to verify the accuracy of the comparison logic itself.

Entropy analysis module testing confirmed that entropy calculation accurately measured randomness within a file, that baseline profiles correctly represented the expected entropy range for normal file types, and that anomaly detection correctly flagged files that deviated meaningfully from those baselines. Test files included highly compressed files, encrypted files, plain text files, and structured binary files deliberately chosen to have known, contrasting entropy characteristics in order to validate both measurement accuracy and the calibration of the thresholds used to flag deviation.

Script scanner module testing verified detection of embedded executable code across different file formats. Test files included PDFs containing embedded JavaScript, Office documents containing macros, archives containing hidden scripts, and files with encoded or otherwise obfuscated script content. Both positive cases (files genuinely containing a script) and negative cases (ordinary files without any embedded script) were tested, in order to validate detection accuracy and to check for an acceptable false-positive rate.

File type verifier module testing confirmed that magic number detection, internal structure validation, and extension comparison together correctly identified a file's actual type. Test files included correctly labeled files, files with deliberately spoofed extensions, files with truncated or corrupted headers, and files constructed to resemble more than one format at once validating type detection across a range of deliberately difficult edge cases, not just straightforward ones.

Heuristic classification engine testing verified that the rule-based logic responsible for the system's five-tier classification correctly weighed the combined results of all five static analysis techniques, together with the VirusTotal lookup result, to produce an appropriate BENIGN-to-MALICIOUS classification. This testing used files exhibiting a single isolated indicator, files exhibiting multiple simultaneous indicators, files with conflicting or ambiguous indicators, and files with no indicators at all, in order to verify that the classification boundaries and the relative weighting assigned to each indicator produced results that matched the researchers' expectations across this full range of cases.

AI explanation layer testing, conducted separately from the heuristic engine's classification testing, verified that the NVIDIA NIM API and the Llama 3.1-70B-Instruct model correctly translated an already-decided classification and its supporting indicators into accurate, understandable plain-language output. This testing checked that the generated explanation correctly referenced the actual indicators the heuristic engine had flagged for a given file, that the explanation's tone and severity matched the assigned threat level, and that no inaccurate or misleading technical claims were introduced during the translation from raw indicator data into natural language.

The consistent use of both valid and malicious test samples across every module ensured comprehensive validation across the full spectrum of files the finished system was expected to encounter. Valid samples tested that the system did not falsely flag legitimate files, while malicious samples tested that genuine threats were reliably detected. This dual approach allowed the researchers to separately quantify false positive and false negative rates at the component level essential, foundational metrics before the same evaluation was repeated at the level of the whole integrated system.

System Testing - In this phase, the researchers focused on how the fully assembled system scanned files end-to-end, whether scanning completed properly, whether the correct threat level was ultimately assigned, and how the system performed as a whole, rather than testing any single component in isolation.

Scanning workflow testing validated the complete process from initial file upload through to final result presentation. This included verifying that uploaded files were correctly received, temporarily stored, and processed; that scanning completed successfully across a range of different file types and sizes; that the heuristic engine's classification logic and the VirusTotal lookup executed correctly in sequence; that the AI explanation layer correctly received and processed the finished classification; and that the final result was presented to the user in a timely and comprehensible way. This workflow-level testing surfaced integration issues that unit testing, by its isolated nature, could not have detected data format mismatches between components, timing issues in asynchronous operations, and resource conflicts arising when multiple parts of the system ran concurrently.

Threat identification accuracy testing evaluated whether the fully integrated system correctly identified and classified threats across the full range of test files used throughout this study. This involved scanning files with already-known characteristics and verifying that the classification the heuristic engine ultimately assigned matched the researchers' expected outcome for that file. This testing quantified overall system performance through metrics including detection rate, false positive rate, and false negative rate, alongside the general distribution of classifications produced across the full test set.

Heuristic engine and AI explanation layer integration testing verified that the two components correctly handed off responsibility to one another as designed that the heuristic engine's finished classification was passed accurately and completely to the AI explanation layer, and that the resulting explanation always remained faithful to that classification rather than introducing any inconsistency between the two. This testing specifically checked for cases where an explanation might understate or overstate the severity implied by its corresponding classification, ensuring the two components stayed in agreement by design rather than by coincidence.

Overall performance assessment examined system behavior under realistic operating conditions. This included measuring response times for a complete file scan, evaluating throughput under concurrent user load, assessing resource consumption in terms of CPU, memory, disk, and network usage, and verifying system stability during extended periods of continuous operation. This performance testing existed to confirm that the system met

reasonable usability expectations and could handle anticipated workloads without a meaningful drop in responsiveness particularly important given the resource ceilings imposed by the PythonAnywhere hosting tier discussed in the Scope and Delimitation.

Taken together, "working properly as intended" was understood throughout this phase to encompass functional correctness, adequate performance, reliability under load, and genuine usability for the system's intended non-technical users. System testing was designed to validate each of these dimensions in turn, providing the empirical evidence needed to judge whether the system was genuinely ready for the evaluation phase that followed.

User Acceptance Testing - This phase involved real users or staff uploading files and evaluating the system's user interface, the clarity of its reports, and their overall experience, gathering direct feedback on whether the system performed effectively and genuinely helped educate users about file security, rather than relying solely on the researchers' own judgment of the finished product.

Involving real users at this stage mattered because it grounded the evaluation in actual user perspectives rather than developer assumptions. Users brought a range of technical backgrounds, work contexts, and expectations that could reveal usability issues the researchers themselves, having built the system, might no longer notice. Their feedback surfaced confusing interface elements, missing features, and workflow mismatches that purely technical testing was never designed to catch in the first place.

File upload validation by real users tested whether the upload process felt intuitive, efficient, and reliable when used with users' own files, across different browsers, devices, and network conditions. Because the system was designed to support files of any size and virtually any extension, this stage of testing specifically validated that this universal support held up in practice, not just under the researchers' own controlled test conditions.

User interface validation assessed whether the interface design genuinely supported the tasks users needed to complete, evaluating navigation clarity, how easily needed information could be found, how discoverable key actions were, and the overall visual hierarchy of each screen. Users were asked to attempt typical tasks, such as scanning a file, interpreting its result, or locating help, while researchers observed where users hesitated, grew confused, or succeeded without difficulty.

Report clarity evaluation examined whether scan results were genuinely comprehensible and actionable for non-technical users specifically. Users were asked to interpret results without direct assistance from the researchers, which revealed whether the five-tier classification communicated risk clearly on its own, whether the AI-generated explanations were genuinely understandable without a technical background, and whether the recommended next steps felt realistic and feasible to follow. Feedback gathered at this stage fed directly back into refinements made to how results were presented and explained.

AI explanation quality evaluation specifically assessed whether users understood the reasoning behind a given classification once they had read the AI-generated explanation. Users were asked to describe, in their own words, why a particular file had received the classification it did, which revealed whether the explanation layer was successfully communicating the heuristic engine's underlying reasoning in a way that actually stuck, rather than being read and immediately forgotten.

Overall experience assessment captured broader impressions that went beyond any single feature whether the system felt trustworthy, professional, and genuinely helpful, and whether users came away feeling it was worth using again. These more subjective impressions carried real weight, since they were what would ultimately determine whether a system like TrustFile saw sustained adoption rather than a single, one-time use.

Feedback on educational effectiveness examined whether the system actually improved users' security awareness, as intended throughout this study. Users were asked whether the explanations they received had increased their understanding, whether they felt better equipped to recognize a suspicious file on their own in the future, and whether their general approach to handling files had shifted at all as a result of using the system. This feedback served as direct validation of TrustFile's educational component, discussed at length in the Significance of the Study, and helped identify specific areas where the explanation layer could still be improved.

3.7 Evaluation Criteria

TrustFile was evaluated according to the following criteria:

Functionality - This criterion assessed whether the system performed all of its main features properly file scanning, file analysis, threat classification through the heuristic engine, and clear AI-generated explanation of why a given file was flagged as harmful. All parts of the system were expected to function as intended, without errors.

File scanning functionality was evaluated by verifying that the system successfully received, processed, and analyzed uploaded files across the full range of supported types and sizes. Tests included normal files, unusually large files, very small files, files with special characters in their filenames, and files across a wide range of formats including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, RAR, and others. Success was defined by successful upload completion, correct scan initiation, full analysis execution, and accurate result generation at the end of the process.

File analysis functionality was evaluated by confirming that each individual analysis method hashing, entropy analysis, script/pattern detection, and file type verification produced correct results for test files with already-known characteristics. This evaluation compared the system's actual output against the expected output for a validated test dataset, quantifying accuracy and identifying any discrepancies that emerged.

Threat classification functionality was evaluated by verifying that the heuristic engine's rule-based logic correctly assigned a threat level based on the combined results of the five static analysis techniques and the VirusTotal lookup. Tests included files with known characteristics expected to trigger a specific classification, files with deliberately ambiguous characteristics intended to test the boundaries between adjacent classification tiers, and files with no suspicious characteristics at all, which were expected to receive a BENIGN classification.

Explanation functionality was evaluated separately, assessing whether the AI-generated explanations accurately described the indicators the heuristic engine had actually detected, correctly explained the relevant technical concepts in accessible language, and provided guidance a user could realistically act on. This evaluation involved expert review of explanation accuracy confirming that the AI layer never introduced a claim inconsistent with the heuristic engine's underlying findings alongside user comprehension testing to verify that the explanations were genuinely accessible to a non-technical reader.

The "without errors" requirement established that functional correctness also included robust handling of unexpected inputs, edge cases, and error conditions. The system was expected not to crash, hang, or produce an incorrect result when encountering a malformed file, a network interruption, a temporarily unavailable external service, or any other exceptional condition outside normal operation.

Usability - This criterion measured whether the system was genuinely easy to use, even for users without any technical background. Users were expected to be able to upload and scan files easily, with results clear and understandable enough that no separate instructions were needed to interpret them.

Ease of use for non-technical users was evaluated through task-based testing, in which representative users attempted typical workflows without any assistance from the researchers. Observers measured task completion rates, time to completion, error rates, and how often users needed to ask for help. Users also provided their own subjective ratings of difficulty and confidence. Low completion rates, unusually long completion times, frequent errors, or low user-reported confidence were all treated as clear indicators of an underlying usability problem.

Upload and scan ease was evaluated by examining the number of steps required to complete a scan, the clarity of on-screen instructions, how intuitive the interface elements felt, and the quality of feedback provided throughout the process. An ideal workflow minimized the number of required actions, relied on familiar interaction patterns, provided clear progress indication during scanning, and confirmed successful completion unambiguously. Because the system supported files of any size and virtually any extension, usability testing specifically validated that this broad compatibility never came at the cost of a more complicated user experience.

Result clarity and understandability was evaluated by testing whether users could correctly interpret a scan result without consulting any reference material. Users were asked to explain, in their own words, what they understood from a given result identifying the threat level, describing what had been found, and stating what action they believed they should take next. Any misinterpretation, confusion, or inability to extract the key

information from a result was treated as a presentation problem requiring correction. The AI-generated explanation layer was evaluated particularly closely against this standard, given that communicating complex technical findings in accessible language was its entire purpose within the system.

The "without needing instructions" criterion reflected a deliberately high usability standard: the system needed to be intuitive enough that a first-time user could accomplish their task successfully without reading a manual, watching a tutorial, or receiving any training beforehand a standard that reflected the practical reality that most users never consult documentation and expect to understand a tool immediately upon using it.

Accuracy of Detection - This criterion evaluated the system's accuracy and reliability in identifying suspicious and potentially malicious files, while minimizing incorrect results, through the heuristic engine's rule-based classification logic.

Detection accuracy was measured using three standard metrics: precision (the proportion of files flagged as a threat that were genuinely threats), recall (the proportion of actual threats the system successfully identified), and the F1-score (the harmonic mean of precision and recall, balancing the two). High precision indicated few false positives; high recall indicated few false negatives; and the F1-score captured the overall balance struck between these two competing objectives.

Because the heuristic engine's classification decision drew on five distinct static analysis indicators together with the VirusTotal lookup, evaluation also examined the accuracy contribution of each individual indicator, alongside the combined classification produced once all indicators were weighed together, helping the researchers understand which specific checks contributed most to correct classifications, and which occasionally introduced noise into the final result. The AI explanation layer, evaluated separately as described under Functionality above, was not assessed against these detection-accuracy metrics at all, since it played no role in the classification decision itself; its evaluation instead focused entirely on the accuracy and clarity of the explanation it generated for an already-determined classification.

Identification of suspicious and potentially malicious files was evaluated using test datasets with already-known labels. The system scanned these labeled files, and its resulting classifications were compared directly against the known ground truth. Accuracy was calculated separately across different file types, threat categories, and obfuscation techniques, in order to ensure the evaluation was comprehensive rather than reflecting performance on only one narrow category of test file.

Minimizing false results encompassed both false positives (benign files incorrectly flagged as threats) and false negatives (malicious files incorrectly classified as safe) both discussed in greater technical detail in the Definition of Terms. False positives caused unnecessary concern, operational disruption, and, in more serious cases, a growing temptation among users to bypass security warnings altogether. False negatives, meanwhile, allowed genuine threats to slip through undetected, directly exposing a system to compromise. Where exactly to strike the balance between the two ultimately depended on organizational risk tolerance, with higher-security

environments generally willing to accept a somewhat higher false-positive rate in exchange for minimizing the more dangerous false-negative case.

Reliability evaluation, closely related to accuracy, assessed the consistency of the heuristic engine's classifications across repeated scans of the same file, across different system configurations, and under varying operating conditions. A reliable system was expected to produce consistent results for identical inputs, maintain its accuracy under load, and degrade gracefully rather than failing outright if a supporting component, such as the VirusTotal API, became temporarily unavailable.

Security - This criterion ensured that TrustFile itself could safely handle files during the scanning process. Files were only ever read and analyzed, never executed or opened, specifically to prevent any possibility of infection if a submitted file turned out to be genuinely malicious. The system was also expected to ensure that the scanning process itself never caused harm to a user's own device.

Secure handling of files during scanning meant ensuring that the scanning process itself could not be exploited to compromise the underlying system or exfiltrate data. This included sandboxing the analysis process to limit its access to broader system resources, validating file contents carefully before processing to prevent exploits targeting the system's own file parsers, restricting network access during analysis to prevent any possibility of data exfiltration, and sanitizing outputs to guard against injection-style attacks.

Read-only analysis without execution was treated as a critical security principle underlying the entire system, consistent with the static analysis approach discussed in the Scope and Delimitation. By examining a file's contents and structure without ever executing it, TrustFile avoided the risk of triggering a malicious payload that could infect the scanning infrastructure itself. This principle was implemented through careful parser design extracting and analyzing a file's structure without ever invoking any executable content it might contain combined with sandboxed processing environments that isolated any accidental execution attempt, and ongoing monitoring designed to detect and immediately prevent any unauthorized execution. The heuristic engine and the AI explanation layer both operated exclusively on already-extracted file features entropy values, header information, structural patterns, and similar characteristics rather than on executable file content itself, preserving this read-only principle throughout the entire pipeline.

Protection of user devices extended beyond the scanning process itself to the system's broader web interface. The interface was expected to guard against cross-site scripting (XSS) attacks that could otherwise compromise a user's browser. File uploads were protected against path traversal and denial-of-service attempts. Data transmitted between the user's browser and the server was expected to be encrypted, preventing interception, and user credentials were expected to be stored and transmitted securely at every stage.

More broadly, the system was expected not to harm a user's device through excessive resource consumption, unauthorized modification, or the introduction of new vulnerabilities meaning resource use had to be carefully managed to prevent anything resembling a denial-of-service condition on the user's end, and the system itself

needed to be designed so that it could never become a vector for the very malware distribution it was meant to prevent.

Efficiency - This criterion assessed whether file scanning completed within a reasonable amount of time, processing and displaying results quickly enough to ensure a smooth overall user experience even for larger files and a diverse range of formats.

What counted as a "reasonable" scanning time depended somewhat on context file size, analysis complexity, and general user expectations all played a role. For typical documents, users generally expected a result within seconds; for larger files or more involved analysis, a slightly longer wait was considered acceptable, provided clear progress feedback was shown throughout. Because the system was designed to scan files of any size, performance optimization work specifically focused on ensuring that larger files did not introduce excessive delay relative to smaller ones. Evaluation established rough benchmarks based on general user expectations, then measured the system's actual performance against those benchmarks.

Achieving quick processing involved optimizing the underlying scanning algorithms, leveraging caching where appropriate, and minimizing unnecessary computation throughout the pipeline. The AI explanation layer, in particular, was implemented to generate its output efficiently, so that waiting for an explanation never became the bottleneck in an otherwise fast scan. Performance profiling was used to identify specific bottlenecks in the pipeline, with targeted optimization applied wherever a slowdown was found. The overall goal throughout this process was sub-second responsiveness for typical files, with graceful, rather than abrupt, degradation for larger or more complex content.

Displaying results without delay required efficient result formatting, transmission, and rendering on the client side. Results were expected to be available to the user immediately upon completion of analysis, with the interface itself optimized for rapid comprehension rather than simply rapid loading. Any noticeable delay between the completion of analysis and the actual display of results risked frustrating users and undermining confidence in the system's overall responsiveness. This broader sense of a "fast and smooth" experience extended beyond raw scanning speed alone to the system's general responsiveness — interface interactions were expected to feel immediate, transitions fluid, and feedback timely throughout every part of the experience, not just during the scan itself.

For offices and organizations that constantly received files, efficiency carried genuine practical weight rather than being a merely cosmetic concern. Users processing a high volume of incoming documents could not reasonably afford extended waiting periods for each individual scan to complete. TrustFile's optimization for rapid scanning across files of any size and virtually any extension was intended specifically to ensure that meaningful security never came at the cost of everyday productivity.

3.8 Supporting Diagrams

The supporting diagrams in this section show how TrustFile is structured and how data moves through it, from the moment a user logs in to the moment a scan report is stored and displayed. They give a visual view of the system's design alongside the written descriptions in the earlier sections of this chapter. They also give the researchers, the evaluators, and any future developers one shared reference for how the platform works.

The diagrams used are Data Flow Diagrams (DFDs). A DFD shows how data moves through a system. It identifies the external parties that send data into the system or receive data from it, the processes that transform that data, the data stores where information is kept, and the data flows that connect them. DFDs were chosen because TrustFile is a pipeline: a file enters, is examined in stages, and leaves as a classified and explained result. A diagram that follows data from stage to stage matches how the system actually works. DFDs also leave out programming-language details, so they can be read by technical and non-technical stakeholders alike.

The diagrams follow a hierarchical decomposition. Level 0, the context diagram, shows the whole system as one process, surrounded by the external entities it exchanges data with. Level 1 opens that single process into its major sub-processes and shows the data stores they use. Levels 2 and 3 each take one Level 1 process and break it down further, so the reader can zoom in only on the part of the system they need. Each lower level keeps the inputs and outputs of the process it expands, so the diagrams stay consistent with one another. This ordering lets a reader start from the overall purpose and scope, then move to the detailed behavior of the scanning pipeline.

The diagrams are tied to the design decisions described in the earlier sections. The Level 1 diagram follows the order in which the pipeline runs: authentication, upload and validation, static analysis, external threat intelligence lookup, AI explanation, risk scoring and classification, and report storage. The Level 2 diagram shows the local static analysis techniques that gather evidence from the file itself, which TrustFile uses to catch disguised, packed, or previously unseen threats. The Level 3 diagram shows how the external VirusTotal lookup is made efficient through a cache and a conditional upload path. Together, the four diagrams show the preventive, multi-layered approach of TrustFile, which examines a file before the user opens it and combines local and external evidence into one result.

The diagrams also show where TrustFile keeps data and which external services it depends on. User accounts and scan records are kept in Firebase, and a separate Firebase cache holds recent VirusTotal results for 24 hours so the same file is not queried repeatedly. VirusTotal supplies the multi-engine detection data, and an AI service supplies the plain-language explanation shown to the user. Showing these dependencies in the diagrams makes clear which parts of the system run locally and which rely on outside services. That matters for privacy, since only a file's hash is sent to VirusTotal for known files, and for reliability, since the cache reduces the number of external requests.

3.8.1 Data Flow Diagram (DFD – Level 0)

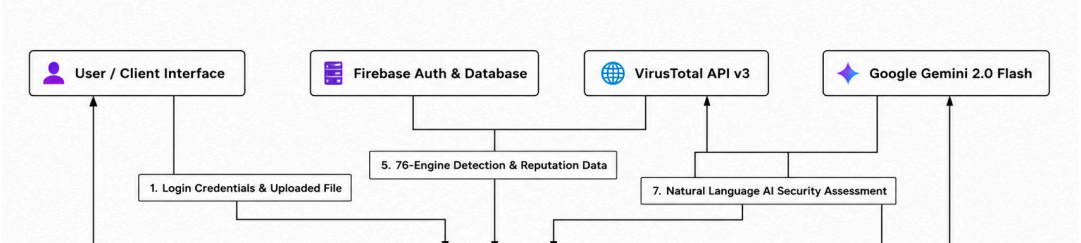


Fig. 3.1. DFD Level 0: Context diagram of the TrustFile Security Platform

The Level 0 diagram, or context diagram, shows the TrustFile Security Platform as a single process interacting with four external entities: the User / Client Interface, Firebase Auth and Database, the VirusTotal API v3, and the AI analysis service. The user sends login credentials and the uploaded file into the platform and receives a scan report, risk score, and AI-generated summary in return. The platform sends the file's SHA-256 and MD5 hashes to VirusTotal and receives multi-engine detection and reputation data. It sends the computed static indicators and risk metrics to the AI service and receives a natural-language security assessment. It also exchanges data with Firebase, which handles authentication and stores user accounts and scan history. At this level the internal workings of the platform are hidden, so the diagram shows only the system boundary and the information that crosses it.

3.8.2 Data Flow Diagram (DFD – Level 1)

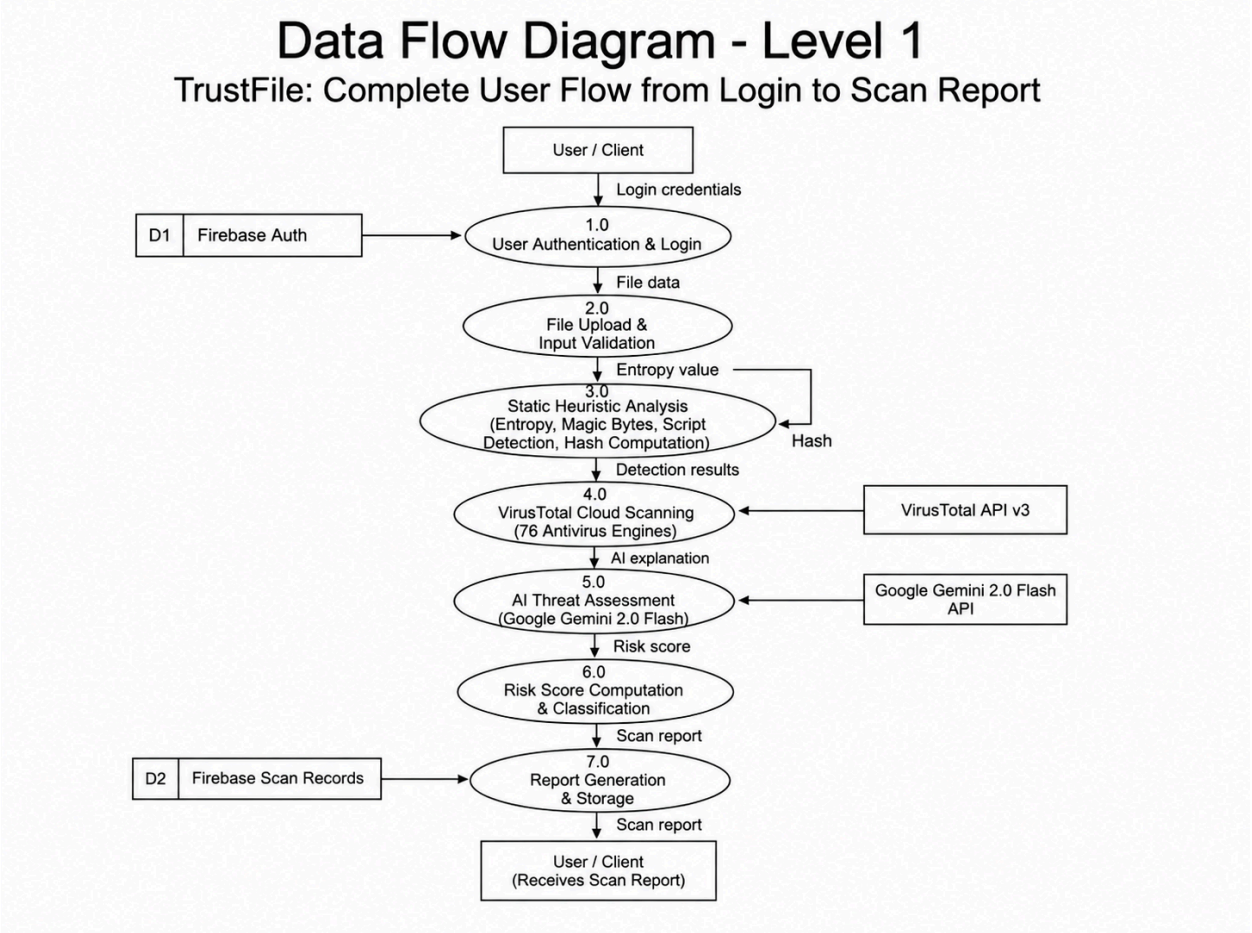


Fig. 3.2. DFD Level 1: User flow from login to scan report

The Level 1 diagram expands the single Level 0 process into seven sequential sub-processes, using two data stores: D1 (Firebase Auth) and D2 (Firebase Scan Records). In Process 1.0, the user's login credentials are validated against D1 before access is granted. In Process 2.0, the uploaded file is validated for type, size, and format. In Process 3.0, the validated file goes through static analysis, which produces the entropy value, the magic-byte result, the script detection findings, and the file hash. In Process 4.0, the hash is checked against VirusTotal's detection network. In Process 5.0, the findings are passed to the AI service, which returns a plain-language explanation. In Process 6.0, the risk score and threat classification are computed. In Process 7.0, the finished scan report is stored in D2 and returned to the user.

3.8.3 Data Flow Diagram (DFD – Level 2)

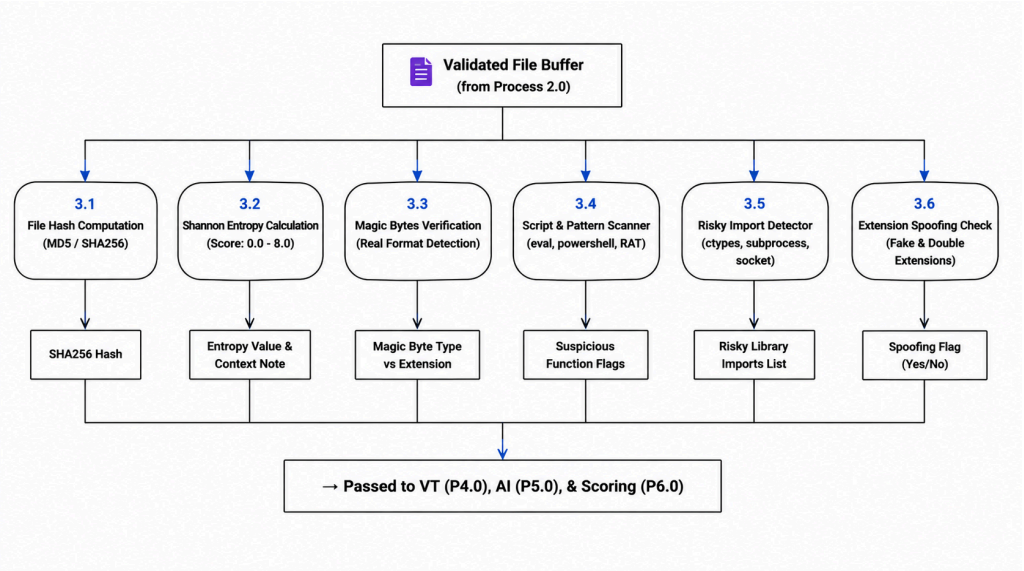


Fig. 3.3. DFD Level 2: Decomposition of Process 3.0 (Static Heuristic Analysis)

The Level 2 diagram expands Process 3.0, Static Heuristic Analysis, into six sub-processes that run on the validated file buffer from Process 2.0. Process 3.1 computes the MD5 and SHA-256 hashes, and the SHA-256 value is later used for the VirusTotal lookup. Process 3.2 calculates the Shannon entropy of the file on a scale from 0.0 to 8.0 and attaches a context note, since high entropy can point to packed, encrypted, or obfuscated content. Process 3.3 reads the magic bytes to find the file's real format and compares it with the claimed extension. Process 3.4 scans for suspicious script patterns such as eval calls, PowerShell commands, and remote-access signatures. Process 3.5 looks for risky library imports such as ctypes, subprocess, and socket. Process 3.6 checks for fake and double extensions, such as invoice.pdf.exe. The six outputs are combined and passed to the VirusTotal scan, the AI assessment, and the risk scoring.

3.8.4 Data Flow Diagram (DFD – Level 3)

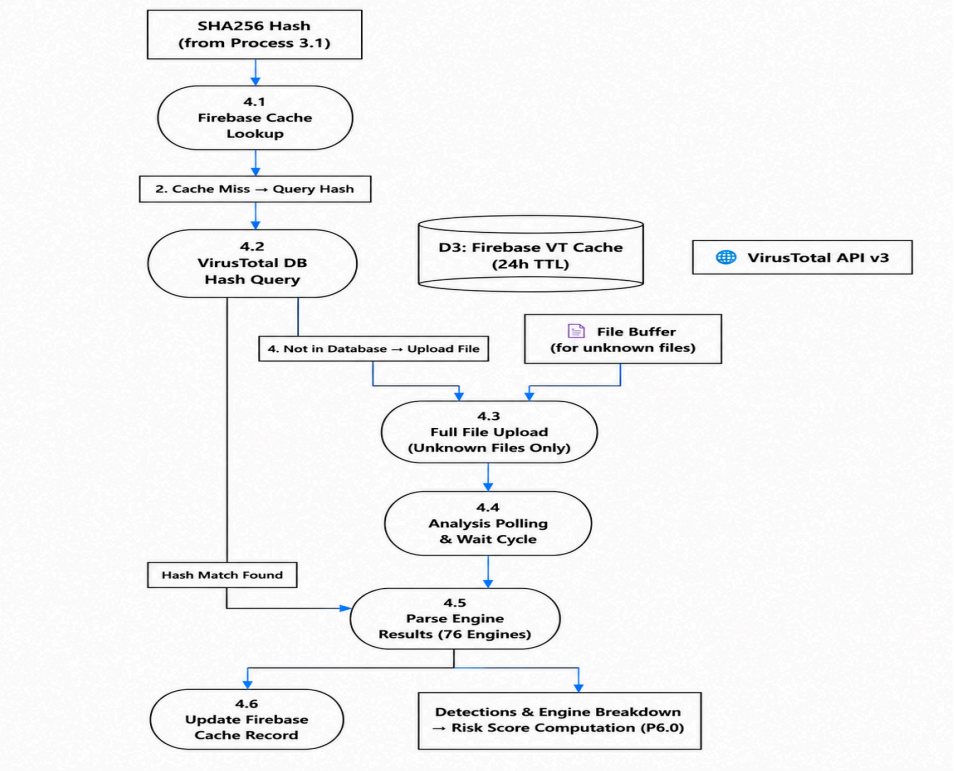


Fig. 3.4. DFD Level 3: Decomposition of Process 4.0 (VirusTotal Cloud Scanning)

The Level 3 diagram expands Process 4.0, VirusTotal Cloud Scanning, into six sub-processes built around a Firebase cache, D3, which keeps VirusTotal results for 24 hours. The SHA-256 hash from Process 3.1 first goes to Process 4.1, which checks the cache. On a cache miss, Process 4.2 queries the VirusTotal database by hash. If the hash is found, the flow goes straight to result parsing. If the hash is not in the database, Process 4.3 uploads the full file, which applies only to unknown files, and Process 4.4 repeatedly polls VirusTotal until the analysis is complete. Process 4.5 then parses the results from the 76 antivirus engines, and Process 4.6 saves them to the cache record so later lookups of the same file are answered locally. The detection counts and per-engine breakdown are passed on to risk score computation in Process 6.0.

3.8.5 Data Flow Diagram (DFD – Level 4)

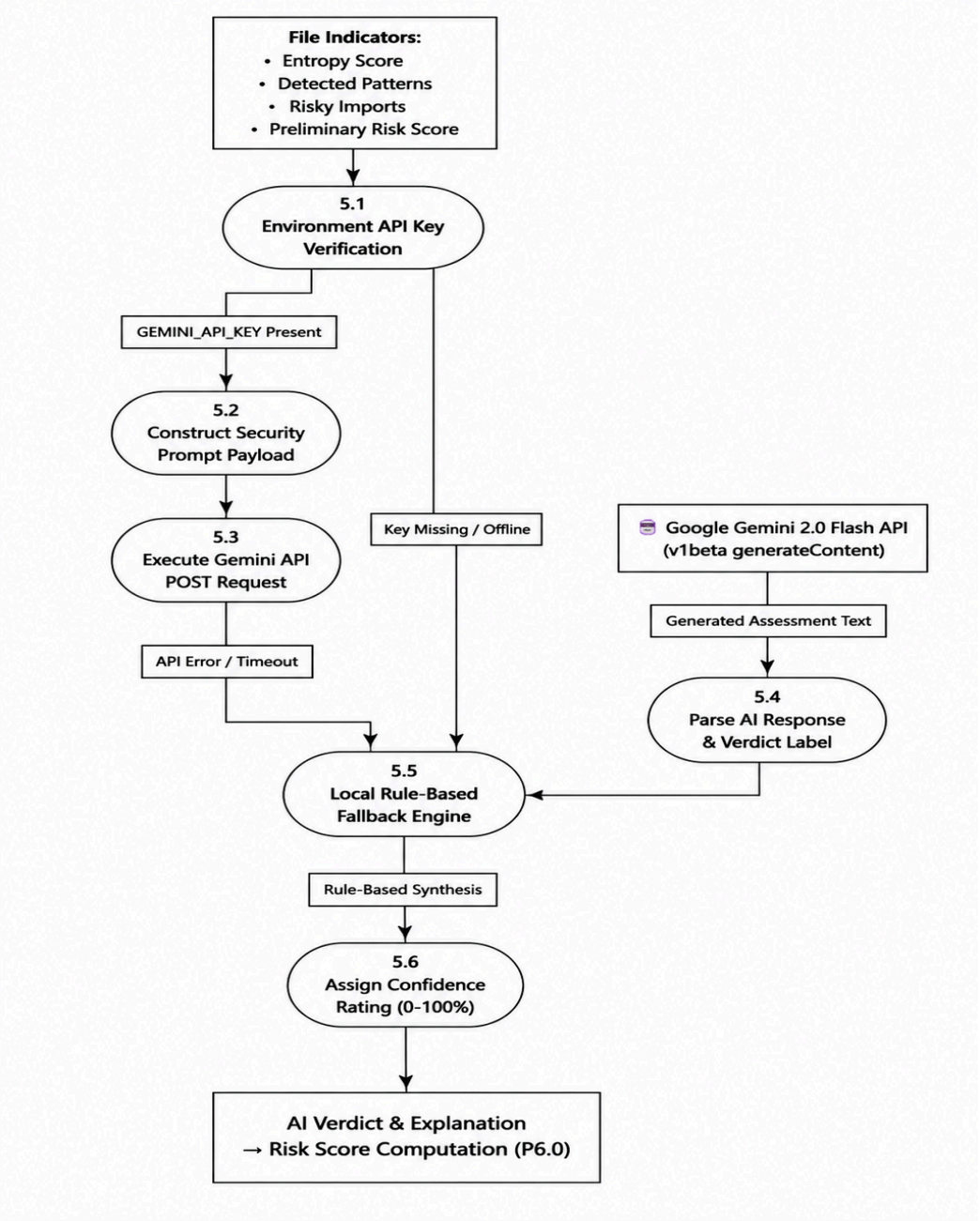


Fig. 3.5. DFD Level 4: Decomposition of Process 5.0 (AI Threat Assessment)

The Level 4 diagram expands Process 5.0, AI Threat Assessment, into six sub-processes that turn the findings of the earlier stages into a plain-language verdict and explanation. The input is the set of file indicators produced earlier, which includes the entropy score, the detected patterns, the risky imports, and the preliminary risk score. Process 5.1 first verifies that the credential needed to reach the AI service is available in the environment. If it is present, Process 5.2 builds a security prompt from the indicators, and Process 5.3 sends it to the AI service as a request. The generated assessment text then goes to Process 5.4, which parses the response and extracts the verdict label. If the credential is missing, the service is offline, or the request returns an error or times out, the flow skips to Process 5.5, a local rule-based fallback engine that builds the explanation from the indicators using

predefined rules. Both the AI path and the fallback path meet at Process 5.5, so a result is always produced. Process 5.6 then assigns a confidence rating from 0 to 100 percent, and the final AI verdict and explanation are passed on to risk score computation in Process 6.0.

3.8.6 Data Flow Diagram (DFD – Level 5)

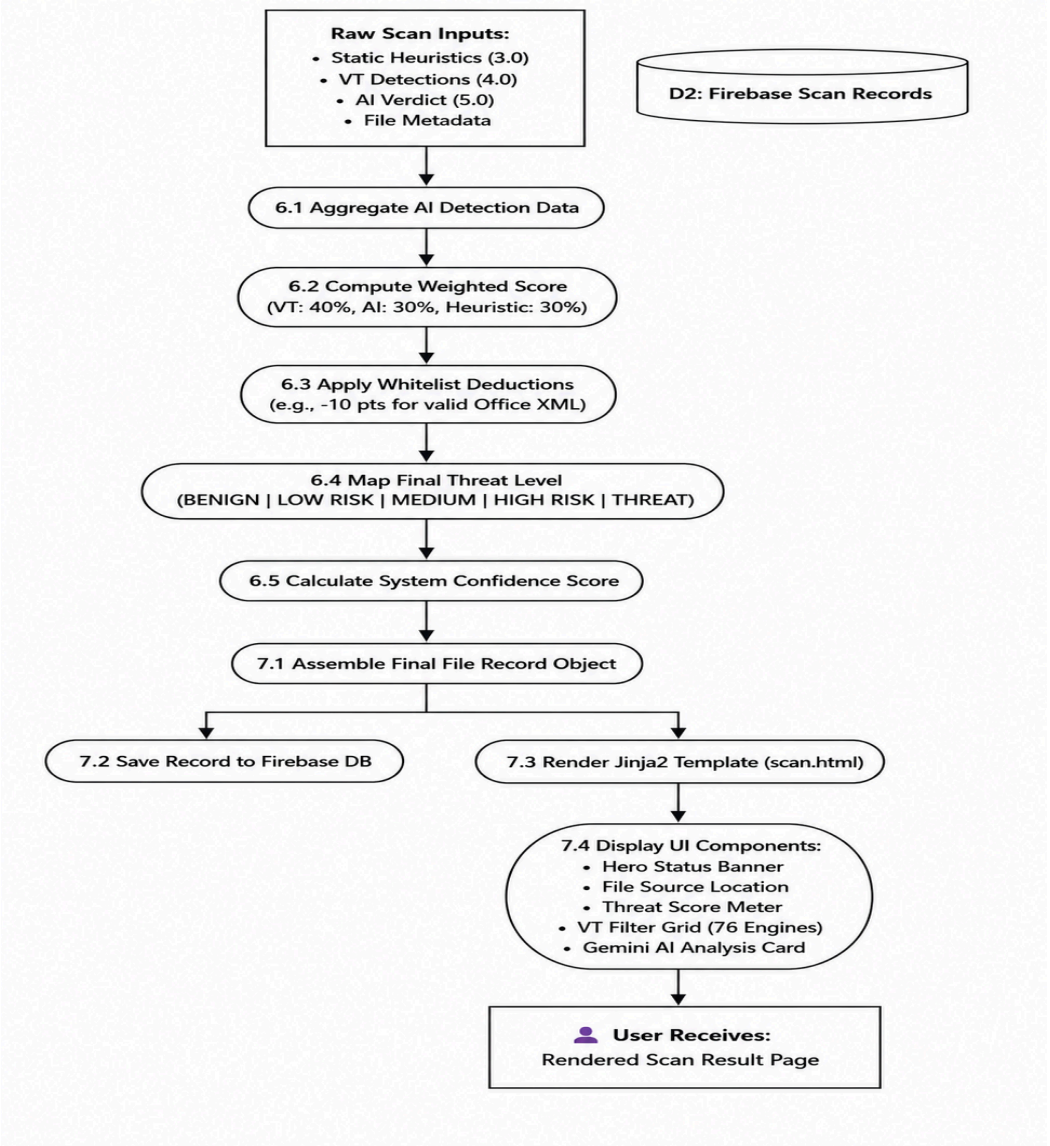


Fig. 3.6. DFD Level 5: Decomposition of Processes 6.0 and 7.0 (Risk Score Computation, Report Generation and Storage)

The Level 5 diagram expands Processes 6.0 and 7.0 into nine sub-processes that turn the raw scan evidence into a classified, stored, and displayed report. The inputs are the static heuristic results from Process 3.0, the VirusTotal detections from Process 4.0, the verdict from Process 5.0, and the file metadata. Process 6.1 first aggregates the detection data from these sources. Process 6.2 then computes a weighted score, with VirusTotal contributing 40 percent and the AI verdict and the heuristic results 30 percent each. Process 6.3 applies whitelist deductions to reduce false positives, for example subtracting 10 points for a valid Office XML structure. Process 6.4 maps the final score to one of the five threat levels (Benign, Low Risk, Medium, High Risk, or Threat), and

Process 6.5 calculates the system's confidence score. In Process 7.1 the results are assembled into a single file record object. That record is saved to the Firebase scan records store (D2) in Process 7.2 and passed to the Jinja2 scan template in Process 7.3. Process 7.4 displays the result page to the user, with the status banner, file source, threat score meter, the 76-engine VirusTotal grid, and the AI analysis card.

Entity Relationship Diagram (ERD)

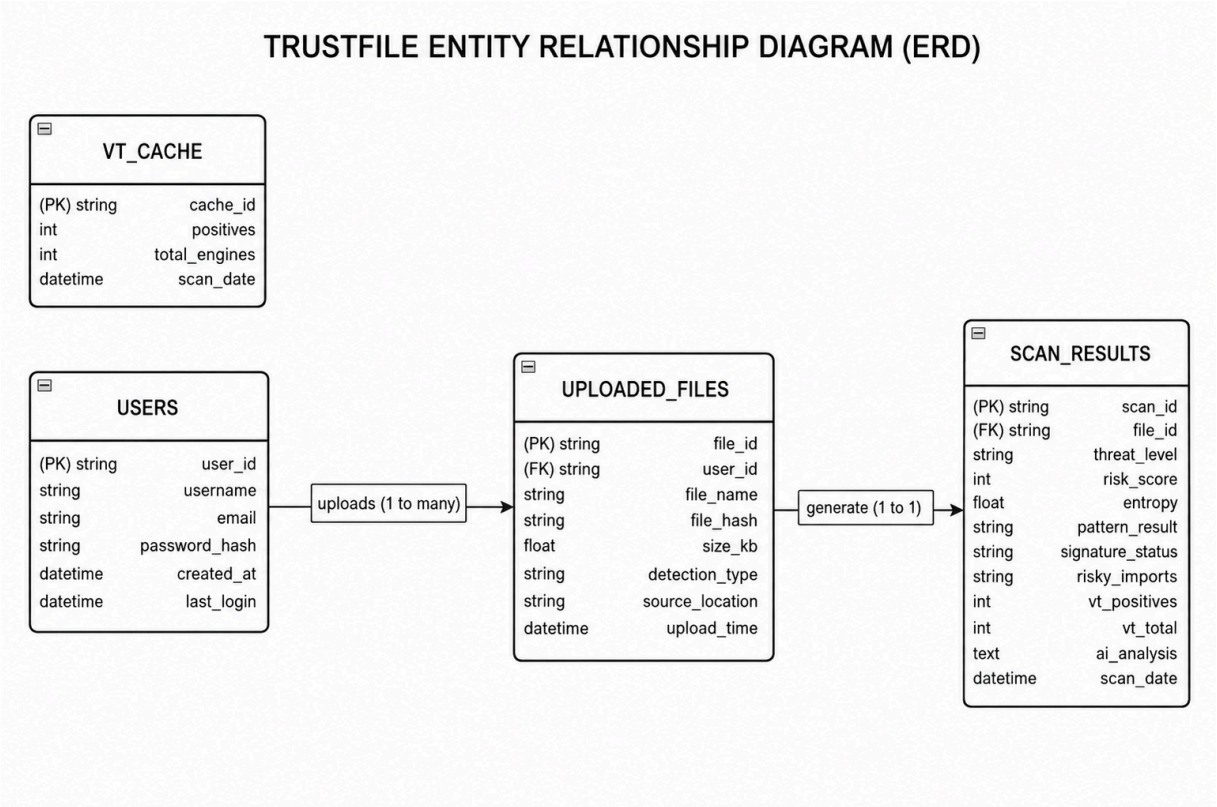


Fig. 3.7. Entity relationship diagram of the TrustFile database

The ERD shows the four data entities that TrustFile stores and how they are connected. The USERS entity holds each registered account, with a unique user_id as its primary key, along with the username, email, a hashed password, the account creation time, and the last login time. Storing only the password hash means the original password is never kept. The UPLOADED_FILES entity records each file a user submits, with file_id as its primary key and user_id as a foreign key that links the file to its owner. It also stores the file name, the file hash, the size in kilobytes, the detection type, the source location, and the upload time. The SCAN_RESULTS entity holds the outcome of each analysis, with scan_id as its primary key and file_id as a foreign key. It stores the threat level, the risk score, the entropy value, the pattern result, the signature status, the risky imports, the VirusTotal positive and total engine counts, the AI-generated analysis text, and the scan date. These fields match the evidence gathered by the static analysis, VirusTotal, and AI layers described in the DFDs. The relationships follow the order of the scanning process. One user can upload many files (1-to-many), and each uploaded file generates exactly one scan result (1-to-1). That structure means every result can be traced back to one file and one owner, so each user sees only their own history. The fourth entity, VT_CACHE, is kept apart from the

others because it is shared across users. It stores a cache_id, the number of engines that flagged the file, the total number of engines, and the scan date, so the system can reuse recent VirusTotal results instead of querying the service again for a file it has already checked.

Use Case Diagram

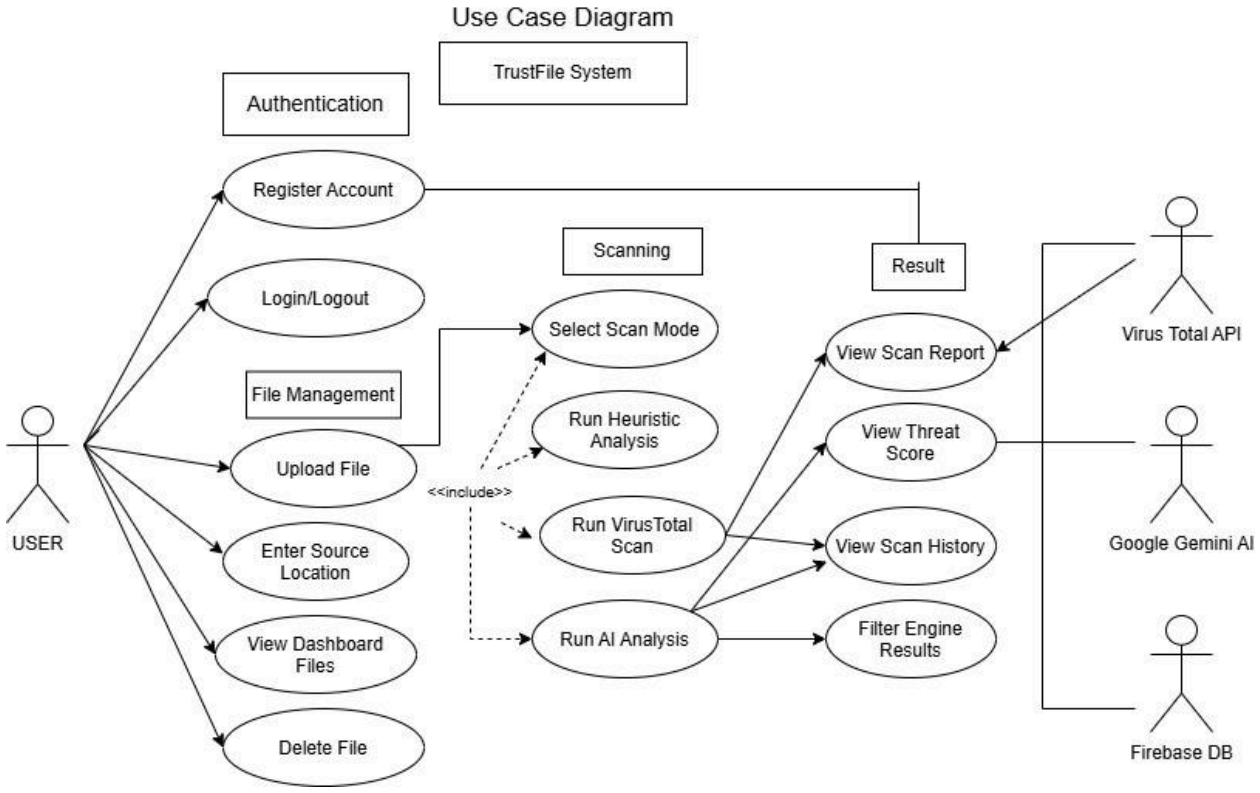


Fig. 3.8. Use case diagram of the TrustFile system

The Use Case Diagram shows what TrustFile lets a user do and which external systems support those actions. There is one human actor, the User, who can perform every function in the system. Three external actors, the VirusTotal API, the AI service, and the Firebase database, support the system's operations behind the scenes. The use cases are grouped into four areas. Under Authentication, the user can register an account and log in or out. Under File Management, the user can upload a file, enter its source location, view the files on the dashboard, and delete a file. Under Scanning, uploading a file includes the Select Scan Mode, Run Heuristic Analysis, Run VirusTotal Scan, and Run AI Analysis use cases. These are connected by include relationships, so a scan always involves these steps instead of being started as separate actions. Under Result, the user can view the scan report, view the threat score, view the scan history, and filter the individual antivirus engine results. The external actors connect to the use cases they serve. VirusTotal supplies the data behind the VirusTotal scan and the scan report, the AI service supplies the analysis behind the AI results, and Firebase stores accounts, scan records, and history. The diagram shows that TrustFile is organized around a single, simple workflow of registering, logging in, uploading a file, receiving a result, and reviewing past scans. It has no administrator role.

REFERENCE

[1] R. C. Richey and J. D. Klein, Design and Development Research: Methods, Strategies, and Issues. Mahwah, NJ, USA: Lawrence Erlbaum Associates, 2007.

[2] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," J. Manage. Inf. Syst., vol. 24, no. 3, pp. 45–77, 2007.

[3] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Q., vol. 28, no. 1, pp. 75–105, Mar. 2004.

[4] Ö. A. Aslan and R. Samet, "A comprehensive review on malware detection approaches," IEEE Access, vol. 8, pp. 6249–6271, 2020, doi: 10.1109/ACCESS.2019.2963724.

[5] Z. Bazrafshan, H. Hashemi, S. M. H. Fard, and A. Hamzeh, "A survey on heuristic malware detection techniques," in Proc. 5th Conf. Inf. Knowl. Technol. (IKT), Shiraz, Iran, 2013, pp. 113–120.

[6] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[7] MITRE Corporation, "Masquerading: Double file extension," MITRE ATT&CK, T1036.007, Aug. 2021. [Online]. Available: https://attack.mitre.org/techniques/T1036/007/ (Accessed: Sep. 25, 2026).

[8] T. Sommestad, H. Holm, and D. Steinvall, "Variables influencing the effectiveness of signature-based network intrusion detection systems," Inf. Secur. J.: A Global Perspective, vol. 31, pp. 711–728, 2021, doi: 10.1080/19393555.2021.1975853.

[9] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," Comput. Commun., vol. 198, pp. 175–185, 2023, doi: 10.1016/j.comcom.2022.11.001.

[10] TechClass, "Cybersecurity for all: Why every employee matters in data protection," TechClass Resources, Apr. 25, 2025. [Online]. Available: https://www.techclass.com/resources/learning-and-development-articles/cybersecurity-for-non-tech-teams-why-everyone-plays-a-role-in-data-protection

[11] A. L. Abdon, "RATTATA: Ransomware analysis via automated impact assessment of Philippine organizations," Int. J. Intell. Syst. Appl. Eng., vol. 12, no. 4, p. 3430, 2024. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/6858

[12] IBM Security, "Cost of a Data Breach Report 2024," IBM Corp., Armonk, NY, USA, Jul. 2024. [Online]. Available: https://www.ibm.com/reports/data-breach

[13] E. Rescorla, “HTTP over TLS,” Internet Engineering Task Force, RFC 2818, May 2000.

[14] R. Rivest, “The MD5 message-digest algorithm,” Internet Engineering Task Force, RFC 1321, Apr. 1992.

[15] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[16] VirusTotal, “VirusTotal API v3 overview,” VirusTotal Documentation. [Online]. Available: https://docs.virustotal.com/reference/overview (Accessed: Oct. 13, 2026).

[17] C. E. Shannon, “A mathematical theory of communication,” Bell Syst. Tech. J., vol. 27, no. 3, pp. 379–423, Jul. 1948.

[18] R. Lyda and J. Hamrock, “Using entropy analysis to find encrypted and packed malware,” IEEE Security & Privacy, vol. 5, no. 2, pp. 40–45, Mar./Apr. 2007.

[19] D. Gibert, C. Mateu, J. Planes, and R. Vicens, “Classification of malware by using structural entropy on convolutional neural networks,” in Proc. AAAI Conf. Artif. Intell., vol. 32, 2018, doi: 10.1609/aaai.v32i1.11409.

[20] O. Khalid, S. Ullah, T. Ahmad, et al., “An insight into the machine-learning-based fileless malware detection,” Sensors, vol. 23, no. 2, Art. no. 612, 2023, doi: 10.3390/s23020612.

[21] M. Sikorski and A. Honig, Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software. San Francisco, CA, USA: No Starch Press, 2012.

[22] European Institute for Computer Antivirus Research (EICAR), “EICAR standard anti-malware test file.” [Online]. Available: https://www.eicar.org/download-anti-malware-testfile/ (Accessed: Oct. 13, 2026).

[23] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” Inf. Process. Manage., vol. 45, no. 4, pp. 427–437, Jul. 2009.

[24] K. Beck et al., “Manifesto for Agile Software Development,” Agile Alliance, 2001. [Online]. Available: https://agilemanifesto.org (Accessed: Oct. 13, 2026).

[25] C. Larman and V. R. Basili, “Iterative and incremental developments: A brief history,” Computer, vol. 36, no. 6, pp. 47–56, Jun. 2003.

[26] W. W. Royce, “Managing the development of large software systems,” in Proc. IEEE WESCON, Los Angeles, CA, USA, Aug. 1970, pp. 1–9.

[27] J. Highsmith and A. Cockburn, “Agile software development: The business of innovation,” Computer, vol. 34, no. 9, pp. 120–127, Sep. 2001.

[28] K. Schwaber and J. Sutherland, “The Scrum Guide,” Nov. 2020. [Online]. Available: https://scrumguides.org (Accessed: Oct. 13, 2026).

[29] T. Dybå and T. Dingsøyr, “Empirical studies of agile software development: A systematic review,” Inf. Softw. Technol., vol. 50, no. 9–10, pp. 833–859, Aug. 2008.

[30] A. Dubey et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[31] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” MIS Q., vol. 13, no. 3, pp. 319–340, Sep. 1989.

[32] A. Adadi and M. Berrada, “Peeking inside the black-box: A survey on explainable artificial intelligence (XAI),” IEEE Access, vol. 6, pp. 52138–52160, 2018.

[33] A. J. Elliot and M. A. Maier, “Color psychology: Effects of perceiving color on psychological functioning in humans,” Annu. Rev. Psychol., vol. 65, pp. 95–120, 2014.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Overview of the Developed System

TrustFile is a web based file security platform designed to help individuals and organizations evaluate the safety of files before opening, executing, or distributing them. Rather than relying on a single detection method, TrustFile combines several complementary layers of analysis into one scanning pipeline. A set of local static analysis techniques inspects the contents and structure of each file, a rule based heuristic engine weighs the

findings together, an integration with the VirusTotal API cross-references the file's hash against the detection results of many commercial antivirus engines, and an AI assisted explanation layer translates the technical findings into a plain language summary that a non-technical user can understand and act on. The result is a single, consolidated risk assessment that tells the user not only whether a file appears safe, but also why it received that assessment.

Unlike traditional antivirus software, which typically operates as a background service installed on a device, TrustFile is delivered as an accessible web application, hosted on the PythonAnywhere cloud platform. Users sign in, upload a file through their browser, and receive the assessment without installing or configuring any specialized security software. This lowers the barrier to entry for individuals and small teams who want file level threat visibility but do not have dedicated IT security staff. The accessibility of this approach is especially relevant in the Philippine context, where the literature reports a lack of cybersecurity investment that has already led to major data leakages and compromised services [1]. Organizations in that position often cannot afford commercial security suites or the personnel to maintain them, and a browser based tool that works from any internet connected device addresses that limitation directly. Because the platform is centrally hosted, improvements to the scanning logic, the heuristic rules, or the explanation layer can also be applied immediately for every user, without any update process on their side.

The central purpose of the developed system is preventive. Most conventional antivirus tools are reactive, in the sense that they remove an infection after it has occurred or block a threat only when it matches a known signature. TrustFile instead evaluates a file before it is opened, so that the decision to trust the file is made with full information. In the manner of the security checkpoint described in the system architecture, each file is examined and judged before the user is given access to it. This moment, right before a user clicks "open," is where many file based attacks succeed, and it is therefore where the system concentrates its protection.

The local analysis consists of five static techniques, each designed to reveal a different kind of warning sign:

1. Cryptographic hashing: The system computes a fixed length fingerprint of the file's contents. Because even a single changed byte produces a completely different hash, the fingerprint can be used to recognize files that are already known to be malicious, and it also provides the value that is sent for the external lookup.
2. Entropy analysis: The system measures how random the data inside the file is. Because compression and encryption increase randomness, unusually high entropy can indicate packed, encrypted, or obfuscated content, which can be valuable against threats that do not yet appear in any threat database [2]. Entropy alone is not treated as a verdict, since many legitimate files such as archives and media are naturally high in entropy, so it is weighed together with the other indicators.
3. Script detection: The system looks for embedded script content that may indicate hidden executable behavior, which is a common technique for delivering malicious code inside files that appear ordinary.
4. File size verification: The system checks whether a file's size falls outside the normal range for its type, since an unusual size can suggest that a file has been padded, altered, or disguised.

5. File type verification: The system compares a file's true format, identified from its internal signature, with the format its extension claims. A mismatch is a strong indicator of extension spoofing, in which a dangerous file is disguised as a harmless one.

All five techniques work without ever executing the file. This is an important design choice, because approaches that observe a program's behavior during execution require an execution environment that introduces risk, performance costs, and complexity [3]. By examining the file in a purely static way, TrustFile can inspect even files that may be dangerous without running the danger. This choice also defines a limit of the system, since static analysis cannot catch threats that reveal themselves only when running, as stated in the Scope and Limitations of the study.

The local findings are complemented by an external threat intelligence lookup. TrustFile computes the file's cryptographic hash and sends only that hash to the VirusTotal service, which compares it against the detection results of many antivirus engines and returns whether the file is already known to be malicious. Sending the hash and not the file protects the privacy of sensitive documents, because the contents of the file never leave the system during the lookup. The lookup gives the system access to global threat intelligence that no single local tool could maintain, and it compensates for a weakness of local analysis, which excels at catching novel threats but may miss a known threat that has already been documented. The two approaches strengthen each other, which is the idea of defense in depth. The system is also designed so that it keeps working if this external service is temporarily unavailable. In that case, the scan continues with the local analysis, and the layered architecture degrades gracefully and does not fail outright.

After the individual checks are completed, the rule based heuristic engine weighs all of the results together to reach a single classification. The engine considers the hash result, the entropy value, whether a script was detected, whether the file size is unusual, whether the file's type matches its extension, and the external intelligence result, and it applies a defined set of rules and thresholds that account for how these indicators relate to each other. This mirrors how an experienced analyst would reason, by weighing several pieces of evidence and not fixating on a single red flag. Each file is placed in one of five threat levels:

1. BENIGN: The file shows no suspicious characteristics.
2. LOW RISK: The file shows minor anomalies that have a reasonable, legitimate explanation.
3. SUSPICIOUS: The file shows unusual characteristics that warrant a closer look.
4. HIGH RISK: Several strong indicators point toward malicious activity, although the file is not confirmed as malicious.
5. MALICIOUS: The file is confirmed as malicious through strong evidence, such as a direct match against known malware or overwhelming corroborating indicators.

A five level scale gives the user a more honest picture than a simple safe or unsafe result, which would treat a file with a single minor anomaly the same as one that matches known malware. Because the rules look for

suspicious structural patterns and known obfuscation techniques and do not depend only on exact matches, the heuristic engine also helps with threats that no external database has yet encountered.

Once the heuristic engine has decided the classification, the finished result and its supporting indicators are passed to the AI explanation layer, which is built on the NVIDIA NIM API and the Llama 3.1 70B Instruct model. The role of this layer is limited to explanation. It does not change or override the classification that the rule based engine reached. It takes the specific combination of indicators found in that individual file and writes an explanation of what was found, why it matters, and what the user should do next, instead of drawing on a generic template. For instance, instead of merely reporting that a threat was detected, the system can explain that an unusually high entropy value suggests packing or encryption, that embedded script content was found, and that the file should be blocked and checked with an updated antivirus tool. This design keeps the classification consistent and rule based while still giving users an explanation in language they can understand. It also responds to the human side of security, since human error is reported to be the root cause in most breaches [4], which makes clear explanations a core part of protection and not an optional extra.

The results are presented in a way that supports quick understanding. Each of the five levels has a distinct color and a short message that reinforces the assessment at a glance, following familiar associations such as green for safe and red for confirmed danger. The most important information, the threat level and any recommended action, appears prominently, while the supporting technical details are available for users who want to see them. In this way the interface hides technical complexity behind a simple presentation, without removing the analytical depth underneath it.

The system also controls who can use it and keeps a record of its use. Access is protected by server-side authentication and session management, so that each user signs in before uploading files, and the data of each user is kept separate from that of others. For every scan, the system maintains a historical record of the scanned file, its assigned threat level, and the indicators that were detected, which the user can review later. This record supports follow-up and reporting, and it lets users see how the system classified files over time.

The completed system consists of several interconnected modules that work together to accept, analyze, store, and present scan results. These modules are:

1. User authentication module: Handles sign in, session management, and the separation of each user’s data.
2. File upload and validation module: Receives files from the browser, checks that they can be scanned, and temporarily stores them during analysis.
3. Static analysis module: Performs the hashing, entropy analysis, script detection, file size verification, and file type verification.
4. Threat intelligence module: Sends the file’s hash to VirusTotal and handles the response, including the case where the service is unavailable.
5. Heuristic classification module: Applies the rules and thresholds to assign one of the five threat levels.
6. AI explanation module: Produces the plain language explanation for the finished classification.

7. Results presentation module: Displays the threat level, explanation, and supporting details in a clear and layered form.
8. Scan history module: Stores the record of each scan for later review.

Each module performs a specific function and passes its output to the next, so that a scan moves from upload to final result in one continuous flow. The system handles one file at a time, which keeps each scan focused and keeps the scope realistic for an academic project. It does not provide real time protection or continuous monitoring, and it scans only the files that a user chooses to upload.

The major objectives achieved by the developed system are summarized as follows:

- Developed a web based platform that allows users to upload files for security scanning before opening them.
- Combined five static analysis techniques, a rule based heuristic engine, and an external threat intelligence lookup into a single scanning pipeline.
- Classified each scanned file into one of five threat levels, ranging from BENIGN to MALICIOUS.
- Applied entropy analysis to help detect encrypted, compressed, or otherwise obfuscated files.
- Connected the system to an external malware detection service while protecting the privacy of the file's contents.
- Generated plain language, AI assisted explanations so that non-technical users can understand the results without security expertise.
- Maintained a historical record of scanned files, threat levels, and detected indicators for later review.
- Ensured secure, authenticated, per-user access to the system and to each user's scan history.

The development of TrustFile followed modern secure software engineering practices. These included server-side authentication and session management, environment based configuration of sensitive credentials so that they are not written into the source code, per-user data isolation, and a layered detection architecture that continues to work when a single detection source is temporarily unavailable. Throughout development, attention was given to usability, accessibility, and maintainability, so that the application remains practical for users with varying levels of technical expertise. The following sections describe how the system was developed, how it is structured, and how it performed during testing.

4.2 System Development Methodology

The development of TrustFile adopted the Agile Software Development Methodology, an iterative approach that emphasizes incremental delivery, continuous testing, and regular refinement based on feedback from working versions of the software. Instead of completing every stage in full before the next one began, as in a strictly sequential model, the work was divided into small cycles in which a portion of the system was designed, built, tested, and reviewed before the next portion was started. Agile was selected because the system's detection logic, particularly the heuristic engine, required repeated tuning as new false positives and edge cases were identified during hands-on testing, which a strictly sequential model would not have accommodated effectively.

The nature of the problem also favored this approach. Malware detection is a field of continuous change, in which the arms race between malware sophistication and detection capability means that defenses must be able to adapt over time [3].

4.2.1 Requirements Gathering

The initial phase focused on identifying the core problem that TrustFile addresses: the lack of an accessible, consolidated way for individuals without dedicated security tooling to evaluate whether a file is safe to open. The requirements were shaped by considering common attack patterns that are worth detecting, including code execution, process spawning, reverse shell behavior, data exfiltration, and obfuscated payloads [3], alongside the practical need for results that non-technical users could understand. The reviewed literature in Chapter II also informed this phase, since it showed which detection techniques were effective and where existing tools fell short.

The requirements were organized into functional and non-functional requirements. The functional requirements described what the system had to do. Authenticated users had to be able to upload a file through a web browser for scanning before they opened it, and the system had to perform static analysis through hashing, entropy analysis, script detection, file size verification, and file type verification. It also had to query an external threat intelligence service using the file's hash, classify each file into one of five threat levels using a rule based heuristic engine, generate a plain language explanation of the classification, and keep a record of each scan for later review.

The non-functional requirements described how the system had to behave. It had to be simple enough for users without technical knowledge, protect the privacy of uploaded files by sending only hashes to external services, remain usable when an external service was unavailable, and be accessible from a standard browser without installation. These requirements became the basis for the design and for the later evaluation of whether the system solved the problems it was meant to address.

4.2.2 System Planning

Following requirements gathering, the project's scope was defined around a Flask based web application with three integrated sources of analysis: the local static analysis and heuristic engine, the VirusTotal service, and the AI explanation layer. The scope was kept deliberately narrow. The system scans one uploaded file at a time and uses static analysis only, which kept the project realistic within an academic timeframe while still addressing the core problem. Planning also included selecting a cloud-hosted data store for scan results and history, and a cloud hosting platform so that users could reach the system without configuring a server themselves.

In keeping with the Agile approach, the work was ordered by priority. The core scanning pipeline was built first, because every other feature depended on it. The external threat intelligence lookup and the explanation layer

followed, and supporting features such as scan history were added afterward. This ordering meant that a working version of the most important function existed early, and that later work could be tested against it.

4.2.3 System Design

During this phase, the application's route structure, detection pipeline, and page layouts were designed. The detection pipeline was planned as a series of stages in which each static technique produced a finding, and the findings were then passed to the heuristic engine for a single, combined decision. The user interface adopted a dark, security themed visual language with glassmorphism panels and a consistent red accent color to reinforce the application's security focused purpose. Each of the five threat levels was paired with its own color and short message so that the outcome could be understood at a glance, and technical details were placed behind progressive disclosure so that they did not overwhelm users who wanted only the verdict.

Wireframes were produced for each major screen, including the upload flow, the scan results view, and the scan history, as well as the supporting screens needed for signing in and managing an account. Designing the screens before building them helped keep the number of steps to a minimum and ensured that the plain language explanations had a clear place in the results layout.

4.2.4 Software Development

Implementation proceeded incrementally. The heuristic engine was developed first as a multi-stage detection pipeline capable of operating on raw file bytes, independent of whether a file remained on disk. This design accommodates cases in which antivirus software on the host machine deletes an uploaded file before TrustFile can complete its analysis, since the analysis does not depend on the file continuing to exist in storage.

The VirusTotal integration and the AI explanation layer were developed as parallel, independently failing modules, so that a fault in one of them would not prevent the others from completing. If the external service could not be reached, for instance, the scan could still finish with the local analysis, and a failure in the explanation layer did not affect the classification that the rule based engine had already decided. Sensitive settings, such as service keys, were kept out of the source code and supplied through configuration. Authentication and session handling were added so that each user's uploads and scan history remained separate from those of other users.

4.2.5 Testing

Each module underwent iterative testing throughout development. The heuristic engine in particular required repeated refinement, because heuristic methods can generate false positives that erode user trust [3]. An early version produced a false positive on a known-clean test script. This led to the introduction of a string-literal-stripping step and tighter pattern matching, which reduced noise from comments and string content

that had been mistaken for executable threats. This outcome reflects the broader finding that misclassification challenges call for continuous refinement of how features are chosen and evaluated [5].

Special handling was also introduced for industry standard benign test files, such as the EICAR test file, so that these files are recognized for what they are and are not presented to the user as genuine threats. Integration testing then verified that the heuristic and VirusTotal results merged correctly into a single consolidated threat level, and that the explanation generated by the AI layer matched the indicators actually found in each file without changing the classification.

4.2.6 Deployment

TrustFile was deployed to the PythonAnywhere cloud platform, which allows the application to be reached from any internet connected device through a browser. The VirusTotal API key, the keys for the AI service, and the application's secret key were provided to the hosting environment as configuration values and not bundled into the source files, in keeping with standard practice for securing credentials in a cloud deployment. Uploaded files are used only transiently during a scan and are not treated as the durable source of truth. Scan metadata, results, and history are stored in the data store, while local disk storage is used only for the time needed to complete a scan.

4.2.7 Maintenance

Ongoing maintenance involves monitoring the heuristic engine for false positives and false negatives, updating detection patterns as new threat behaviors are identified, and responding to changes in the VirusTotal API. Maintenance of this kind is necessary because threat actors continuously refine their techniques in response to defensive measures [1], so a detection system that is not updated gradually becomes less effective. Because the AI explanation layer depends on an external service, maintenance also includes verifying that the quality of the explanations remains consistent as the analysis inputs change. Together, these activities allow the system to keep improving after its initial release, following the same iterative principle that guided its development.

4.3 System Architecture

TrustFile utilizes a Three-Tier Client-Server Architecture, separating the presentation layer, application layer, and data layer. This separation improves maintainability by isolating detection logic from interface rendering, and improves scalability by allowing the data layer to be hosted independently of the application server. Each tier has a single, well-defined responsibility and communicates with its neighbors only through clearly defined interfaces. As a result, a change inside one tier, such as redesigning a page, adding a detection rule, or moving the database to a different cloud region, can be made without rewriting the other two. This was an important consideration for a project built iteratively, since the detection logic went through repeated refinement during testing while the interface and storage layers remained stable.

The three-tier model also fits the nature of the problem. A file security platform has to accept untrusted input from users, process it with computationally heavy analysis, and present the results in a form that non-technical readers can act on. Keeping the user-facing pages apart from the analysis code means a malformed or hostile file only ever interacts with the application layer, never directly with the interface or the stored records. Keeping the data layer separate means that scan history and user accounts survive independently of whatever happens to the application server, which matters on a hosting platform with an ephemeral filesystem. In the sections that follow, each layer is described in terms of what it does, why it was designed that way, and how it connects to the others.

4.3.1 Presentation Layer

The presentation layer is implemented using Flask's Jinja2 templating engine, rendering server-side HTML pages styled with a custom dark, glassmorphism-based interface. Users interact with the system through a standard web browser on desktop or mobile devices, with no plugin, extension, or local installation required. Because the pages are rendered on the server, the browser receives finished HTML rather than raw detection data, which keeps the client lightweight and keeps sensitive analysis logic out of reach of anyone inspecting the page in their browser. The dark, glassmorphism-based styling uses translucent panels, soft contrast, and consistent spacing to keep the interface readable during long sessions and to make the severity colors attached to each threat level stand out clearly against the background. This visual consistency supports one of the study's central goals: that a person with no cybersecurity background should be able to look at a result page and immediately understand how worried they should be.

The entry point to the system is the Login and Sign-Up pages, which let each user create an account and sign in before reaching any scanning functionality. Tying every scan to an individual account is what makes personal history, per-user settings, and private dashboards possible, and it also means a user's files and results are never mixed with those of other users.

Once signed in, the user lands on the Dashboard, which serves as the system's home screen and gives an at-a-glance view of their overall file safety. The dashboard displays summary statistics and a live feed of recent detections. Rather than requiring the user to refresh the page, the dashboard live-polls the server at regular intervals so that new detections, including those raised by the real-time monitor, appear almost as soon as they are recorded. This gives the user continuous situational awareness without needing to remember to check.

The File Upload page is where the core workflow begins. It gives the user a simple way to select a file and submit it for analysis, with feedback on the progress of the scan so that the user is never left wondering whether anything is happening. The page was kept deliberately minimal, because the upload step is the one every user must complete, and any confusion here would discourage use of the whole system.

After a scan completes, the Scan Results page presents the outcome. Its most distinctive feature is the per-engine breakdown, which shows the findings of the heuristic engine, the VirusTotal lookup, and the AI explanation as

separate, clearly labeled sections. Showing the engines separately, instead of merging everything into one opaque verdict, directly addresses the “black box” problem identified earlier in this study. A user can see what the local analysis found, what the global threat intelligence community says about the same file, and how the AI explains the final classification in plain language. The threat level itself is displayed prominently, with its assigned color, so that the most important information comes first and the supporting detail follows.

The History page keeps a record of every file the user has previously scanned, along with the results. This lets a user return to an earlier result, compare how different files were classified, and look back over what they have checked. Over time, it also serves as an informal learning record, since users can revisit earlier explanations and notice patterns in what gets flagged.

The Reports page summarizes the user’s activity at a higher level. It provides a severity breakdown showing how many scanned files fell into each threat level, and a seven-day trend visualization showing how detections have changed over the past week. These visual summaries make it easier to spot unusual spikes in suspicious files and give organizations a simple way to understand their overall exposure without reading individual scan reports one by one.

Finally, the Settings page lets users adjust their scan preferences and control the real-time monitor. This keeps the system flexible: users who want stricter handling of risky files, or who want to enable or disable monitoring, can do so without involving an administrator. Because the settings are stored per user, one person’s preferences never affect another’s experience.

4.3.2 Application Layer

The application layer, implemented in Python using the Flask framework, contains the system’s business logic. It is the working center of TrustFile, since every important decision the system makes, from whether a request is allowed to how a file is classified, happens here. Python was a natural choice because of its strong ecosystem for file handling, byte-level analysis, and network communication, and because Flask provides a lightweight foundation that does not impose more structure than a project of this size needs. Its responsibilities are described below.

The first responsibility is User Authentication and Session Management, handled through Flask-Login. When a user signs in, the application layer establishes a session that identifies them on every subsequent request, and it refuses access to protected pages for anyone who is not signed in. This is the foundation on which the rest of the system’s privacy guarantees rest, because every later operation, including uploading, viewing results, and reading history, is performed on behalf of an identified user.

The second responsibility is File Upload Handling and Validation. When a file arrives, the application layer checks that the submission is well-formed and can actually be scanned before any analysis begins. It also manages the temporary storage of the file during analysis. Treating every upload as untrusted input, and

validating it before anything else touches it, is a basic security practice, and it matters especially in a system whose entire purpose is to handle files that may be hostile.

The third and most substantial responsibility is Advanced Heuristic Analysis, a multi-stage static analysis process that examines a file without ever executing it. It begins with file-type and magic-byte verification, which checks the fixed byte sequence at the start of a file against what its extension claims, catching disguised files such as an executable renamed to look like a document. It continues with embedded-content and polyglot detection, which looks for files that are valid in more than one format at once, or that carry a second file hidden inside the first. Indicators of compromise (IOCs) are then extracted from the file's contents, meaning observable traces such as embedded addresses or other artifacts commonly associated with malicious activity. Entropy analysis measures the randomness of the file's data to detect encryption or packing, and it is applied with false-positive guards, so that legitimately high-entropy formats such as compressed archives and media files are not unfairly penalized. For executable files, PE import inspection examines which system functions the program declares it will use, since certain combinations of imports are characteristic of malicious behavior. Script and VBA macro analysis searches for executable code embedded in documents and other files that should not normally contain any, which is one of the most common ways malicious files are disguised. Archive inspection looks inside compressed containers rather than treating them as opaque blobs, so that a threat tucked inside a ZIP or RAR file does not escape notice simply because it is wrapped. Finally, the findings of all these stages are combined through weighted multi-signal scoring with confidence tiers. Rather than letting any single check decide a file's fate, each signal contributes to a combined score according to its importance, and the result carries a confidence indication showing how strongly the evidence supports it. This design reflects a principle that runs through the whole study: several weak signals that agree can be more meaningful than one strong signal alone, and a single ambiguous signal should never be enough to condemn a file.

The fourth responsibility is the VirusTotal API Integration, which connects the system to global threat intelligence. The application layer can look up a file's hash against VirusTotal's aggregated database to learn whether it has already been identified as malicious, and it caches the results of these lookups so that repeated queries for the same file do not consume additional API quota or slow down the scan. The integration also supports full-file scanning, which allows a file to be submitted to VirusTotal for analysis when a hash lookup alone is not enough. Because this external service is a separate organization's infrastructure, the application layer is also responsible for handling situations in which it is slow or unavailable, so that a problem on the external side does not prevent a scan from completing.

The fifth responsibility is AI-Assisted Explanation Generation. After the classification has been decided, the application layer sends the finished result and its supporting indicators to the AI model through the NVIDIA NIM API, and receives a plain language explanation written in terms a non-technical user can understand. The AI layer does not influence the threat level. Its role is to translate an already decided technical outcome into guidance that a person can act on, which keeps the classification consistent, rule-based, and reproducible while still delivering an explanation tailored to the specific indicators found in that particular file.

The sixth responsibility is Risk Score Aggregation and Threat-Level Classification. Here the application layer takes the heuristic score and the VirusTotal result together and maps them onto one of the five threat levels: Benign, Low Risk, Suspicious, High Risk, or Malicious. This final step turns a collection of technical findings into one clear verdict that can be displayed, stored, and acted upon. It is also where the system's response to a file is determined, including whether the file is allowed through, restricted, or quarantined.

The application server acts as the intermediary between the presentation layer and the data layer, and additionally coordinates outbound requests to the third-party VirusTotal API. In practice, this means that the browser never talks directly to the database or to any external service. Every request travels from the browser to the application server, which decides what to do with it, performs whatever analysis or lookups are needed, reads or writes the data layer on the user's behalf, and returns a finished page. This single point of mediation makes it far easier to enforce access control, validate input, and keep credentials for external services private.

4.3.3 Data Layer

The data layer stores all application data in Firebase Realtime Database, a cloud-hosted NoSQL database. It is responsible for persisting user accounts, uploaded file metadata, scan results, and per-user settings, and for supporting the low-latency reads required by the dashboard's live-updating detection feed. Firebase was chosen for two main reasons. First, because it is cloud-hosted and independent of the application server, it keeps data safe even if the server's local storage is reset, which is a real concern on hosting platforms with an ephemeral filesystem. Second, its real-time design suits the dashboard, which needs to reflect new detections quickly without heavy database queries on every refresh. Its document-style, NoSQL structure also fits the shape of the data well, since scan results contain a varying set of indicators and details for each file, which would be awkward to force into fixed relational tables.

The stored information falls into three groups. User Accounts hold the identity information needed for sign-in and for tying records to the right person. Uploaded File Records hold the outcome of each scan, including the file's metadata, its risk score, its threat classification, and the detection details that explain how the result was reached. These records are what feed the History page, the Reports page, and the dashboard's live feed. User Settings hold each person's scan preferences and their preferences for quarantine and notification behavior, so that the system acts the way the user has asked it to every time they sign in.

Communication between the application server and Firebase is authenticated using service-account credentials, meaning the application server proves its identity to the database before any data is read or written, and no anonymous or unauthenticated access is possible. In addition, all data-layer queries are scoped to the requesting user's identifier to prevent cross-account access to another user's files or scan history. This means that even if a user attempted to request a record belonging to someone else, the query itself would be limited to their own data. Together, these two controls protect both the database as a whole and each individual user's privacy within it.

4.4 Hardware and Software Requirements

The successful implementation and operation of TrustFile requires the following hardware and software resources.

4.4.1 Hardware Requirements

Table 4.1 presents the recommended hardware specifications for development and local use

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3	Intel Core i5 or higher
Memory (RAM)	4 GB	8 GB or higher
Storage	128 GB SSD	256 GB SSD
Monitor	1366 × 768 Resolution	Full HD Display
Internet Connection	10 Mbps	25 Mbps or higher

4.4.2 Software Requirements

Table 4.2 presents the software requirements for development and deployment.

Software	Purpose
Window 11/Ubuntu Linux	Development Operating System
Visual Studio Code	Source Code Editor
Python 3.x	Backend Language Runtime
Flask	Backend Web Framework
Firebase Realtime Database	Cloud Data Storage
Firebase Admin SDK	Server-Side Firebase Authentication
VirusTotal API	Third-Party Threat Intelligence
Jinja2/HTML/CSS/JavaScript	Frontend Templating and Styling

Gunicorn	Production WSGI Server
Render	CLoud Application Hosting
Google Chrome	Web Browser
GIT/Github	Version Control

The selection of these tools was based on their stability, active community support, and suitability for a security-focused web application requiring both real-time data synchronization and reliable third-party API integration.

4.5 Database Design

TrustFile's data layer is built on Firebase Realtime Database, a cloud-hosted NoSQL database that stores data as a single, hierarchical JSON tree rather than a set of relational tables. This choice was made deliberately: because the dashboard's real-time detection feed and monitor-status indicators depend on low-latency, push-based updates, a NoSQL store with native real-time synchronization better suited the application's interactive requirements than a traditional relational database would have. It also removed the need to provision and maintain a separate, always-on relational database server, which was impractical given the ephemeral filesystem of the target cloud hosting environment.

Rather than normalized tables joined by foreign keys, TrustFile's schema is organized into top-level nodes, each holding a collection of records keyed by a unique identifier. Application-level logic enforces referential relationships for example, filtering an uploaded_files node by a matching user_id in place of database-level foreign-key constraints.

Table 4.5.1 Users Table

Field	**Description**
uid	Unique User Identifier (node key)
username	Display / Login Username
email	Account Email Address
password	Hashed Password (PBKDF2)

Table 4.5.2 Uploaded_Files Table

Field	Description
id	Unique File Record Identifier
user_id	Owning User's UID
filename	Original Uploaded Filename
filepath	Server-Side Storage Path
upload_time	ISO 8601 Upload Timestamp
hash	SHA-256 File Hash
status	Pending/Safe/Threat/Quarantined
threat_level	Low/Medium/High/Critical
risk_score	Consolidated Risk Score(0-100)
entropy	Calculated Shannon Entropy
pattern_result	Detected Suspicious Patterns
signature_status	Detected Heuristic Signatures
risky_imports	Detected Risky Code Modules
ai_analysis	AI-Generated Explanation Text
advanced_heuristics	Full Heuristic Engine Result Object
detected_type	Identified File / Payload Type
confidence	Heuristic Engine Confidence Score
iocs	Extracted Indicators of Compromise
size	File Size (formatted)

Table 4.5.3 User_Settings Table

Field	Description
auto_scan_enabled	Whether uploads are scanned automatically
auto_scan_mode	Quick or Comprehensive scan default
scan_types	Default set of scan engines to run
notify_on_threat	Whether to notify on threat detection
notify_safe	Whether to notify on safe results

REFERENCE

[1] A. L. Abdon, "RATTATA: Ransomware analysis via automated impact assessment of Philippine organizations," Int. J. Intell. Syst. Appl. Eng., vol. 12, no. 4, p. 3430, 2024. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/6858

[2] Mheboobkhan, "Entropy analysis: A critical test for malware's," Medium, Feb. 13, 2019. [Online]. Available: https://whiteheart0.medium.com/entropy-analysis-a-critical-test-for-malwares-69939f5b8b1

[3] Ö. A. Aslan and R. Samet, "A comprehensive review on malware detection approaches," IEEE Access, vol. 8, pp. 6249–6271, 2020, doi: 10.1109/ACCESS.2019.2963724.

[4] TechClass, "Cybersecurity for all: Why every employee matters in data protection," TechClass Resources, Apr.25,2025.[Online].Available:https://www.techclass.com/resources/learning-and-development-articles/cybersecurity-for-non-tech-teams-why-everyone-plays-a-role-in-data-protection

[5] Wee Ki Joon, "Leveraging CNNs and entropy based feature selection to identify potential malware artifacts of interest," SANS Internet Storm Center Guest Diary, Mar. 26, 2025. [Online]. Available: https://isc.sans.edu/diary/31790

PHINMA UNIVERSITY OF ILOILO
College of Information Technology Education

CHAPTER V

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter brings the study to a close by presenting the summary of findings, the conclusions, and the recommendations drawn from the development and evaluation of TrustFile, a web-based file security platform that assessed the safety of digital files before users opened them. It begins by summarizing the results of the study, restating how the system's static analysis techniques, heuristic classification engine, VirusTotal threat intelligence lookup, and AI-generated explanation layer worked together to classify each file and explain the result in plain language. It then examines each of the study's objectives in turn to determine whether it was achieved, drawing on the testing and evaluation results presented in the previous chapter. From these findings, the chapter states the conclusions of the researchers about the effectiveness, usability, and educational value of the system for its intended users, such as offices, private businesses, and other organizations that regularly receive large volumes of files. Finally, it offers recommendations for the future enhancement of TrustFile and for its continued deployment, including improvements that could strengthen its detection capability and guidance for future researchers who may build on this work.

5.2 Summary of Findings

The primary objective of this study was to design, develop, and evaluate TrustFile: A File Security Platform Leveraging Multi-Engine Static Analysis and Explainable AI Detection, to help individuals and organizations assess the safety of files before opening them. Using the Agile Software Development Methodology and a three-tier architecture (built with HTML and CSS for the presentation layer, Python for the application layer, and Firebase for the data layer), together with the VirusTotal API v3 and Google Gemini 2.0, the system was constructed and subjected to evaluation. The findings corresponding to the specific objectives set out in Section 1.3.2 are summarized below:

- Web-Based File Upload and Scanning
    - TrustFile delivered file security checking through an ordinary browser, so users could upload a file and receive an assessment without installing any security software.
    - Every submitted file was examined before the user could access its contents, which supported the system's preventive purpose.
- Rule-Based Heuristic Classification with AI-Generated Explanation
    - The system successfully consolidated heuristic analysis, VirusTotal threat intelligence, and AI-generated explanation into a single, coherent scan result, reducing the need for users to interpret multiple disconnected security tools.
    - The heuristic engine weighed the combined static analysis and VirusTotal results through a defined set of rules and thresholds and assigned each file to one of five threat levels, from BENIGN to MALICIOUS.

    - Google Gemini 2.0 only explained the finished classification and its supporting indicators in plain language. It did not decide the result, which kept the classification consistent and made the explanation faithful to it.
- Entropy Analysis for Detecting Obfuscated Files
    - Shannon entropy analysis helped identify encrypted, compressed, or packed content that could hide a threat.
    - Entropy was treated as one indicator among several instead of a verdict on its own, which helped avoid wrongly flagging legitimate compressed files.
- External Malware Intelligence Integration
    - The VirusTotal API v3 extended detection beyond what the system could determine locally by providing the results of many antivirus engines.
    - Caching the results in Firebase allowed repeated lookups to be answered without sending the same request again.
- Clear Scan Results and Visual Reporting
    - The dashboard and reporting modules provided clear visual summaries, including severity breakdowns and 7-day trend charts, supporting at-a-glance awareness of a user's overall file safety.
    - The scan history kept a record of each file's threat level and detected indicators for later review.

The development and evaluation also produced the following additional findings:

- Reduced false positives. Iterative refinement of the heuristic engine reduced false-positive detections, notably through a string-literal-stripping step that prevented suspicious-looking comments or string content from being misread as executable threats.
- Proactive threat response. The real-time monitoring and auto-quarantine features enabled proactive handling of threats, automatically isolating files classified at the highest threat level without requiring manual user intervention.
- A data layer suited to the hosting environment. Moving the data layer from a relational database to Firebase Realtime Database resolved the deployment constraints of the cloud hosting environment's ephemeral filesystem, while also enabling real-time dashboard updates.

5.3 Conclusion

TrustFile shows that individuals and small organizations can get file-level threat visibility without the cost and complexity of enterprise security infrastructure. By combining local heuristic analysis, VirusTotal threat intelligence, and Google Gemini 2.0 plain-language explanations, the system lowers the technical barrier usually involved in reading malware scan results. The study met its core objective of bringing scattered detection methods into one workflow that a non-technical user can follow, with the heuristic engine deciding each file's threat level and the AI only explaining it.

The iterative Agile process also allowed the detection logic to be refined through hands-on testing instead of assumptions made at design time, such as the reduction of false positives through string-literal stripping. As an academic prototype built on static analysis and tested on a limited set of samples, TrustFile does not guarantee protection against every threat. It does, however, provide a practical foundation for transparent, user-friendly file security that can be extended in future work.

5.4 Recommendations

To ensure the continued relevance and efficacy of TrustFile, the following recommendations are proposed:

System Improvements

- Expand the heuristic engine's pattern library to cover additional scripting languages and packer and obfuscation techniques as they are identified.
- Integrate real-time monitoring of command-line execution to help identify potentially malicious activities that occur outside of uploaded file analysis.
- Add configurable notification channels (email or push notification) so that users can be alerted to detections without needing to have the dashboard open.
- Introduce two-factor authentication to further protect user accounts and their associated scan history.

Future Research Directions

- Explore integrating a trained machine learning classifier alongside the existing rule-based heuristic engine to improve detection of novel, previously unseen malware variants.
- Investigate sandboxed dynamic analysis, executing suspicious files in an isolated environment, to complement the current static analysis approach.
- Conduct a longitudinal study measuring detection accuracy and false-positive rate across a larger, more diverse file corpus over an extended period of real-world use.

Implementation Strategies

- Organizations should train staff to scan every unfamiliar file, especially email attachments and downloads, with TrustFile before opening or sharing it.
- Users should read the AI-generated explanations and not only the threat level, so they learn to recognize suspicious signs on their own.
- TrustFile should be used alongside an installed antivirus program, since it relies on static analysis and does not provide real-time protection.
- Files classified as HIGH RISK or MALICIOUS should not be opened and should be reported to IT personnel for review.

5.5 Impact of the Study

TrustFile contributes to the practical use of layered, multi-source malware detection in a resource-constrained setting. By combining local heuristic analysis, external threat intelligence from VirusTotal, and AI-generated

explanations in a single accessible web application, the project shows that useful security tooling can be delivered without the infrastructure that enterprise antivirus platforms usually require. Its plain-language explanations also help users understand why a file was flagged, which supports better security awareness. The system's detection accuracy remains limited by the coverage of its heuristic rules and by how completely VirusTotal's engines cover any given file. Even so, the results show the potential of accessible, cloud-hosted security tools to extend baseline protection to users who would otherwise have none.

5.6 Summary

In summary, TrustFile is a practical integration of heuristic malware analysis, third-party threat intelligence from VirusTotal, and AI-generated explanation from Google Gemini 2.0 within a modern, cloud-deployed web application. It shows how an iterative Agile development process, combined with a real-time NoSQL data layer suited to the constraints of cloud hosting, can produce a functional and extensible security tool. This work provides a foundation for future developers to build upon, particularly in extending detection coverage and introducing more sophisticated, learning-based analysis techniques.